Please update a copy of
scan-of-declaration-of-authorship.pdf

Bachelor Thesis

Ein systematischer Vergleich von Ansätzen zum "Student Course Timetable Problem" am Beispiel der Semesterplanung Studierender der Wirtschaftsuniversität Wien im Bachelorstudium

Philipp Scheer

Date of Birth: 28.01.2003
Student ID: 12242922

Subject Area: Information Business

Studienkennzahl: J123456789

Supervisor: Assoz.Prof PD Dr. Stefan Sobernig

Date of Submission: XX. XXX 2025

Department of Information Systems & Operations Management, Vienna University of Economics and Business, Welthandelsplatz 1, 1020 Vienna, Austria

Contents

1 Einleitung **9**
1.1 Motivation . . . 9
1.2 State of the Art . . . 10
1.3 Offene Probleme . . . 11
1.4 Einordnung und Umsetzung in dieser Arbeit . . . 11
1.5 Aufbau der Arbeit . . . 12
1.6 Überblick der Beiträge . . . 13
1.7 Artefakte . . . 14
1.8 Herausforderungen . . . 14

2 Hintergrund **16**
2.1 Problemstellung . . . 16
2.2 Zeittafelprobleme . . . 16
2.2.1 University Course Timetabling Problem (UCTTP) . . 17
2.2.2 Student Sectioning Problem (SSP) . . . 17
2.2.3 Single Student Scheduling . . . 18
2.3 Lösungsverfahren . . . 20
2.3.1 Brute-Force als langsames, exaktes Lösungsverfahren . 20
2.3.2 Integer Linear Programming als schnelles, exaktes Lösungsverfahren . . . 21
2.3.3 Heuristische Verfahren als schnelle, nicht exakte Lösungsverfahren . . . 22
2.4 Replikationsarbeit . . . 23
2.4.1 Definition und Arten der Replikation . . . 24
2.4.2 Anforderungen an eine wissenschaftliche Replikation . . 24

3 Zielsetzung und Abgrenzung **25**

4 Datenbereitstellung **27**
4.1 Datenstruktur . . . 27
4.2 Extraktionsprozess . . . 29
4.3 Deskriptive Analyse . . . 31

5 Analyse **35**
5.1 Implementierung des ILP Baseline Ansatzes . . . 35
5.1.1 Variablen, Parameter und Zielfunktion . . . 35
5.1.2 Nebenbedingungen (Constraints) . . . 37
5.1.3 CPU vs. GPU . . . 37
5.2 Implementierung der Hill-Climbing Algorithmen . . . 38

5.2.1 Hill-Climbing v1 38
5.2.2 Hill-Climbing v3 38
5.2.3 Suchstrategie 38
5.2.4 Abbruchkriterien 38
5.2.5 Zur Nicht-Implementierung von Hill-Climbing v2 . . . 39
5.3 Implementierung des Offering-Order Algorithmus 40
5.3.1 Sortierung 40
5.3.2 Suchstrategie 40
5.4 Szenariendefinition 42
5.4.1 Freier Tag 42
5.4.2 Beliebte Zeiten 42
5.4.3 Ähnlicher Problemstellungen 42
5.4.4 Vergleichbarkeit 43
5.5 Versuchsaufbau 44
5.5.1 Zielfunktion 44
5.5.2 Durchführung 45

6 Ergebnisse 48
6.1 Szenario 1: Keine Kurse an Montagen 48
6.2 Szenario 2: Keine Kurse an Freitagen 50
6.3 Szenario 3: Jeder Tag der Woche bis 9:45 frei 52
6.4 Szenario 4: Keine Kurse an Donnerstagen und Freitagen . . . 54
6.5 Szenario 5: Erhöhte Priorität (+90) für Nachmittags, negative Priorität (-90) für Abend 56
6.6 Szenario 6: Erhöhte Priorität (+70) für Vormittag, neutrale Priorität (0) für Nachmittag, negative Priorität (-80) für Abend 58
6.7 Szenario 7: Studierende müssen von 12:00 bis 13:00 Zeit für ein Mittagessen haben 60
6.8 Szenario 8: Studierende müssen mehr als einen Kurs pro Tag besuchen 62
6.9 Szenario 9: Studierende dürfen nicht mehr als drei Kurse pro Tag besuchen 64
6.10 Szenario 10: Alle Kurse können frei von Constraints gewählt werden 66
6.11 Szenario 11: 25%- 75% aller Kurse können nicht belegt werden. 68
6.12 Szenario 12: 1 - 7 Kurse sind vorgegeben 70
6.13 Szenario 13: Keine Kurse zwischem 6:00 morgens und 13:00 . 72
6.14 Szenario 14: Keine Kurse Montags, Dienstags und Mittwochs . 74

7 Diskussion **76**
7.1 Ergebnisse in der Ursprungsarbeit 76
7.2 Vergleich mit der Ursprungsarbeit 76
7.2.1 Lösungsqualität (Score) 76
7.2.2 Laufzeit und Performance 77
7.3 Eigene Ergebnisse im erweiterten Kontext 77
7.3.1 Der Einfluss moderner Solver (ILP) 77
7.3.2 GPU vs. CPU 78
7.4 Einschränkungen 78

8 Fazit **80**
8.1 Verwandte Arbeiten 80
8.2 Anwendungsfälle und Weiterentwicklung 80
8.3 Schlussworte 80

List of Figures

1 Hierarchie von Zeittafelproblemen (adaptiert aus [16]) 19
2 UML Klassendiagramm des Vorlesungsverzeichnis-Modells . . 28
3 Verteilung der *Offerings* pro *Course* 31
4 Verteilung der Kurse auf Wochentage 32
5 Verteilung der Kurs-Startzeiten (in Prozent) 33
6 Verteilung der Terminlänge der Buchungsstunden in 15-Minuten-Intervallen 34
7 Mathematische Formulierung des ILP-Modells zur Stundenplanoptimierung 36

List of Tables

1 Auszug des Ergebnisses der Datenbereitstellung 30

Abstract

Todo

1 Einleitung

Kaum eine Entscheidung im Alltag von Studierenden erscheint so simpel und erweist sich gleichzeitig so komplex wie die Erstellung des eigenen Stundenplans. Was auf den ersten Blick nach einer organisatorischen Routineaufgabe aussieht, entpuppt sich bei näherer Betrachtung als vielschichtiges Optimierungsproblem: Überschneidungen zwischen Lehrveranstaltungen, begrenzte Kursplätze und individuelle Präferenzen wie Arbeitszeiten und Lerngewohnheiten machen die Planung zu einer Herausforderung

In einer Zeit, in der viele Entscheidungsprozesse bereits durch Algorithmen unterstützt werden, stellt sich die Frage, ob auch die Stundenplanerstellung automatisiert und optimiert werden kann. Hier setzt die folgende Arbeit an: Sie untersucht, wie sich das *Student Schedulung Problem (SSP)* mithilfe von verschiedenen Optimierungsverfahren modellieren und lösen lässt.

1.1 Motivation

Studierende sehen sich im Studium mit einem umfangreichen Kursangebot konfrontiert und müssen ihre Stundenpläne meist händisch zusammenstellen. Dieser manuelle Prozess ist jedoch mehr als eine rein administrative Aufgabe; er ist eine Optimierungsentscheidung, bei der persönliche und akademische Rahmenbedingungen gegeneinander abgewogen werden müssen.

Studierende berichten häufig von Schwierigkeiten bei der Erstellung ihrer Semesterpläne [8, 17, 22]. Zu den zentralen Ursachen zählt insbesondere die Vereinbarkeit mit einem Nebenjob; so bevorzugen 81% der Befragten mindestens einen vollständig freien Wochentag [22]. Zudem geben 74% der Studierenden an, dass eine gleichmäßige Verteilung von Lehrveranstaltungen und Prüfungen für sie das wichtigste Kriterium bei der Stundenplanerstellung darstellt [11]. Weiterhin möchten 82% vermeiden, mehr als eine Prüfung pro Tag zu absolvieren [11]. Bezüglich der zeitlichen Präferenzen werden Mittagsstunden am häufigsten gewählt, gefolgt von frühen und späten Zeitslots. Rund 31% der Studierenden bevorzugen es, an bestimmten Tagen keine Prüfungen zu haben; besonders unpopulär sind Samstag, Sonntag, Freitag und Montag [11].

Obwohl sich diese Präferenzen grundsätzlich leicht modellieren lassen, stehen derzeit keine Programme zur Verfügung, die eine automatisierte Erstellung entsprechender Stundenpläne an der WU ermöglichen. Lediglich der Studienplaner der Österreichischen Hochschüler*innenschaft der WU [38] erlaubt die manuelle Kombination von Lehrveranstaltungen und Prüfungen im Hinblick auf Überschneidungsfreiheit, bietet jedoch keine Möglichkeit zur Optimierung nach individuellen Präferenzen.

1.2 State of the Art

Bestehende Lösungen zur Stundenplanung im universitären Umfeld lassen sich grob in zwei Kategorien unterteilen: institutionelle Planungssysteme und studentenzentrierte Hilfsmittel.

Auf institutioneller Ebene existieren ausgereifte Softwarelösungen wie UniTime [40] [19], ein quelloffenes System der Apereo Foundation, das Universitäten bei der Erstellung konfliktfreier Master-Stundenpläne unterstützt und dabei das Student Sectioning Problem berücksichtigt. Ähnliche kommerzielle Lösungen wie Infosilem oder Coursedog adressieren ebenfalls die Ressourcenplanung aus Sicht der Institution - also die in Kapitel 2.2.1 und 2.2.2 beschriebenen Problemebenen. Diese Systeme sind jedoch auf die Bedürfnisse von Administratoren ausgelegt und lösen das Problem der Stundenplanerstellung aus der Perspektive der gesamten Universität, nicht des einzelnen Studierenden.

Auf studentenzentrierter Ebene gibt es eine Reihe von Kalender- und Planungsapps wie MyStudyLife [18] oder den uAchieve Schedule Builder [39], die es Studierenden erlauben, bereits fixierte Lehrveranstaltungen zu verwalten und in einem persönlichen Kalender darzustellen. Diese Tools setzen jedoch voraus, dass der Studierende seine Kurswahl bereits getroffen hat - eine Optimierung nach individuellen Präferenzen findet nicht statt.

Im österreichischen Kontext bietet der Studienplaner der Österreichischen Hochschüler*innenschaft der WU [38] die Möglichkeit, Lehrveranstaltungen manuell auf Überschneidungsfreiheit zu prüfen. Auch dieses Tool optimiert jedoch nicht nach individuellen Präferenzen wie freien Tagen, bevorzugten Zeitslots oder der Vereinbarkeit mit einem Nebenjob.

Die beschriebene Lücke — ein automatisiertes, präferenzbasiertes System zur Erstellung individueller Stundenpläne auf Basis des tatsächlichen WU-Kursangebots — ist der Ausgangspunkt der vorliegenden Arbeit.

Anstatt jedoch eine vollständig neue Lösung zu entwickeln, stellt sich die Frage: Existieren bereits algorithmische Ansätze, die dieses Problem formal gelöst haben — und sind sie unter modernen Bedingungen noch relevant? Feldman und Golumbic [14] entwickelten 1990 genau solche Algorithmen für das Student Scheduling Problem. Ihre Arbeit wurde seither nie mit realen, aktuellen Universitätsdaten validiert. Genau hier setzt die vorliegende Arbeit an.

Sie knüpft dabei an die algorithmischen Grundlagen von Feldman und Golumbic [14] an und überprüft deren Anwendbarkeit unter heutigen Bedingungen — ein Ansatz, der in der Wissenschaft als Replikationsstudie bezeichnet wird: die Überprüfung, ob Ergebnisse einer früheren Arbeit unter veränderten Rahmenbedingungen standhalten. Was das konkret bedeutet

und wie diese Arbeit dabei vorgeht, wird in Kapitel 2.4 erläutert.

1.3 Offene Probleme

TODO: Was ist gemeint?

1.4 Einordnung und Umsetzung in dieser Arbeit

Die vorliegende Bachelorarbeit ordnet sich als Replikationsstudie der Arbeit von Feldman und Golumbic [14] ein. Sie kann als konzeptionelle Replikation mit Erweiterung angesehen werden.

Eine exakte Replikation — also die Reproduktion der ursprünglichen Ergebnisse mit denselben Daten und derselben Implementierung — wäre in diesem Fall grundsätzlich nicht möglich: Weder der originale Datensatz von Feldman und Golumbic noch eine Referenzimplementierung der Algorithmen sind verfügbar. Zudem lässt der in der Originalarbeit beschriebene Pseudocode an mehreren Stellen Interpretationsspielraum offen, was eine Nachbildung erschwert. Ziel dieser Arbeit ist es daher nicht, die exakten Ergebnisse von 1990 zu reproduzieren, sondern die Validität der vorgeschlagenen Lösungsansätze (Hill-Climbing und Offering-Order) in einem neuen Kontext zu überprüfen. Dies entspricht der von Burman et al. [5] beschriebenen validierenden Replikation, bei der geprüft wird, ob Ergebnisse robust gegenüber Veränderungen in Datensätzen und Zeiträumen sind.

Um die wissenschaftlichen Anforderungen an diese Replikation zu erfüllen, werden in dieser Arbeit folgende Maßnahmen getroffen:

- **Methodische Treue:** Die Algorithmen werden basierend auf der Beschreibung in der Originalarbeit von Feldman und Golumbic [14] exakt nachprogrammiert (*"re-implemented"*), um die methodische Vergleichbarkeit zu gewährleisten.

- **Neuer Datensatz:** Die Evaluation erfolgt anhand eines aktuellen, realen Datensatzes der Wirtschaftsuniversität Wien (Sommersemester 2025).

- **Erweiterte Baseline:** Da der ursprüngliche Vergleichsmaßstab (Brute Force) für die Komplexität des neuen Datensatzes nicht praktikabel ist, wird eine neue Baseline mittels Integer Linear Programming (ILP) eingeführt. Dies stellt eine methodische Modernisierung dar, wahrt aber das Ziel, die heuristischen Verfahren gegen eine optimale Lösung zu benchmarken.

- **Transparenz (Computational Reproducibility):** Im Sinne der "Ten Simple Rules" von Sandve et al. [25] werden alle Schritte der Datenextraktion und Analyse dokumentiert. Der Quellcode der Implementierungen ist, wie in der Arbeit referenziert, öffentlich einsehbar, um vollständige Nachvollziehbarkeit zu garantieren [27].

Durch dieses Vorgehen leistet die Arbeit einen Beitrag zur Überprüfung der Generalisierbarkeit historischer Optimierungsansätze unter modernen Rahmenbedingungen.

1.5 Aufbau der Arbeit

Die Arbeit gliedert sich in acht Kapitel, die aufeinander aufbauen und den Weg von der Problemstellung bis zur Diskussion der Ergebnisse nachvollziehbar machen.

Nach der Einleitung legt Kapitel 2 die theoretischen Grundlagen fest. Es führt in die Hierarchie der Zeittafelprobleme ein - vom University Course Timetabling Problem über das Student Sectioning Problem bis hin zum Single Student Scheduling - und verortet die vorliegende Arbeit innerhalb dieser Problemfamilie. Zusätzlich werden die wissenschaftlichen Anforderungen an eine Replikationsstudie erläutert, da die Arbeit als konzeptionelle Replikation der Studie von Feldman und Golumbic [14] einzuordnen ist.

Kapitel 3 präzisiert die Zielsetzung und grenzt den Forschungsumfang ab. Es wird klargestellt, welche Aspekte des Scheduling-Problems untersucht werden und welche - etwa die universitäre Ressourcenplanung oder die Entwicklung einer vollständigen Anwendung - bewusst ausgeklammert sind.

Kapitel 4 beschreibt die Datengrundlage der Arbeit. Es erläutert, wie die Vorlesungsdaten der Wirtschaftsuniversität Wien automatisiert aus dem Vorlesungsverzeichnis extrahiert wurden, und gibt anhand einer deskriptiven Analyse einen Überblick über die Struktur und Komplexität des Datensatzes - etwa die Verteilung der Lehrveranstaltungen auf Wochentage und Tageszeiten.

Kapitel 5 bildet den methodischen Kern der Arbeit. Es beschreibt die Implementierung der drei untersuchten Algorithmen: den ILP-Baseline-Ansatz (inklusive einer GPU-optimierten Variante), die beiden Hill-Climbing-Algorithmen (v1 und v3) sowie den Offering-Order-Algorithmus. Darüber hinaus werden die 14 Testszenarien definiert und der Versuchsaufbau inklusive der verwendeten Zielfunktion erläutert.

Die Ergebnisse der Benchmarks werden in Kapitel 6 szenarioweise präsentiert. Für jedes der 14 Szenarien werden Lösungsqualität, Laufzeit

und Speichernutzung der Algorithmen grafisch dargestellt und einander gegenübergestellt.

Kapitel 7 interpretiert die Ergebnisse und setzt sie in Bezug zur Originalarbeit von Feldman und Golumbic [14]. Es diskutiert, wo die Ergebnisse übereinstimmen, wo sie abweichen und welche Einschränkungen bei der Interpretation zu berücksichtigen sind.

Kapitel 8 schließt die Arbeit mit einem Fazit ab, das verwandte Arbeiten einordnet, mögliche Anwendungsfälle und Weiterentwicklungen skizziert und die wesentlichen Erkenntnisse zusammenfasst.

1.6 Überblick der Beiträge

Die vorliegende Bachelorarbeit leistet folgende wesentliche Beiträge:

- Die Algorithmen von Feldman und Golumbic [14] — Hill-Climbing v1, Hill-Climbing v3 und Offering-Order — wurden nachprogrammiert und mit einem aktuellen Datensatz der Wirtschaftsuniversität Wien (Hauptstudium Wirtschaftsinformatik, Sommersemester 2025) evaluiert. Die Ergebnisse der Ursprungsarbeit konnten bestätigt werden.

- **Überlegenheit der Hill-Climbing-Varianten:** In den meisten der 14 getesteten Szenarien erzielten Hill-Climbing v1 und v3 eine höhere Lösungsqualität (*Mark*) als der Offering-Order-Algorithmus. Insbesondere bei Szenarien mit vorgegebenen Kursen (Szenario 12) kann man die Schwäche der statischen Sortierung sehen: Der Algorithmus verfängt sich in lokalen Optima und erfüllt in manchen Fällen die Mindestanzahl an zu wählenden Lehrveranstaltungen nicht. Offering-Order war jedoch in allen Szenarien die schnellste Heuristik.

- **Effizienz moderner ILP-Solver:** Ein zentrales und unerwartetes Ergebnis dieser Arbeit ist, dass ein moderner Integer-Linear-Programming-Solver (CBC/Gurobi via Python-MIP) die heuristischen Algorithmen in der Laufzeit in fast allen Szenarien übertrifft. Lediglich in Szenario 8 ("Mehr als ein Kurs pro Tag") war der ILP-Ansatz langsamer als die Heuristiken.

- **GPU vs. CPU:** Der Vergleich einer GPU-optimierten ILP-Implementierung (NVIDIA RTX A5000, CUDA 12.4) mit der CPU-Variante zeigt, dass die GPU-Implementierung bei kleinen und mittleren Probleminstanzen langsamer ist als die CPU, jedoch bei steigender Komplexität aufholt.

1.7 Artefakte

Um die Nachvollziehbarkeit dieser Replikationsstudie zu gewährleisten, sind alle im Rahmen dieser Arbeit entstandenen Materialien öffentlich zugänglich. Das zentrale Repository der Arbeit ist unter folgendem Link erreichbar: [27].

Es umfasst die folgenden Artefakte:

- **Implementierungen:** Die Quellcodes aller implementierten Algorithmen: ILP (CPU) [28], ILP (GPU) [29], Hill-Climbing v1 [30], Hill-Climbing v3 [31] sowie Offering-Order [32].
- **Datensatz:** Die extrahierten Vorlesungsdaten der Wirtschaftsuniversität Wien für das Sommersemester 2025 als DataFrame [33], inklusive des Extraktionsskripts [34] (vgl. Kapitel 4.2).
- **Szenarien:** Die Konfigurationsdateien aller 14 Testszenarien, mit denen die in Kapitel 5.4 definierten Constraints exakt nachvollzogen werden können. [**?**]
- **Messdaten:** Die vollständigen Rohdaten der Benchmarks [35], inklusive aller 50 Messwiederholungen pro Algorithmus, Szenario und Schwierigkeitsgrad, sowie die Auswertungsskripte [36] zur Reproduktion der Grafiken aus Kapitel 6.

1.8 Herausforderungen

Die Umsetzung dieser Arbeit war mit einer Reihe von Herausforderungen verbunden, die im Folgenden kurz skizziert werden.

- **Unvollständiger Pseudocode:** Die zentrale Schwierigkeit war die exakte Nachprogrammierung der Algorithmen von Feldman und Golumbic [14]. Der in der Originalarbeit beschriebene Pseudocode lässt an mehreren Stellen Interpretationsspielraum offen, was für eine Replikationsstudie besondere Sorgfalt erfordert. Da keine Referenzimplementierungen oder aktuellen Codebeispiele verfügbar sind, mussten Unklarheiten durch eigene Interpretation und iteratives Testen aufgelöst werden. Dies betrifft insbesondere die Zielfunktion (*Mark*), deren exakte Berechnung ebenfalls nicht vollständig spezifiziert ist und daher auf Basis der Beschreibung im Paper rekonstruiert werden musste.
- **Rekursive Implementierung des Offering-Order-Algorithmus:** Der Offering-Order-Algorithmus basiert auf einer rekursiven

Backtracking-Strategie. Die korrekte Implementierung dieser Rekursion erforderte einige Iterationen.

- **Testumgebung mit benchexec:** Die Einrichtung der Testumgebung mittels benchexec [1] gestaltete sich aufwändiger als erwartet. Das Tool ist extrem umfangreich und eine Implementierung für die "einfache" Messung von Laufzeit und Speichernutzung erforderte einige Einarbeitungszeit.

- **Datenbeschaffung:** Der automatisierte Scraping-Prozess der Vorlesungsdaten aus dem Vorlesungsverzeichnis der WU (vgl. Kapitel 4.2) stellte eine Herausforderung dar, da keine offizielle API zur Verfügung steht und die HTML-Struktur der Webseite zuerst entsprechend analysiert werden musste.

2 Hintergrund

2.1 Problemstellung

Die Erstellung eines optimalen Semesterplans ist eine zentrale Herausforderung für Studierende. Das Angebot an Lehrveranstaltungen lässt sich nicht frei kombinieren, da auf zeitliche Konflikte, Erfüllung einer Mindestanzahl an ECTS pro Semester, Arbeitstätigkeit, etc. geachtet werden muss. Die Herausforderung besteht darin, aus diesem Pool an Lehrveranstaltungen eine zulässige und optimale Kombination zu wählen. Zulässig gilt eine Zeittafel, wenn keine zeitlichen Konflikte auftreten (keine *Hard Constraints* wurden verletzt). Optimal ist eine Zeittafel, wenn eine gegebene Zielfunktion maximal ist. Die Zielfunktion beinhaltet alle Präferenzen des Studierenden, die seine Lebensrealtät widerspiegeln sollen (*Soft Constraints*). Dazu zählen etwa die Berücksichtigung von Arbeitstätigkeiten, die durch Blockieren oder Priorisieren von gewissen Stunden in der Woche ausgedrückt werden, sowie die Favorisierung von Kursen, die durch eine Priorität ausgedrückt wird. Diese Problemstellung wird in der Literatur als Zeittafelproblem oder *Student Scheduling Problem (SSP)* bezeichnet.

2.2 Zeittafelprobleme

Zeittafelprobleme werden als Unterkategorie kombinatorischer Optimierungsprobleme gesehen, bei denen Aktivitäten (z.B. Lehrveranstaltungen, Prüfungen oder Schichten) unter Einhaltung von *Hard Constraints* und *Soft Constraints* auf Ressourcen wie Zeitfenster und Räume verteilt werden. Educational Timetabling ist wiederum eine Unterkategorie der Zeittafelprobleme, die sich speziell mit Optimierungsproblemen im Bildungskontext beschäftigt [7].

Innerhalb der Educational-Timetabling-Forschung hat sich ein Standard etabliert, der High-School-Timetabling, University Course Timetabling und Examination Timetabling als grundlegende Problemfamilien unterscheidet. Diese Probleme sind in der Regel NP-schwer, was bedeutet, dass mit wachsender Problemgröße und zunehmender Zahl an *Constraints* keine effizienten exakten Algorithmen bekannt sind und daher heuristische Ansätze angewandt werden. Carter und Laporte [6] zeigen anhand des Examination Timetabling, dass realistische Probleminstanzen typischerweise nur mit Heuristiken, die an das Problem angepasst sind, z.B. Greedy-Konstruktionen, Tabu Search oder Simulated Annealing, praktikabel lösbar sind, und prägten damit die algorithmische Behandlung von Zeittafelproblemen im universitären Umfeld [6].

Die, im weiteren Verlauf dieser Arbeit betrachtete Hierarchie vom University Course Timetabling über das Student Sectioning bis hin zum Single Student Scheduling lässt sich als zunehmende Individualisierung innerhalb der Problemfamilie der Educational-Timetabling-Probleme interpretieren: Während beim University Course Timetabling institutionelle Zielfunktionen wie Auslastung und Konfliktfreiheit auf Ebene der gesamten Organisation im Vordergrund stehen, rücken beim Student Sectioning kohorten- und gruppenspezifische *Constraints* in den Fokus. Single Student Scheduling ist schließlich die feinste Granularität, in der die Zielfunktion auf die Nutzenmaximierung eines einzelnen Studierenden ausgelegt ist, die sich durch persönliche Präferenzen (z.B. Mindest-ECTS pro Semester, Freie Tage, etc.) ergibt und damit einen studentenzentrierten Spezialfall der allgemeinen Zeittafel- und Scheduling-Probleme bildet [24] [19].

2.2.1 University Course Timetabling Problem (UCTTP)

Auf der obersten Ebene steht das University Course Timetabling Problem (UCTTP). Dieses Problem befasst sich mit der Erstellung des "Master-Stundenplans". Die Kernaufgabe besteht darin, eine Menge von Lehrveranstaltungen (Events) einer Menge von Zeitfenstern und Räumen zuzuordnen [26]. Dabei müssen Ressourcenbeschränkungen (z. B. Raumgrößen) und *Hard Constraints*, wie die Verfügbarkeit von Dozenten, berücksichtigt werden.

Carter und Laporte beschreiben dieses Problem als NP-hart [6]. Das bedeutet, dass mit steigender Anzahl an Veranstaltungen und Restriktionen der Rechenaufwand für eine exakte Lösung exponentiell wächst. Das Ziel des UCTTP ist hauptsächlich die Machbarkeit und Effizienz aus Sicht der Institution: Es muss sichergestellt werden, dass alle Kurse stattfinden können, ohne dass Dozenten oder Räume doppelt belegt werden. Individuelle Studierendenpräferenzen spielen auf dieser Ebene oft nur eine untergeordnete Rolle oder werden aggregiert betrachtet.

2.2.2 Student Sectioning Problem (SSP)

Sobald der Master-Stundenplan fixiert ist (die Veranstaltungen haben feste Zeiten und Räume), folgt die nächste Ebene: das Student Sectioning Problem (SSP). Hier verlagert sich der Fokus von der Ressourcenzuordnung (Wann findet der Kurs statt?) zur Belegung (Wer besucht welchen Kurs?).

Das SSP befasst sich mit der Verteilung der Studierenden auf Übungsgruppen oder Parallelveranstaltungen (Sections) eines Kurses [19]. Ein klassisches Beispiel ist ein Modul, das aus einer zentralen Vorlesung und zehn

verschiedenen Labor-Terminen besteht. Das Ziel ist es, alle Studierenden so auf die Labore zu verteilen, dass Überschneidungen vermieden und Kapazitätsgrenzen eingehalten werden. Müller et al. unterscheiden hierbei zwei Ansätze [19]:

1. **Batch Sectioning:** Alle Studierendenwünsche werden gesammelt und global optimiert. Das maximiert die Fairness und die Gesamtauslastung, findet aber meist vor Semesterbeginn statt.

2. **Online Sectioning:** Die Zuteilung erfolgt sequenziell in Echtzeit (z. B. während der Anmeldephase via LPIS im Kontext der WU). Dies entspricht einem "Greedy"-Ansatz, bei dem für späte Anmeldungen oft nur suboptimale Restplätze vergeben werden können.

2.2.3 Single Student Scheduling

Die granularste Ebene der Hierarchie — und der Fokus dieser Bachelorarbeit — ist das Single Student Scheduling. Während das SSP versucht, eine Menge von Studierenden global auf Kurse aufzuteilen, betrachtet das Single Student Scheduling das Problem ausschließlich aus der Perspektive eines einzelnen Studierenden.

Die Fragestellung lautet hier nicht: "Wie fülle ich die Kursräume optimal?", sondern: "Welche Kombination aus den verfügbaren Kursangeboten maximiert meinen persönlichen Nutzen?" [13].

Obwohl der Suchraum für einen einzelnen Studierenden deutlich kleiner ist als für die gesamte Universität, bleibt die Komplexität hoch. Da ein Studierender oft aus hunderten möglichen Kombinationen von Vorlesungen und Übungen wählen kann, die unterschiedliche zeitliche Attribute und Abhängigkeiten aufweisen, handelt es sich auch hier um ein kombinatorisches Optimierungsproblem. Die Herausforderung besteht darin, harte Restriktionen (keine zeitlichen Überschneidungen) zu erfüllen und gleichzeitig weiche Restriktionen (z. B. "keine Kurse am Freitag", "kompakter Stundenplan") zu optimieren.

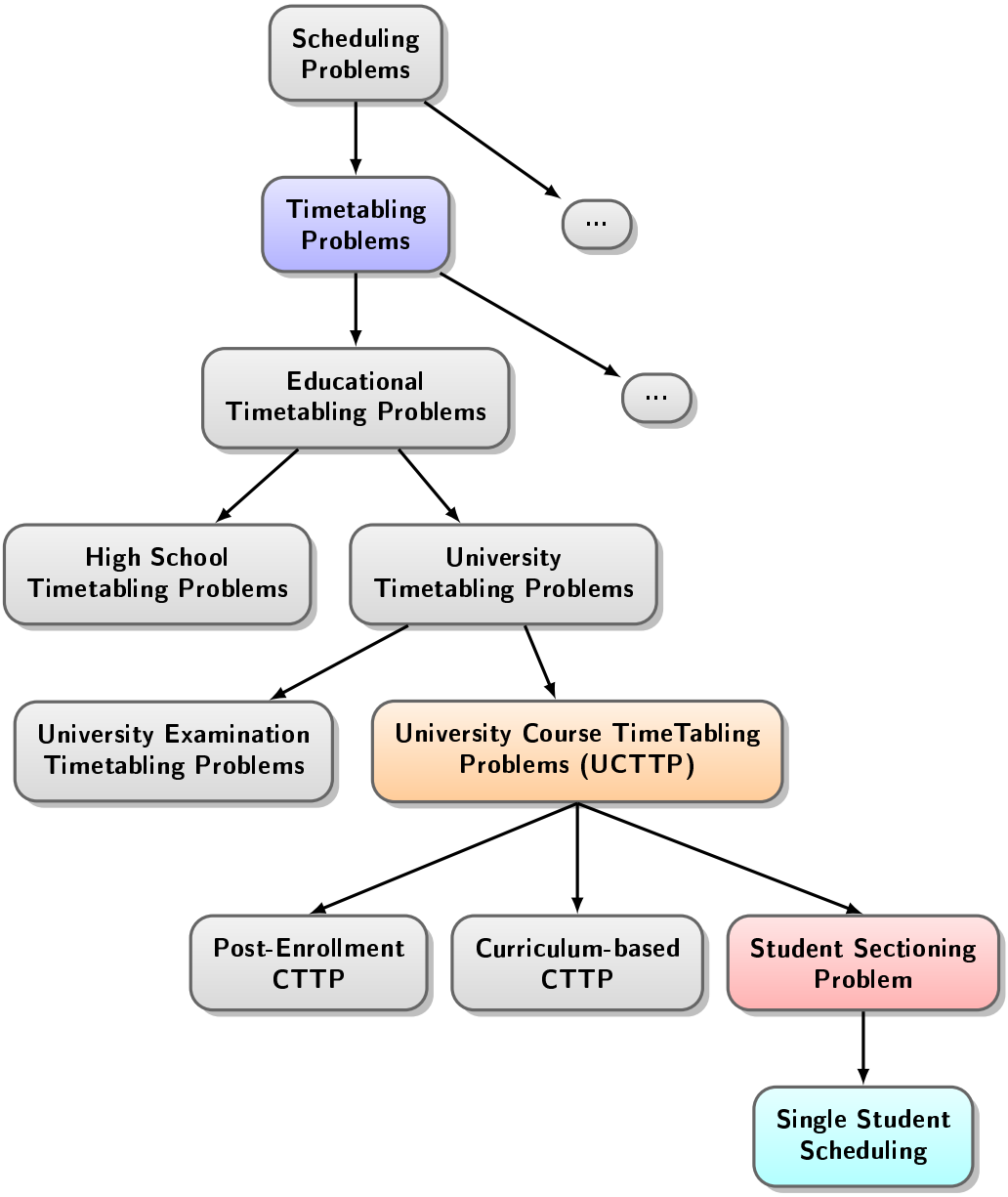


Figure 1: Hierarchie von Zeittafelproblemen (adaptiert aus [16])

2.3 Lösungsverfahren

2.3.1 Brute-Force als langsames, exaktes Lösungsverfahren

Der einfachste Ansatz zur exakten Lösung des Student Scheduling Problems ist es, alle möglichen Stundenplankombinationen mit der Zielfunktion zu bewerten. Das globale Optimum ist damit garantiert auffindbar, da keine Kombination ausgelassen wird.

Der Nachteil dieses Ansatzes liegt in der Komplexität: Die Anzahl der zu prüfenden Kombinationen wächst exponentiell mit der Problemgröße. Feldman und Golumbic nutzten Brute-Force in ihrer Arbeit von 1990 als Baseline, was für die dort betrachteten kleinen Probleminstanzen ("30 courses", was als 30 *Offerings* interpretiert wurde [14]) praktikabel war. Für den WU-Datensatz dieser Arbeit mit 381 *Offerings* und 28 *Courses* wäre ein Brute-Force-Ansatz rechnerisch nicht mehr durchführbar — die Anzahl möglicher Kombinationen übersteigt jede akzeptable Laufzeit bei weitem.

Die Frage, wie ein exaktes Verfahren dennoch für größere Probleminstanzen eingesetzt werden kann, beantwortet ein Ansatz, der seit den 1990er Jahren erheblich an praktischer Bedeutung gewonnen hat: Integer Linear Programming [2] [20].

2.3.2 Integer Linear Programming als schnelles, exaktes Lösungsverfahren

Integer Linear Programming (ILP) ist ein mathematisches Optimierungsverfahren, das auf linearer Programmierung basiert. Zusätzlich müssen einige oder alle Entscheidungsvariablen ganzzahlig sein. Für kombinatorische Optimierungsprobleme wie das Student Scheduling Problem ist diese Eigenschaft wichtig, da Entscheidungen wie "Kurs wird belegt" oder "Kurs wird nicht belegt" von Natur aus binär sind.

Während ILP als Verfahren bereits in den 1950er Jahren entwickelt wurde, war seine praktische Anwendbarkeit auf reale Probleminstanzen lange Zeit durch die verfügbare Rechenleistung stark eingeschränkt. Feldman und Golumbic griffen in ihrer Arbeit von 1990 daher auf Brute-Force als Baseline zurück. Für kleine Probleminstanzen, wie sie 1990 untersucht wurden, war dies praktikabel. Für die Größenordnung des WU-Datensatzes mit 381 Offerings und 28 Courses wäre ein Brute-Force-Ansatz jedoch rechnerisch nicht mehr durchführbar [2] [20].

Seit den 1990er Jahren haben sich ILP-Solver durch verschiedenste Fortschritte — insbesondere Branch-and-Bound, Branch-and-Cut sowie verbesserte Präprozessierungsverfahren — und die gestiegene Rechenleistung moderner Hardware zu effizienten Werkzeugen entwickelt. Moderne Solver wie CBC oder Gurobi sind in der Lage, kombinatorische Probleme, die für Brute-Force zu komplex sind, in akzeptabler Zeit exakt zu lösen. In dieser Arbeit wird ILP daher als zeitgemäße Baseline eingesetzt, die dieselbe Rolle übernimmt wie Brute-Force in der Ursprungsarbeit: die Ermittlung des nachweislich optimalen Stundenplans, gegen den die heuristischen Verfahren bewertet werden [10].

Trotz der enormen Fortschritte moderner ILP-Solver gibt es Problemklassen, bei denen auch exakte Verfahren an ihre Grenzen stoßen — etwa wenn die Problemgröße weiter wächst, Echtzeit-Anforderungen bestehen oder die Modellierung als ILP nicht praktikabel ist. In solchen Fällen bieten heuristische Verfahren eine Alternative.

2.3.3 Heuristische Verfahren als schnelle, nicht exakte Lösungsverfahren

Heuristische Verfahren verzichten auf die Garantie, das globale Optimum zu finden, und tauschen diese Garantie gegen deutlich kürzere Laufzeiten ein. Anstatt den gesamten Lösungsraum systematisch zu durchsuchen, navigieren sie durch den Suchraum anhand von problemspezifischen Faustregeln oder lokalen Verbesserungsschritten — mit dem Ziel, in akzeptabler Zeit eine möglichst gute, wenn auch nicht notwendigerweise optimale Lösung zu finden. In der Timetabling-Literatur haben sich dabei zwei grundlegende Klassen etabliert: Greedy-Verfahren und Lokale Suchheuristiken.

Greedy-Konstruktionsheuristiken bauen eine Lösung von Grund auf schrittweise auf, indem sie bei jedem Schritt lokal die beste verfügbare Entscheidung treffen — ohne spätere Schritte zu berücksichtigen. Der Vorteil: Sie sind einfach und schnell; der Nachteil ist, dass frühe Entscheidungen nicht revidiert werden und so zu suboptimalen Gesamtlösungen führen können (lokale Optima). Eine verbreitete Erweiterung ist das Backtracking, das lokalen Optima nachträglich auflöst. Der Offering-Order-Algorithmus von Feldman und Golumbic [14] folgt genau diesem Prinzip: Er sortiert Offerings nach ihrer Bewertung und wählt sie sequenziell aus, ergänzt um eine Backtracking-Strategie für den Fall von Konflikten. Eine weitere bekannte Greedy-Konstruktionsheuristik im Timetabling-Kontext ist der Graph-Coloring-Ansatz [6], bei dem Kurse als Knoten eines Graphen modelliert werden und Konflikte als Kanten — das Stundenplanproblem wird damit auf ein Färbungsproblem reduziert.

Lokale Suchheuristiken hingegen starten von einer bestehenden, oft greedy konstruierten Lösung und verbessern sie iterativ durch kleine, gezielte Änderungen. Sie sind in der Lage, initial schlechte Lösungen schrittweise zu verbessern, laufen jedoch ebenfalls Gefahr, in lokalen Optima stecken zu bleiben. Fortgeschrittenere lokale Suchheuristiken versuchen, das Problem lokaler Optima zu umgehen. Tabu Search [8] führt eine Verbotsliste kürzlich besuchter Lösungen, um Zyklen zu vermeiden und den Suchraum breiter zu erkunden. Simulated Annealing [7] erlaubt mit einer zu Beginn hohen, schrittweise sinkenden Wahrscheinlichkeit auch verschlechternde Schritte, um lokalen Optima zu entkommen. Evolutionäre Algorithmen wie Genetische Algorithmen [8] kombinieren mehrere Lösungen durch Crossover- und Mutationsoperatoren und simulieren damit einen evolutionären Selektionsprozess.

Die vorliegende Arbeit untersucht mit Hill-Climbing und Offering-Order zwei der grundlegendsten Vertreter von Greedy-Konstruktionsheuristiken und prüft, ob ihre Ergebnisse unter heutigen Bedingungen Bestand haben — und ob ILP-Solver diese Heuristiken mittlerweile auch in der Laufzeit

übertreffen.

Die Frage, unter welchen wissenschaftlichen Anforderungen ein solcher Vergleich durchgeführt werden soll und was es bedeutet, Ergebnisse einer früheren Arbeit unter neuen Bedingungen zu überprüfen, wird im folgenden Kapitel geklärt.

2.4 Replikationsarbeit

Unter einer Replikationsarbeit versteht man den Versuch, Ergebnisse einer früheren Studie durch eine Wiederholung mit demselben oder einem sehr ähnlichen Design zu bestätigen oder zu widerlegen. Replikationsarbeiten gelten als "cornerstone of the scientific method" [37], weil oft wiederholte, konsistente Ergebnisse das Vertrauen in die wissenschaftliche Aussagekraft stärken. Ziel solcher Arbeiten ist es, die Verlässlichkeit (Reliabilität) und Gültigkeit (Validität) von wissenschaftlichen Erkenntnissen zu prüfen und sicherzustellen, dass veröffentlichte Ergebnisse nicht auf Zufall oder Fehler der ursprünglichen Arbeit beruhen [23].

Reliabilität von Ergebnisse ist gegeben, wenn die Messung bei Wiederholung unter gleichen Bedingungen stabile Werte liefert. Zentrale Aspekte hierbei sind unter anderem:

- Test-Retest-Reliabilität (Konsistenz über Zeit)
- Interne Konsistenz (Kohärenz von Messelementen)
- Interrater-Reliabilität (Übereinstimmung zwischen unterschiedlichen Beobachtern)

[15]

Validität ist gegeben, wenn die Messungen auch wirklich das gemessen haben, was gemessen werden sollte. Eine invalide Messung liegt beispielsweise vor, wenn die Mathekompetenz gemessen werden soll, die Textaufgaben allerdings zu kompliziert formuliert sind, sodass es mehr auf Sprachverständnis als Mathematik ankommt.

Hohe Validität setzt in der Regel ausreichende Reliabilität voraus, geht aber darüber hinaus, indem sie fordert, dass die Messwerte inhaltlich sinnvoll interpretierbar sind [9].

In computergestützter Forschung wird unter Reproduzierbarkeit zusätlich die Möglichkeit verstanden, die Ergebnisse auf Basis der zugrunde liegenden Daten und Algorithmen exakt nachzuvollziehen.Sandve et al. formulierten hierfür zehn Prinzipen ("Ten Simple Rules for Reproducible Computational Research") [25], die von der Dokumentation aller Auswertungsschritte bis

zur öffentlichen Zugänglichkeit von Code und Ergebnissen reichen und so die Überprüfbarkeit sicherstellen.

2.4.1 Definition und Arten der Replikation

In der Literatur wird häufig zwischen verschiedenen Arten der Replikation unterschieden, wobei die Terminologie je nach Disziplin variiert. Ein zentraler Unterschied besteht zwischen der "exakten" oder "nahen" Replikation (Close Replication) und der "konzeptionellen" Replikation.

- Bei der **exakten Replikation** wird versucht, die Methoden und Prozeduren der Originalstudie so exakt wie möglich zu reproduzieren. Das Ziel ist es, idealerweise nur jene Unterschiede zuzulassen, die unvermeidbar sind (z. B. ein anderer Zeitpunkt oder eine andere Stichprobe aus derselben Population).

- **Konzeptionelle Replikationen und Erweiterungen** überprüfen die Robustheit der Ergebnisse unter veränderten Rahmenbedingungen, beispielsweise durch die Verwendung neuer Datensätze, variierter Parameter oder alternativer Schätzverfahren. Bonett [3] bezeichnet dies auch als *comparable follow-up study*, bei der zwar das gleiche Design und die gleichen Messskalen verwendet werden, aber die Population oder der Kontext variieren kann.

2.4.2 Anforderungen an eine wissenschaftliche Replikation

Damit eine Replikationsarbeit wissenschaftlichen Standards genügt, müssen bestimmte Anforderungen erfüllt sein. Brandt et al. [4] entwickelten hierfür das *Replication Recipe*, welches unter anderem folgende Kriterien hervorhebt:

- **Exakte Definition der zu replizierenden Methoden:** Die Methoden der Originalstudie müssen sorgfältig analysiert und so getreu wie möglich nachgebildet werden.

- **Hohe statistische Aussagekraft:** Die Replikationsstudie muss über eine ausreichende Stichprobengröße verfügen, um Effekte zuverlässig nachweisen zu können.

- **Transparenz und vollständige Dokumentation:** Alle Details zur Durchführung, inklusive Code und Daten, sollten verfügbar gemacht werden, um eine Überprüfung durch Dritte zu ermöglichen.

- **Kritischer Vergleich:** Die Ergebnisse der Replikation müssen statistisch fundiert mit denen der Originalstudie verglichen werden, anstatt sich rein auf Signifikanztests zu verlassen.

Im Bereich der Management-Mathematik und Optimierung wird außerdem gefordert, dass Algorithmen so spezifiziert sind, dass unabhängige Forscher dieselben Ergebnisse erzielen können. Burman et al. [5] betonen weiters die Notwendigkeit, dass der replizierende Forscher unabhängig von den ursprünglichen Autoren ist.

Die vorliegende Bachelorarbeit ordnet sich als Replikationsstudie der Arbeit von Feldman und Golumbic [14] ein. Sie kann als konzeptionelle Replikation mit Erweiterung klassifiziert werden.

3 Zielsetzung und Abgrenzung

Die vorliegende Bachelorarbeit hat zum Ziel, die beschriebene Problemstellung unter den aktuellen technischen Rahmenbedingungen mithilfe verschiedener Lösungsansätze zu untersuchen, diese anhand eines aktuellen Datensatzes zu validieren und anschließend einem Leistungsvergleich zu unterziehen.

1. **Validierung:** Im Mittelpunkt steht die exakte Nachprogrammierung der Algorithmen von Feldman und Golumbic [14]. Es soll überprüft werden, ob diese Verfahren unter modernen Rechenbedingungen und anhand realer Daten aus dem Vorlesungsverzeichnis der Wirtschaftsuniversität Wien in der Lage sind, einen optimalen Semesterplan zu erstellen.

2. **Vergleichende Leistungsanalyse:** Die implementierten Algorithmen werden hinsichtlich der Qualität der erreichten Lösungen (*Score*), ihrer Geschwindigkeit sowie ihres Speicherbedarfs systematisch miteinander verglichen.

Folgende Aspekte sind explizit vom Forschungsumfang ausgeschlossen:

1. Das Problem wird ausschließlich als Single Student Scheduling für einen einzelnen Studierenden gelöst. Es wird **keine Rücksicht auf andere Studierende**, die maximale Kapazität von Räumen, die Verfügbarkeit von Professoren oder die allgemeine universitäre Ressourcenplanung genommen.

2. Die Planung beschränkt sich auf **ein einzelnes Semester** des Hauptstudium Wirtschaftsinformatik.

3. Die Entwicklung einer **voll funktionsfähigen Applikation** zur Dateneingabe, Speicherung von Planungen, oder komplexer Visualisierung wird ausgeschlossen. Das Ergebnis des Programms wird durch ein computerlesbares Format (JSON) ausgegeben.

4 Datenbereitstellung

Um einen möglichst aktuellen Datensatz zu erzeugen, wurden die Vorlesungsdaten aus dem Vorlesungsverzeichnis der Wirtschaftsuniversität Wien [12] automatisiert ausgelesen (siehe Kapitel 4.2). Jedem Kurs (*Offering*) ist eine Lehrveranstaltungs-ID zugewiesen, welche durch eine Zahl von 1 bis 9999 dargestellt wird. Weiters gehört jeder Kurs einem Planpunkt (*Course*) an.

4.1 Datenstruktur

Der Datensatz des Vorlesungsverzeichnisses der Wirtschaftsuniversität Wien für das Sommersemester 2025 bildet die Grundlage für die vorliegende Arbeit. Die Daten wurden automatisiert unter Verwendung des beschriebenen Prozesses 1 extrahiert.

Die Modellierung des Studiums an der WU basiert auf zwei zentralen Entitäten, die für das Verständnis der Optimierungsalgorithmen wichtig sind:

- **Course (Planpunkt):** Ein *Course* repräsentiert ein zu absolvierendes Modul, das mit einer festen ECTS-Anzahl assoziiert ist. Aus den verfügbaren *Offerings* eines Planpunkts darf maximal eines ausgewählt werden.

- **Offering (Lehrveranstaltung):** Ein *Offering* ist eine Lehrveranstaltung, die einem Planpunkt angehört. Jedes Offering besitzt eine eindeutige ID (von $1 - 9999$) und ist mit konkreten Termin- und Zeitangaben hinterlegt, die für die Prüfung von zeitlichen Konflikten verwendet wird.

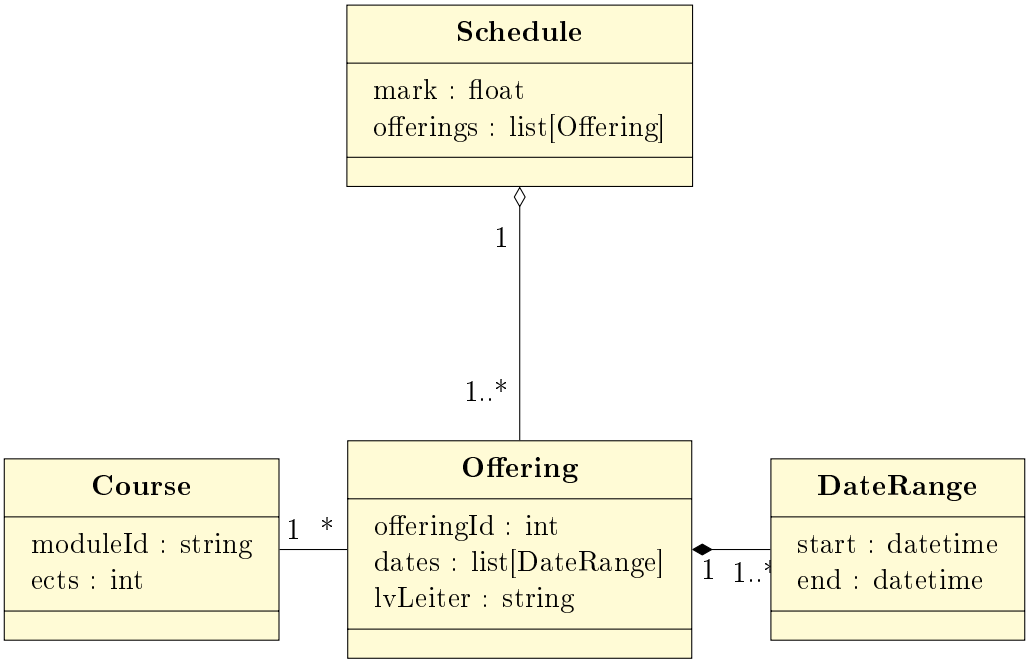


Figure 2: UML Klassendiagramm des Vorlesungsverzeichnis-Modells

4.2 Extraktionsprozess

Der Prozess der automatisierten Extraktion wurde folgendermaßen realisiert:

Algorithm 1 Datenextraktion aus Vorlesungsverzeichnis

1: $\mathit{offerings} \leftarrow \emptyset$
2: $\mathit{courses} \leftarrow \emptyset$
3: **for** $\mathit{id} \leftarrow 1, 9999$ **do**
4: $\quad \mathit{url} \leftarrow \textsc{CraftCourseURL}(\mathit{id})$
5: $\quad \mathit{page} \leftarrow \textsc{HttpGet}(\mathit{url})$
6: $\quad$ **if** $\textsc{Contains}(\mathit{page}\text{"}\mathit{Keine\ Lehrveranstaltung\ gefunden}\text{"})$ **then**
7: $\quad\quad$ **continue**
8: $\quad$ **end if**
9: $\quad (\mathit{course}, \mathit{dates}) \leftarrow \textsc{ExtractDates}(\mathit{page})$
10: $\quad$ **if** $\mathit{course} \notin \mathrm{dom}(\mathit{courses})$ **then**
11: $\quad\quad \mathit{ects} \leftarrow \textsc{ExtractECTSFromCourse}(\mathit{course})$
12: $\quad\quad \mathit{courses} \leftarrow \mathit{courses} \cup \{(\mathit{course}, \mathit{ects})\}$
13: $\quad$ **end if**
14: $\quad \mathit{offerings} \leftarrow \mathit{offerings} \cup \{(\mathit{id}, \mathit{dates}, \mathit{courses}_{\mathit{course}})\}$
15: **end for**

Das Ergebnis sieht wie folgt aus:

offeringId	dates	lvLeiter	moduleIds	ects
5877	[{"start": "2025-03-11 09:00", "end": "2025-03-11 11:00"}, ...]	Assoz.Prof PD Dr. Sabrina Kirrane	6590	5
6427	[{"start": "2025-03-13 16:30", "end": "2025-03-13 20:30"}, ...]	Dr. Robert Kosik	5160	6
4676	[{"start": "2025-05-06 14:30", "end": "2025-05-06 17:00"}, ...]	Dr. Stefan Treitl	6012	4
6295	[{"start": "2025-03-06 08:00", "end": "2025-03-06 12:00"}, ...]	Anna-Lena Klug, MSc. MSc.	5155	8
5745	[{"start": "2025-03-10 18:00", "end": "2025-03-10 20:00"}, ...]	Dr. Reinhard Zuba	5108	4
6258	[{"start": "2025-04-01 14:00", "end": "2025-04-01 19:00"}, ...]	Dr. Nikolaus Obwegeser	5162	3
5944	[{"start": "2025-05-08 09:30", "end": "2025-05-08 12:00"}, ...]	Univ.Prof. Dr. Jonas Puck	5107	4
5724	[{"start": "2025-03-10 16:00", "end": "2025-03-10 20:00"}, ...]	Alexander Oberreiter, M.Sc.	5105	8
5752	[{"start": "2025-05-06 12:00", "end": "2025-05-06 14:30"}, ...]	Univ.Prof. Dipl.Wirtsch.-Math.Dr. Birgit Rudloff	6023	4
6354	[{"start": "2025-05-06 15:30", "end": "2025-05-06 18:00"}, ...]	Zhenyi Wang, M.Sc.	5059	4
6447	[{"start": "2025-03-13 12:30", "end": "2025-03-13 16:30"}, ...]	Anita Neumannova, PhD, MSc (WU), MIM (CEMS)	5161	4
5329	[{"start": "2025-09-01 14:00", "end": "2025-09-01 17:00"}, ...]	Dr. Tingmingke Lu	5056	4
6317	[{"start": "2025-03-05 14:00", "end": "2025-03-05 18:00"}, ...]	Dipl.-Wirt.-Ing.(FH) Mark Nicolas Grünsteidl	5158	8
6130	[{"start": "2025-05-06 09:30", "end": "2025-05-06 12:00"}, ...]	Jana Hlavinova, Ph.D.	6024	4
6182	[{"start": "2025-03-11 10:00", "end": "2025-03-11 12:00"}, ...]	Dr. Sonja Sperber	5136	3
6089	[{"start": "2025-09-10 09:00", "end": "2025-09-10 13:00"}, ...]	Dr. Nikolai Neumayer	5106	4
4086	[{"start": "2025-05-06 14:30", "end": "2025-05-06 17:00"}, ...]	Vanessa Kofler, LL.M. (WU)	5109	4
5762	[{"start": "2025-03-10 16:30", "end": "2025-03-10 18:00"}, ...]	Univ.Prof. Jonas Bunte, Ph.D.	5117	4

Table 1: Auszug des Ergebnisses der Datenbereitstellung

4.3 Deskriptive Analyse

Um die Komplexität des Datensatzes und des Optimierungsproblems zu quantifizieren, wurde eine deskriptive Analyse durchgeführt. In der Analyse wurden nur *Offerings* aus dem Hauptstudium Wirtschaftsinformatik des Bachelor Wirtschafts- und Sozialwissenschaften im Sommersemester 2025 berücksichtigt.

Der vollständige Datensatz besteht aus insgesamt 381 *Offerings* ($|\mathcal{O}|$) und 28 *Courses*, woraus sich eine durchschnittliche Anzahl von 13.6 *Offerings* pro *Course* ergibt. Eine genauere Analyse zeigt, dass die *Offerings* sehr ungleichmäßig verteilt sind: Während die Mehrheit der *Courses* (über 50 %) zwei oder drei Angebote aufweist, gibt es einzelne Ausreißer mit bis zu 44 Angeboten. Diese starke Rechtsschiefe der Verteilung zeigt, dass der statistische Mittelwert durch wenige, sehr angebotsintensive *Courses* nach oben verzerrt wird, während der Median deutlich unter dem Durchschnitt liegt.

Grafik 3 zeigt die Verteilung der *Offerings* auf die *Courses*.

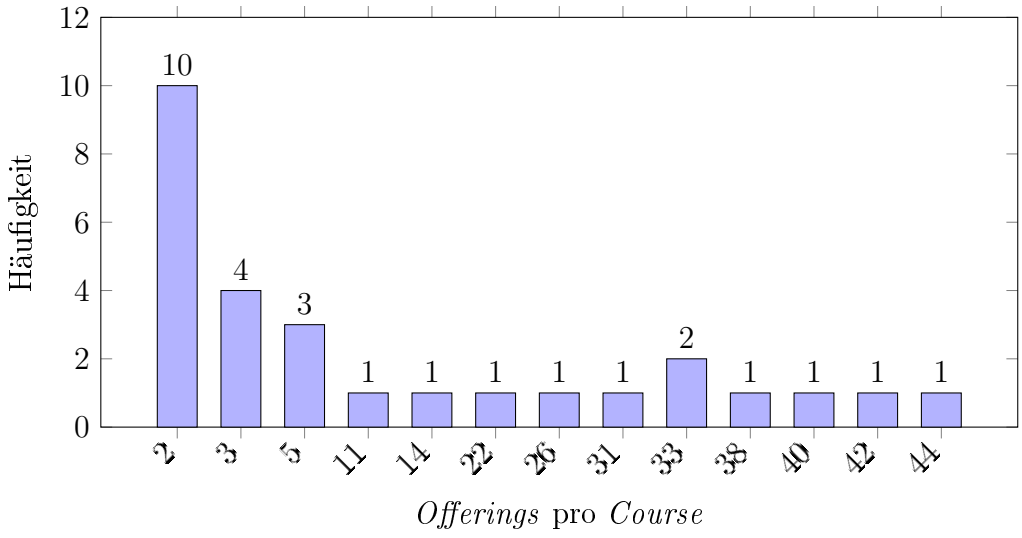


Figure 3: Verteilung der *Offerings* pro *Course*

Ein wesentlicher Faktor für die Schwierigkeit der Stundenplanerstellung ist die zeitliche Ungleichverteilung der angebotenen Kurse. Tabelle 4 zeigt die Verteilung der Lehrveranstaltungen über die Wochentage.

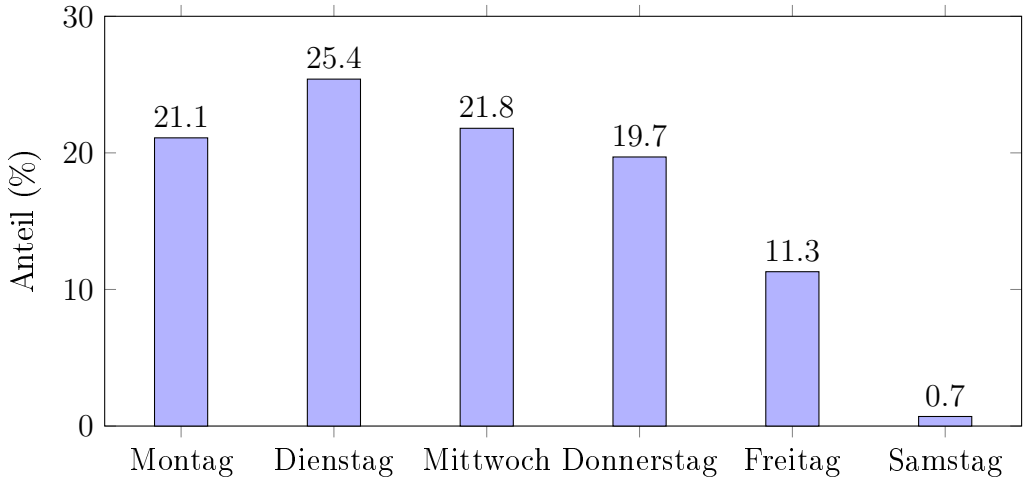


Figure 4: Verteilung der Kurse auf Wochentage

Wie aus den Daten ersichtlich wird, konzentriert sich der Großteil des Lehrangebots (ca. 88 %) auf die Kernzeit von Montag bis Donnerstag, während der Freitag mit 11.3 % bereits deutlich seltener belegt ist. Samstage spielen im regulären Lehrbetrieb nahezu keine Rolle. Diese ungleiche Verteilung verschärft die Problematik zeitlicher Überschneidungen ($\mathcal{F}_{Zeit}$) an Tagen mit besonders vielen Kursen.

Zusätzlich zur Verteilung auf die Tage ist die zeitliche Platzierung innerhalb eines Tages entscheidend für die Modellierung von Präferenzen. Generell dürfen Vorlesungen an der WU Werktags von 8:00 bis 21:30 und Samstags von 8:00 bis 17:30 stattfinden. Tabelle 5 schlüsselt die Startzeiten der einzelnen Kurseinheiten auf. Der Einfachheit halber werden angefangene Stunden bis 15 Minuten abgerundet und ab 30 Minuten aufgerundet. Vorlesungen dürfen nur im 15-Minuten-Takt beginnen.

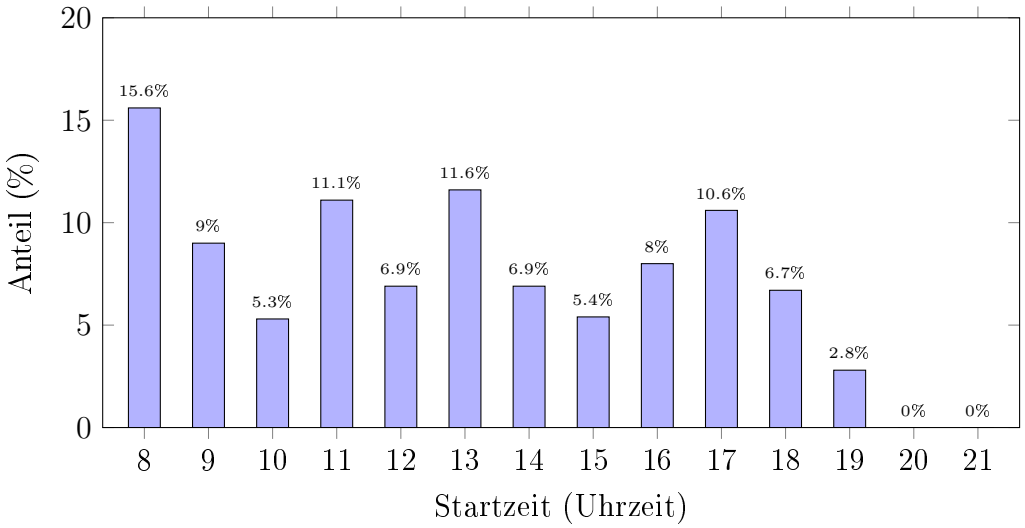


Figure 5: Verteilung der Kurs-Startzeiten (in Prozent)

Die Analyse der Startzeiten zeigt, dass die WU Wien ein sehr breites Zeitfenster nutzt. Ein signifikanter Anteil der Kurse beginnt bereits um 08:00 Uhr (15.6 %), was für Studierende mit Präferenzen für einen späteren Vorlesungsbeginn (vgl. Szenario 5.4.1) eine hohe Anzahl an potenziell zu vermeidenden Einheiten bedeutet.

Neben dem Beginn der Einheiten variiert auch die Dauer der einzelnen Termine, was die Berechnung von Überschneidungsfreiheiten komplexer gestaltet als bei starren Zeitslots. Tabelle 6 stellt die Dauer der Einheiten dar.

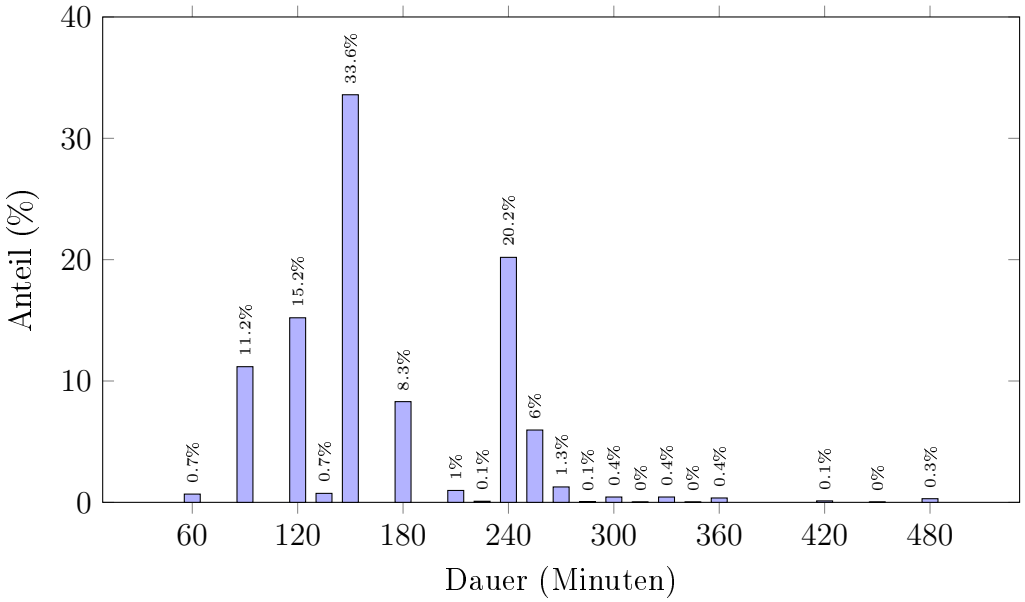


Figure 6: Verteilung der Terminlänge der Buchungsstunden in 15-Minuten-Intervallen

Die überwiegende Mehrheit der Veranstaltungen (79,1 %) dauert zwischen zwei und vier Stunden.

5 Analyse

5.1 Implementierung des ILP Baseline Ansatzes

Feldman und Golumbic vergleichen in Ihrem Paper [14] die Outputs der Algorithmen "Hill-Climbing" und "Offering-Order" mit dem optimalen Stundenplan, der mittels Brute-Force ermittelt wurde. Da ein Brute-Force Algorithmus bei der großen Anzahl der Vorlesungen im VVZ der WU eine zu lange Laufzeit hätte, wurde eine Integer Linear Programming (ILP) Lösung als Benchmark gewählt, die ebenfalls unter Berücksichtigung aller Nebenbedingungen den optimalen Stundenplan erzeugt [28].

5.1.1 Variablen, Parameter und Zielfunktion

Zur formalen Definition des Modells werden die folgenden Mengen, Parameter und Entscheidungsvariablen verwendet:

- **Mengen:**
  - $\mathcal{O}$: Menge aller verfügbaren Kursangebote (*Offerings*).
  - $\mathcal{D}$: Menge aller Kalendertage, an denen Kurse stattfinden.
  - $\mathcal{G}$: Menge der Kursgruppen (z. B. verschiedene Parallelveranstaltungen desselben Kurses).
  - $\mathcal{G}_g \subseteq \mathcal{O}$: Teilmenge der Angebote, die zur Gruppe $g \in \mathcal{G}$ gehören.
  - $\mathcal{D}_d \subseteq \mathcal{O}$: Teilmenge der Angebote, die Termine am Kalendertag $d \in \mathcal{D}$ haben.
  - $\mathcal{F}_{\text{Zeit}}$: Menge aller Paare (i, j) von Angeboten, die eine zeitliche Überschneidung aufweisen.
  - $\mathcal{M}_{\text{Muss}}, \mathcal{M}_{\text{Blockiert}} \subseteq \mathcal{O}$: Mengen von Angeboten, die zwingend gewählt bzw. ausgeschlossen werden müssen.
- **Parameter:**
  - $\text{Mark}(i)$: Der berechnete Nutzen für das Kursangebot i.
  - $C_{\min}, C_{\max}$: Unter- und Obergrenze für die Gesamtzahl der zu wählenden Kurse.
  - $D_{\min}, D_{\max}$: Unter- und Obergrenze für die Anzahl der Kurse an einem belegten Tag.
- **Entscheidungsvariablen:**

- $y_i \in \{0,1\}$: Binäre Variable; 1 wenn Kursangebot i gewählt wird, sonst 0.
- $u_d \in \{0,1\}$: Binäre Hilfsvariable; 1 wenn am Tag d mindestens ein Kurs belegt wird, sonst 0.

Mathematische Formulierung des ILP-Modells

$$\text{maximiere: } \sum_{i \in \mathcal{O}} \text{Mark}(i) \cdot y_i \tag{1}$$

unter den Nebenbedingungen:

$$\sum_{i \in \mathcal{O}} y_i \geq C_{\min} \tag{2}$$

$$\sum_{i \in \mathcal{O}} y_i \leq C_{\max} \tag{3}$$

$$\sum_{i \in \mathcal{G}_g} y_i \leq 1 \quad \forall g \in \mathcal{G} \tag{4}$$

$$\sum_{i \in \mathcal{D}_d} y_i \geq D_{\min} \cdot u_d \quad \forall d \in \mathcal{D} \tag{5}$$

$$\sum_{i \in \mathcal{D}_d} y_i \leq D_{\max} \cdot u_d \quad \forall d \in \mathcal{D} \tag{6}$$

$$y_i + y_j \leq 1 \quad \forall (i,j) \in \mathcal{F}_{\text{Zeit}} \tag{7}$$

$$y_i = 1 \quad \forall i \in \mathcal{M}_{\text{Muss}} \tag{8}$$

$$y_i = 0 \quad \forall i \in \mathcal{M}_{\text{Blockiert}} \tag{9}$$

$$y_i, u_d \in \{0,1\} \quad \forall i \in \mathcal{O}, \forall d \in \mathcal{D} \tag{10}$$

Figure 7: Mathematische Formulierung des ILP-Modells zur Stundenplanoptimierung

5.1.2 Nebenbedingungen (Constraints)

- **Anzahl der Lehrveranstaltungen** (Gleichungen 2 und 3): Begrenzen die Gesamtzahl der gewählten Lehrveranstaltungen, wobei $C_{\min}$ und $C_{\max}$ die definierten Mindest- bzw. Maximalanzahlen darstellen.

- **Gruppen-Beschränkung** (Gleichung 4): Stellt sicher, dass aus jeder vordefinierten Gruppe $g \in \mathcal{G}$ (verschiedene *Offerings* desselben Kurses) maximal eine Lehrveranstaltung ausgewählt wird.

- **Tägliche Kursanzahl** (Gleichungen 5 und 6): Regelt die Anzahl der Kurse pro Kalendertag $d \in \mathcal{D}$. Durch die binäre Hilfsvariable u_d wird sichergestellt, dass an einem Tag entweder gar kein Kurs ($u_d = 0$) oder eine Anzahl zwischen $D_{\min}$ und $D_{\max}$ ($u_d = 1$) belegt wird.

- **Zeitliche Überschneidungen** (Gleichung 7): Verhindert die gleichzeitige Auswahl von Lehrveranstaltungen, deren Termine sich zeitlich überschneiden. Die Menge $\mathcal{F}_{\text{Zeit}}$ enthält alle Paare (i, j), die einen Konflikt verursachen.

- **Feste Zuordnungen** (Gleichungen 8 und 9):

  1. $\mathcal{M}_{\text{Muss}}$ (8): Erzwingt die Auswahl von Lehrveranstaltungen, die als zwingend notwendig (Priorität = 100) definiert sind.
  2. $\mathcal{M}_{\text{Blockiert}}$ (9): Verbietet die Auswahl von Lehrveranstaltungen, die nicht gewählt werden dürfen (Priorität = -100).

Durch das Lösen dieses Modells wird ein optimaler Satz an Entscheidungsvariablen y_i (Stundenplan) bestimmt, der die Zielfunktion (*Mark*) maximiert, während alle Nebenbedingungen erfüllt werden. Das Ergebnis ist der optimal mögliche Stundenplan gemäß der definierten Constraints.

5.1.3 CPU vs. GPU

Um der technologischen Entwicklung seit der Veröffentlichung der ursprünglichen Algorithmen Rechnung zu tragen, wurde die ILP-Lösung sowohl für CPU- als auch für GPU-Architekturen (NVIDIA CUDA) implementiert [28][29]. Das ermöglicht eine Bewertung der Rechenzeit unter modernen Bedingungen, die über die Möglichkeiten der ursprünglichen Studie hinausgehen.

5.2 Implementierung der Hill-Climbing Algorithmen

Basierend auf der Methodik von Feldman und Golumbic [14] wurden die Optimierungsalgorithmen *Hill-Climbing v1* und *Hill-Climbing v3* implementiert. Das Ziel dieser Algorithmen ist es, inkrementell, durch lokal optimale Entscheidungen einen Stundenplan zu finden, der die Zielfunktion (*Mark*) maximiert.

5.2.1 Hill-Climbing v1

Der *Hill-Climbing v1*-Algorithmus wählt bei jedem Schritt das beste verfügbare *Offering* aus **allen verfügbaren Offerings** gemessen am *Mark* des Stundenplans inklusive des gewählten *Offering* [30].

5.2.2 Hill-Climbing v3

Der *Hill-Climbing v3*-Algorithmus unterscheidet sich vom *v1*-Ansatz dadurch, dass der Suchraum für einen neuen Kurs reduziert wird. Anstatt alle verfügbaren *Offerings* zu evaluieren, beschränkt *v3* die Auswahl auf eine Teilmenge der nach *Mark* sortierten Liste. Konkret wird in der Implementierung nur die zweite Hälfte — die Offerings mit der individuell besten *Mark* — der verfügbaren *Offerings* betrachtet [31].

5.2.3 Suchstrategie

Ausgehend vom aktuellen Stundenplan wird für jeden möglichen nächsten Zustand die *Mark* berechnet, indem der Stundenplan ergänzt um ein weiteres *Offering* durch die Evaluationsfunktion bewertet wird. Anschließend wird jenes Lehrangebot ausgewählt, das zu der höchsten Bewertung führt, und in den Stundenplan übernommen.

$$a^* = \arg \max_{a \in \text{available_offerings}} (\mathit{Mark}(\text{schedule} \cup \{a\})) \tag{11}$$

Nur jene *Offerings* werden in die Auswahl aufgenommen, die keine Nebenbedingungen verletzen (zeitliche Überschneidung, Planpunkt bereits erfüllt, *Offering* bereits gewählt, *Offering* in Liste verbotener Kurse). Die Schleife wird fortgesetzt, solange ein gültiges *Offering* gefunden wird, das zu einer Verbesserung oder Erweiterung des Stundenplans führt.

5.2.4 Abbruchkriterien

Der Algorithmus bricht ab, wenn einer der folgenden Zustände eintritt:

1. Der Stundenplan ist valide und hat die maximale Anzahl von Lehrveranstaltungen erreicht.

2. Der Stundenplan ist valide, und es kann keine weitere gültige *Offering* gefunden werden, ohne dass eine Nebenbedingung verletzt wird.

3. Es kann keine weitere *Offering* gefunden werden, die keine Constraints verletzt. Wenn die minimale Anzahl der Lehrveranstaltungen noch nicht erfüllt ist, führt das zu einem ungültigen Ergebnis.

Algorithm 2 Hill-Climbing

```
schedule ← ∅
availableOfferings ← FilterConflicts(offerings)
while availableOfferings ≠ ∅ do
    nextOffering ← arg max_{o∈availableOfferings} Mark(schedule ∪ {o})
    if IsValid(schedule ∪ {nextOffering}) then
        schedule ← schedule ∪ {nextOffering}
    end if
    if |schedule| = MaxScheduleLength then
        return schedule
    end if
    availableOfferings ← FilterConflicts(offerings)
end while
return schedule
```

5.2.5 Zur Nicht-Implementierung von Hill-Climbing v2

Der von Feldman und Golumbic vorgeschlagene *Hill-Climbing v2*-Algorithmus priorisiert zusätzlich auf der Ebene von *Gruppen* (*Groups*). Im Originalkontext des Papers beziehen sich Gruppen auf die Unterscheidung zwischen Pflichtveranstaltungen (*Mandatory*) und Wahlveranstaltungen (*Elective*), wobei die Priorität der Gruppen die Auswahl von Pflichtveranstaltungen vor Wahlveranstaltungen erzwingt [14]. Da es im Hauptstudium Wirtschaftsinformatik keine Unterscheidung zwischen Pflicht- und Wahlveranstaltungen gibt, wäre die Implementierung funktional identisch mit *v1*.

5.3 Implementierung des Offering-Order Algorithmus

Basierend auf der Methodik von Feldman und Golumbic [14] wurde der Optimierungsalgorithmus *Offering-Order* implementiert, der auf einer heuristischen Sortierung basiert.

5.3.1 Sortierung

Der Algorithmus nutzt die berechneten *Marks*, um eine Suchreihenfolge festzulegen:

1. Alle *Offerings* innerhalb des selben Planpunkts werden absteigend nach *Mark* sortiert. Innerhalb eines Planpunkts wird somit das Offering mit dem höchsten *Mark* zuerst in Betracht gezogen.

2. Die Planpunkte selbst werden nach dem *Mark* des besten *Offering* innerhalb des Planpunkts sortiert.

5.3.2 Suchstrategie

Der optimale Stundenplan wird durch eine *Forward-Checking Backtracking* Strategie gesucht. Es wird folgendermaßen vorgegangen:

1. Die Planpunkte werden iterativ durchlaufen. Gestartet wird mit dem Planpunkt, der das *Offering* mit der höchsten *Mark* beinhaltet (siehe 5.3.1).

2. Für jeden Planpunkt wird versucht, das *Offering* mit der höchsten *Mark* auszuwählen. Bevor die Auswahl getroffen wird, wird geprüft, ob das Hinzufügen des *Offerings* zu dem aktuellen Teilstundenplan zu einer zeitlichen Überschneidung führt oder andere Constraints verletzt (**Forward-Checking**)

3. Führt das Hinzufügen eines *Offerings* zu keinem gültigen Endergebnis in den nachfolgenden Planpunkten, wird die Auswahl rückgängig gemacht und das nächstbeste *Offering* der aktuellen Gruppe wird getestet (**Backtracking**)

Algorithm 3 Offering-Order Algorithm

```
schedule ← ∅
i ← 0
loop
    select an offering o ∈ groups_i
    if there is no o then
        backtrack to preceding state
    else
        schedule ← schedule ∪ {o}
        for all groups_j (i < j ≤ n) do
            remove from groups_j all violating offers
        end for
        if any groups_j becomes empty then
            backtrack to preceding state
        else
            i ← i + 1
            if i = n then
                return schedule
            end if
        end if
    end if
end loop
```

5.4 Szenariendefinition

Um die beschriebenen Algorithmen unter realistischen Bedingungen zu testen, wurden folgende Szenarien definiert:

5.4.1 Freier Tag

Laut einer Umfrage an der Hochschule Trier gaben 81% der Studierenden an, gerne einen komplett freien Tag zu haben, um einem Nebenjob nachzugehen. 13% der Studierenden gaben an, jeden Tag erst um 9:45 Uhr beginnen zu wollen, und 6% möchten einen oder mehrere freie Vormittage haben. Dabei wurde am häufigsten der Freitag (31%) vor dem Montag (6%) gewünscht. 64% der Studierenden wäre es egal, welchen Tag sie freibekommen [22]. Aus diesen Ergebnissen wurden folgende Szenarien abgeleitet:

- Keine Kurse an Montagen
- Keine Kurse an Freitagen
- Jeder Tag der Woche bis 9:45 frei
- Keine Kurse an Donnerstagen und Freitagen

5.4.2 Beliebte Zeiten

Laut einer weiteren Umfrage [11] bevorzugen 74% der Studierenden Prüfungen und Kurse, die über das Semester verteilt stattfinden. Teilt man den Tag in die drei Zeitslots Vormittag, Nachmittag und Abend ein, so war der Nachmittag der beliebteste Zeitslot, gefolgt vom Vormittag und dem Abend. Aus diesen Ergebnissen wurden folgende Szenarien abgeleitet:

- Erhöhte Priorität (+90) für Nachmittags, negative Priorität (−90) für Abend
- Erhöhte Priorität (+70) für Vormittag, neutrale Priorität (0) für Nachmittag, negative Priorität (−80) für Abend

5.4.3 Ähnlicher Problemstellungen

Hinzu kommen klassische Constraints, die in University-Course-Timetabling-Problemen häufig vorkommen [8]. Diese Constraints betreffen meist die universitäre Planung des Curriculums und die Zuteilung von Kursen zu Räumen, es gibt jedoch auch Constraints, die die Erstellung individueller Stundenpläne der Studierenden betreffen:

- Studierende müssen von 12:00 bis 13:00 Zeit für ein Mittagessen haben
- Pro Tag werden entweder keine oder mindestens 3 Stunden Vorlesungen besucht
- Pro Tag dürfen nicht mehr als 8 Stunden Vorlesungen besucht werden

5.4.4 Vergleichbarkeit

Zusätzlich zu den oben genannten Constraints sollen die Algorithmen auch in Extremfällen hinsichtlich ihrer Performance getestet werden. Daher werden ergänzend folgende Szenarien definiert, die in verwandten Arbeiten nicht explizit untersucht wurden, jedoch zur Vergleichbarkeit zwischen den Algorithmen sinnvoll sind:

- Alle Kurse können frei von Constraints gewählt werden.
- 25% - 75% aller Kurse können nicht belegt werden.
- 1 - 7 Kurse sind vorgegeben
- Keine Kurse zwischem 6:00 morgens und 13:00.
- Keine Kurse Montags, Dienstags und Mittwochs

5.5 Versuchsaufbau

Das Paper von Feldman und Golumbic [14] wählte einen Brute-Force-Algorithmus als Baseline für Performance-Vergleiche. Da ein Brute-Force Algorithmus bei der großen Anzahl der Vorlesungen im VVZ der WU eine zu lange Laufzeit hätte, wurde eine Integer Linear Programming (ILP) Lösung als neue Baseline gewählt. Diese gewährleistet, ebenso wie der Brute-Force-Ansatz für kleine Instanzen, die Ermittlung des optimalen Stundenplans (mit der höchsten *Mark*). Die Ergebnisse der heuristischen Algorithmen (*Hill-Climbing v1*, *Hill-Climbing v3*, *Offering-Order*) werden folglich relativ zur optimalen Lösung des ILP-Ansatzes bewertet. Zusätzlich zu den heuristischen Algorithmen wird auch eine GPU-optimierte Variante des ILP-Ansatzes getestet.

Als Maß für die Schwierigkeit (*Difficulty*) der Probleminstanz wird in dieser Arbeit nicht die Laufzeit des Brute-Force-Algorithmus verwendet, sondern die Anzahl der zu planenden Kurse (N).

Die Algorithmen wurden anhand der folgenden Leistungskennzahlen evaluiert:

1. **Mark**: Die Qualität der gefundenen Lösung (in Prozent relativ zum Baseline Ansatz).

2. **Laufzeit**: Die zur Lösungsfindung benötigte Zeit.

3. **Speicherbedarf**: Der benötigte Hauptspeicher.

5.5.1 Zielfunktion

Die *Mark* dient als Bewertungsfunktion für die Qualität eines erstellten Stundenplans und entspricht der im Paper von Feldman und Golumbic [14] definierten Zielfunktion. Sie quantifiziert, wie gut ein Stundenplan die Präferenzen und Constraints der Studierenden erfüllt. Die Berechnung erfolgt ausschließlich für *gültige* Stundenpläne, d. h. solche, die keine restriktiven Constraints ($|p| = P$) verletzen.

Jede Präferenz oder Restriktion ist im Modell durch eine *Priorität* p im Wertebereich $[-P, P]$ definiert. Positive Prioritäten kennzeichnen wünschenswerte Eigenschaften (z. B. bevorzugte Lehrveranstaltungen oder Zeitfenster), negative Prioritäten hingegen unerwünschte. Die Stärke des Präferenzwerts ist proportional zum Absolutwert der Priorität.

Die *Mark* eines gültigen Stundenplans S ergibt sich aus der Summe der positiven Beiträge durch gewählte Kurse und den Abzügen für Verletzungen sogenannter *fixed* und *non-fixed* Constraints:

$$Mark(S) = \underbrace{\sum_{c \in C_S} q_c}_{\text{Kursprioritäten}} - \underbrace{\sum_{h \in H^{(f)}_{\text{violated}}} |p_h^{(f)}|}_{\text{Strafen für fixed-Constraints}} - \underbrace{\sum_{t \in T^{(n)}_{\text{violated}}} v_t \cdot |p_t^{(n)}|}_{\text{Strafen für non-fixed-Constraints}} , \tag{12}$$

wobei gilt:

- C_S: Menge aller im Stundenplan S enthaltenen Kurse.
- q_c: Priorität des Kurses c.
- $H^{(f)}_{\text{violated}}$: Menge aller Stunden, in denen ein *fixed*-Constraint verletzt wurde.
- $p_h^{(f)}$: Priorität der Stunde h. Eine Verletzung liegt vor, wenn
  - $p_h^{(f)} < 0$ und die Stunde aktiv ist (der Studierende hat in einer ungewünschten Stunde Unterricht), oder
  - $p_h^{(f)} > 0$ und die Stunde inaktiv ist (der Studierende hat in einer gewünschten Stunde keinen Unterricht).
- $T^{(n)}_{\text{violated}}$: Menge der verletzten *non-fixed*-Constraints.
- $p_t^{(n)}$: Priorität des Constraints t.
- v_t: Anzahl der verletzten Stunden (bzw. des Zeitumfangs), der durch Constraint t betroffen ist.

Je höher der Wert von $M(S)$, desto besser erfüllt der Stundenplan die angegebenen Präferenzen. Ziel der Optimierungsalgorithmen ist daher die Maximierung von $M(S)$.

5.5.2 Durchführung

Die experimentelle Evaluation der Algorithmen erfolgte durch systematische Benchmarks, die Performance und Lösungsqualität der implementierten Ansätze unter verschiedenen Szenarien und Schwierigkeitsgraden testet.

Die Experimente wurden auf dem Hybridair-GPU-Server der Wirtschaftsuniversität Wien durchgeführt. Die Hardware Infrastruktur hat folgende Spezifikationen:

- **Prozessor:** AMD EPYC 9334

- **Arbeitsspeicher:** 128 GB RAM
- **Grafikkarte (ILP GPU):** Zwei NVIDIA RTX A5000 GPUs (jeweils 24 GB VRAM) mit CUDA 12.4
- **Betriebssystem:** Ubuntu/Debian via Proxmox Kernel (6.8.12-11-pve)

Um eine verlässliche und gleichbleibende Testumgebung zu garantieren, wurde das Tool **benchexec** genutzt. Jeder Testlauf hatte ein Zeitlimit von 15 Sekunden und lief auf einem Prozessorkern. Um die Ergebnisse nicht zu verfälschen, wurde der Server während der Tests nicht anderweitig verwendet.

Die Algorithmen wurden in 14 verschiedenen Szenarien getestet. Jedes Szenario stellt eine vorkonfigurierte Zusammensetzung von Constraints dar (definiert in JSON-Konfigurationsdateien, z.B. `constraint1.json`).

Der Schwierigkeitsgrad (*Difficulty*) einer Probleminstanz wird durch die Anzahl der zu planenden Kurse N definiert, im Gegensatz zur Laufzeit des Brute-Force-Algorithmus, die in der Originalarbeit verwendet wurde. Die Tests wurden inkrementell für jede mögliche Kursanzahl N, beginnend bei $N = 1$ bis zur maximal möglichen Kursanzahl im jeweiligen Szenario, durchgeführt. Die maximale Kursanzahl wurde dabei vorab mithilfe des ILP-Baseline-Ansatzes ermittelt.

Für jeden Algorithmus ($A \in \{$ILP, ILP GPU, Offering-Order, Hill-Climbing V1, Hill-Climbing für jede Kursanzahl N und für jedes Szenario S wurde die Messreihe wie folgt durchgeführt:

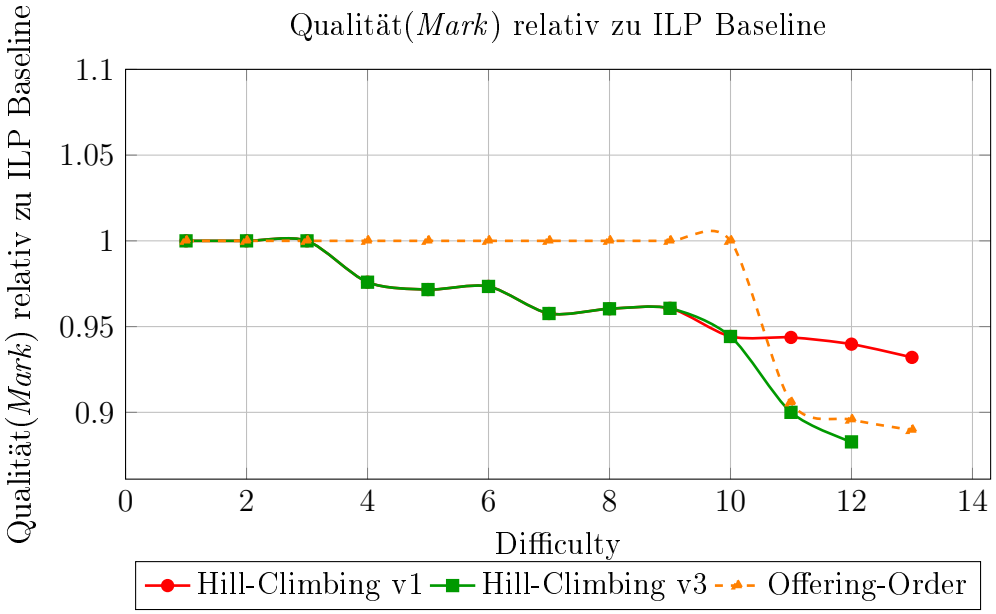
1. Jeder Algorithmus wurde für eine feste Kursanzahl N 50 Mal unabhängig voneinander ausgeführt.
2. Die 50 Messungen pro Algorithmus und Schwierigkeitsgrad wurden sequenziell durchgeführt, um sicherzustellen, dass die Messergebnisse für die Laufzeit und den Speicherbedarf unabhängig sind.
3. Es wurden folgende Metriken für jede der 50 Ausführungen erfasst:
   - Laufzeit (t): Die zur Lösungsfindung benötigte Zeit (in Sekunden).
   - Hauptspeicher (M): Der während der Ausführung belegte Hauptspeicher (in Megabyte).
   - Qualität (*Mark*)
4. Für die 50 Messwerte der Laufzeit und des Hauptspeichers wurden folgende statistische Kennzahlen berechnet:

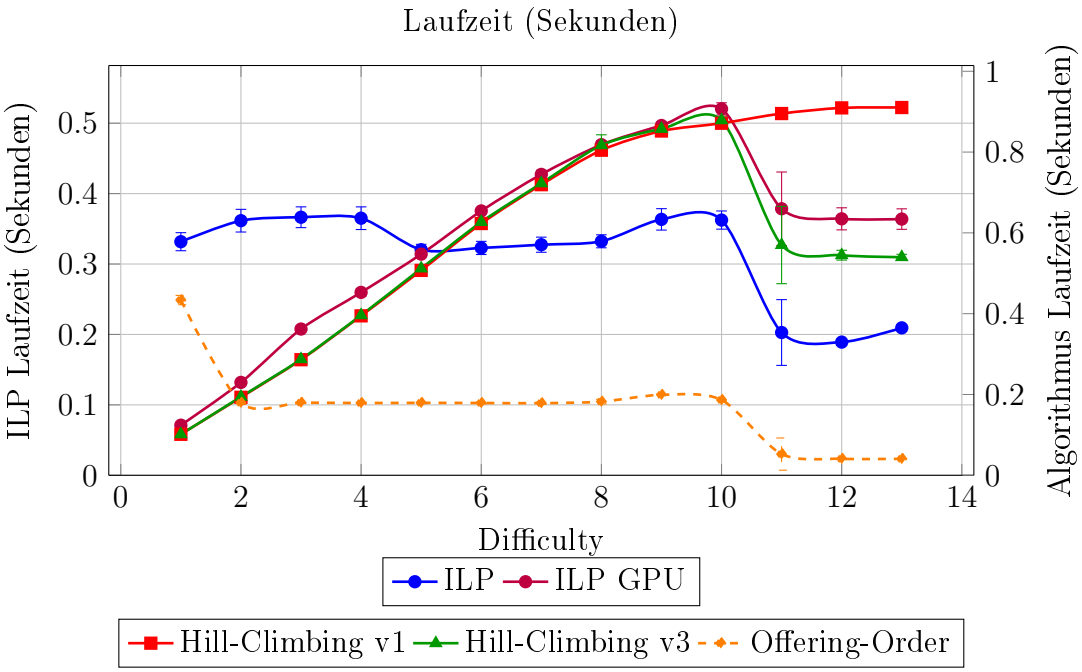
- Median ($t_{\text{median}}, M_{\text{median}}$)
- Mittelwert ($t_{\text{mean}}, M_{\text{mean}}$)
- Minimalwert ($t_{\text{min}}, M_{\text{min}}$)
- Maximalwert ($t_{\text{max}}, M_{\text{max}}$)
- Standardabweichung ($t_{\text{stdev}}, M_{\text{stdev}}$)

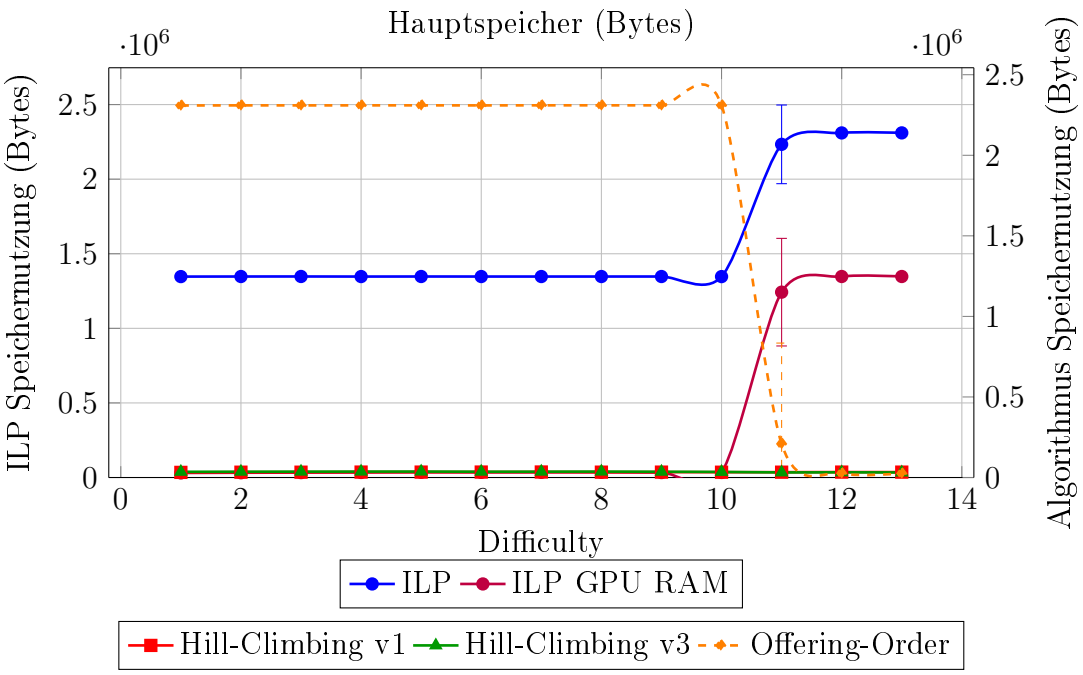
Um die Algorithmen vergleichbar zu halten, wurde der gemessene Zeit- und Speicherbedarf auf die Ausführung des Algorithmus selbst begrenzt. Das Preprocessing — insbesondere die Vorabberechnung der *Mark* für jedes Offering, die einmalig vor der Hauptschleife durchgeführt wird — ist nicht in den Messwerten enthalten.

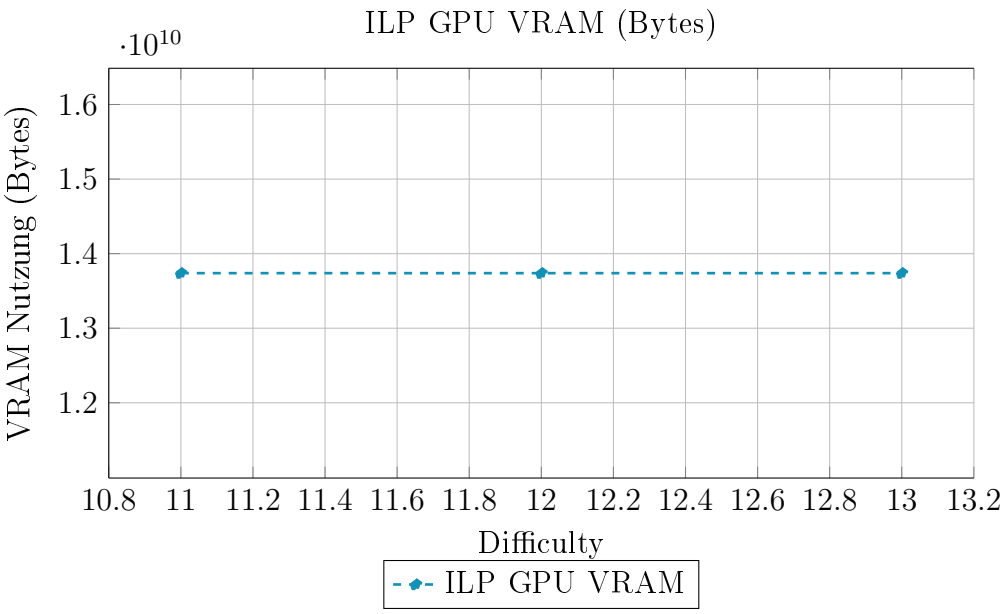
6 Ergebnisse

6.1 Szenario 1: Keine Kurse an Montagen

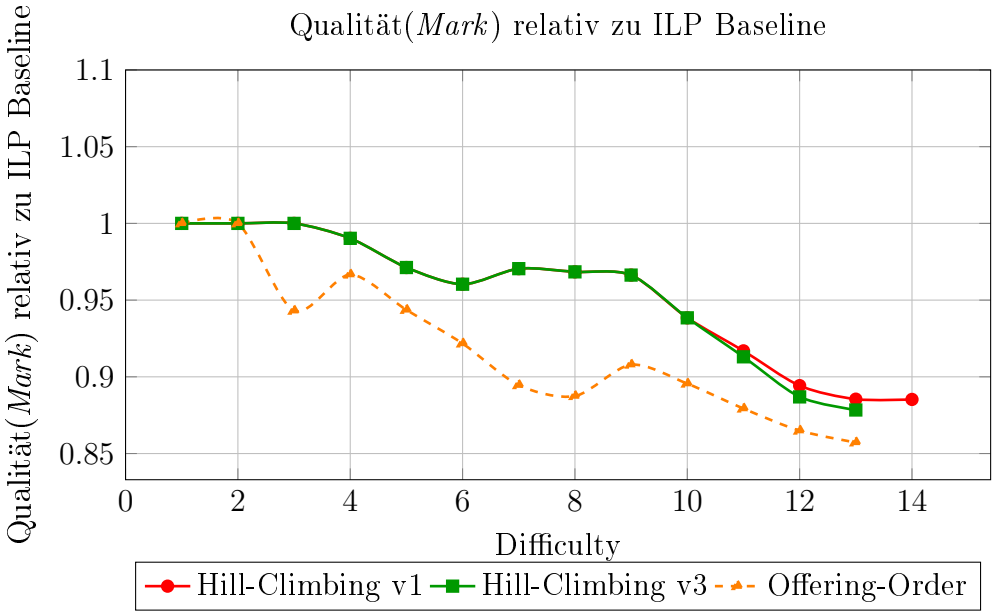


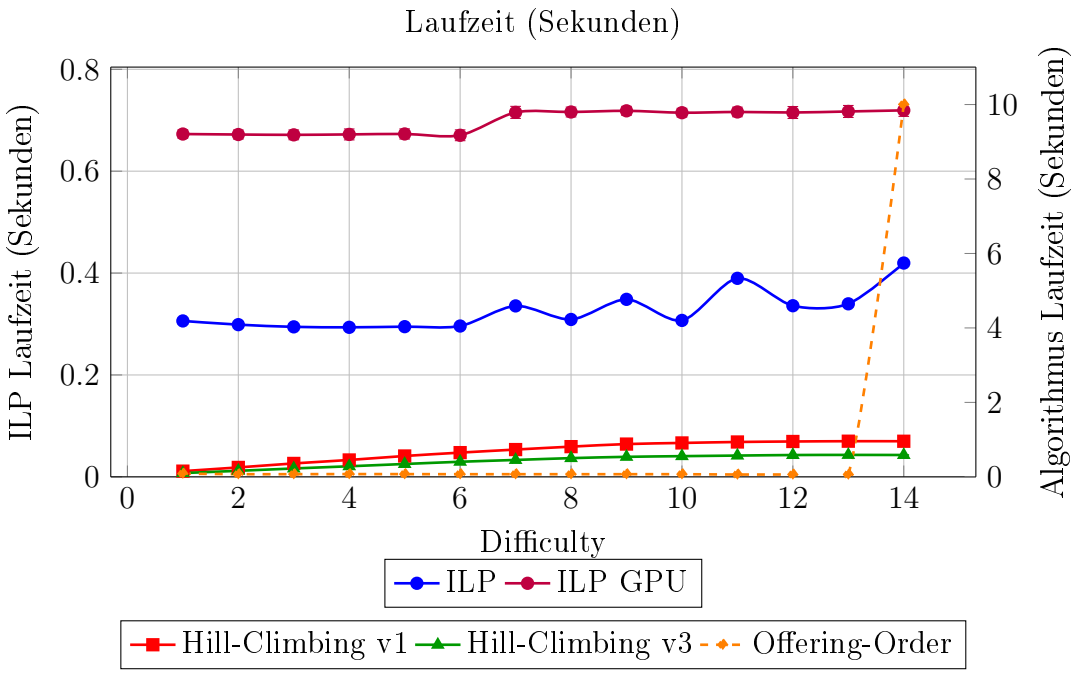


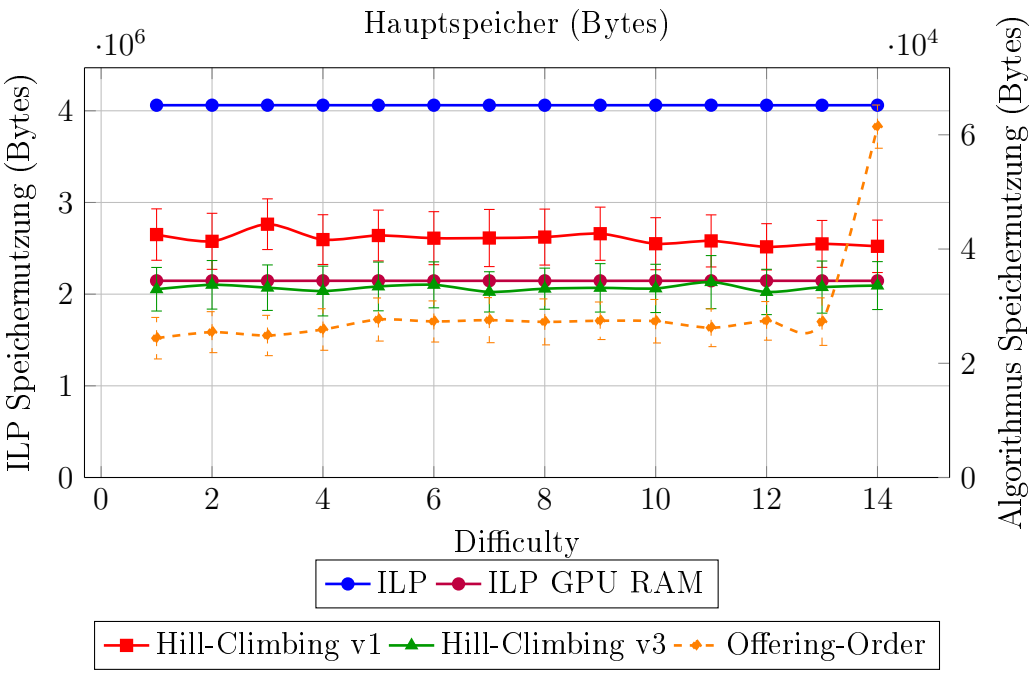

Hauptspeicher (Bytes)
$\cdot 10^6$
$\cdot 10^6$
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
0
0.5
1
1.5
2
2.5
0
2
4
6
8
10
12
14
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order

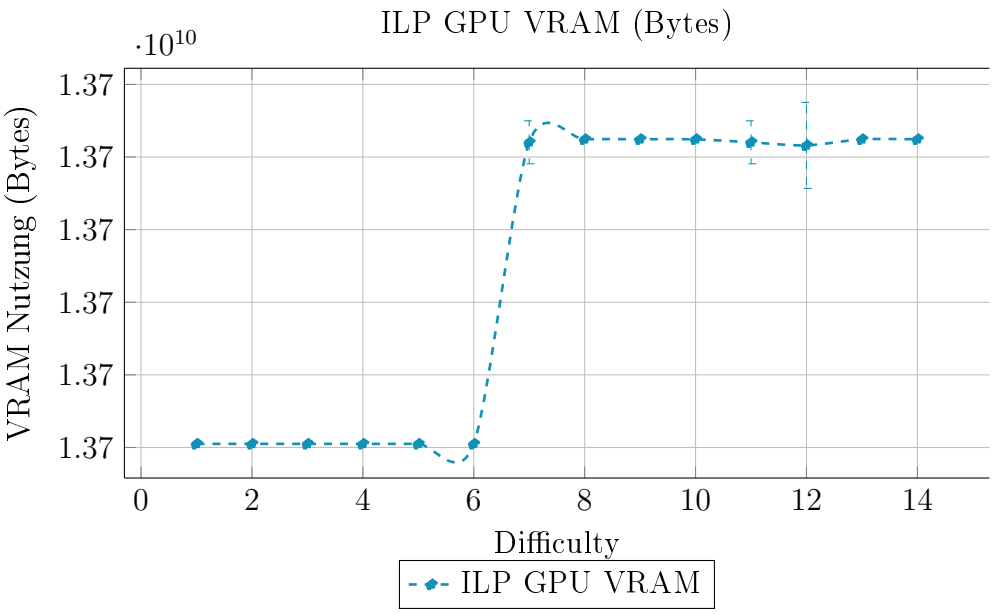

ILP GPU VRAM (Bytes)
$\cdot 10^{10}$
VRAM Nutzung (Bytes)
1.2
1.3
1.4
1.5
1.6
10.8
11
11.2
11.4
11.6
11.8
12
12.2
12.4
12.6
12.8
13
13.2
Difficulty
ILP GPU VRAM

6.2 Szenario 2: Keine Kurse an Freitagen

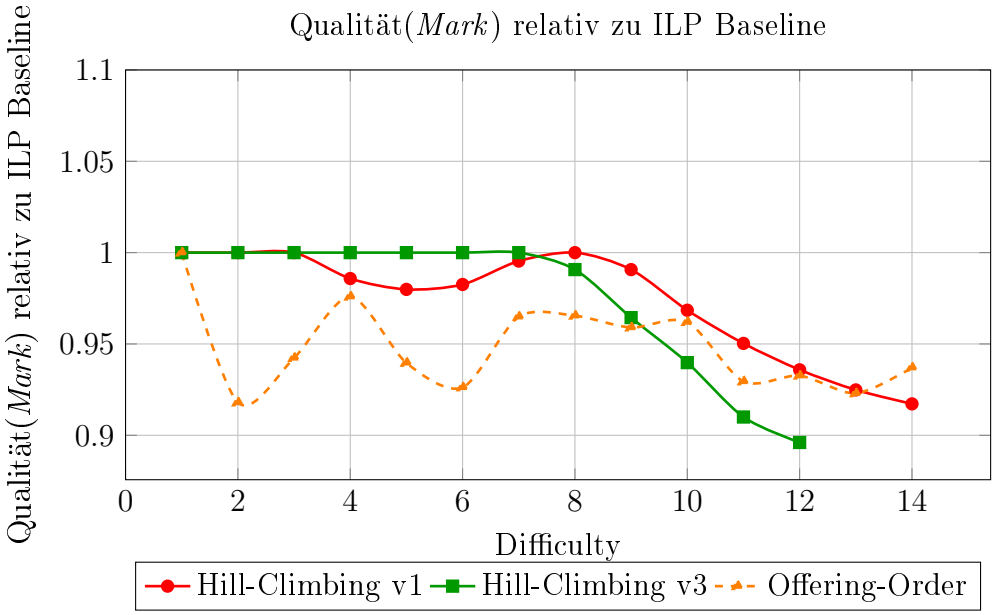


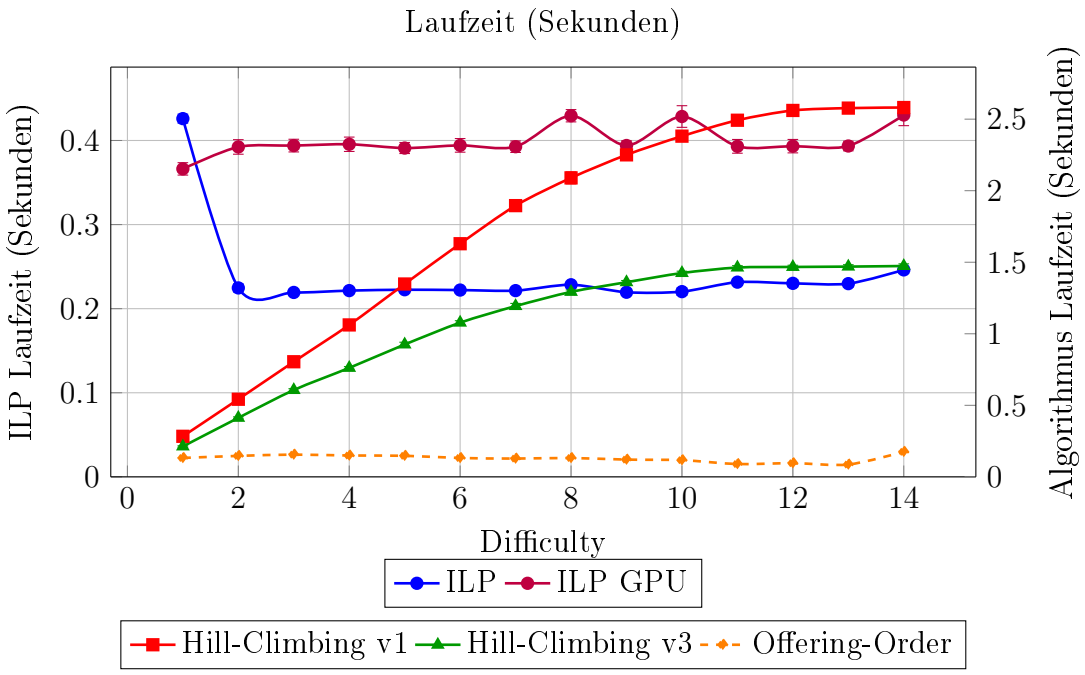


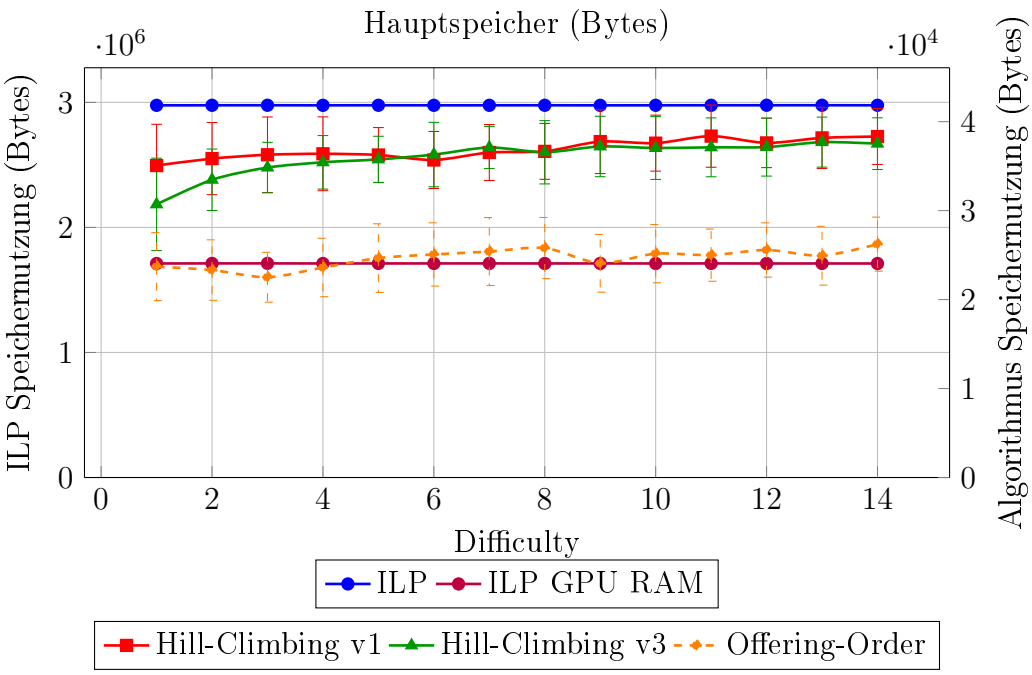

Hauptspeicher (Bytes)
$\cdot 10^6$
$\cdot 10^4$
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
0
1
2
3
4
0
2
4
6
0
2
4
6
8
10
12
14
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order

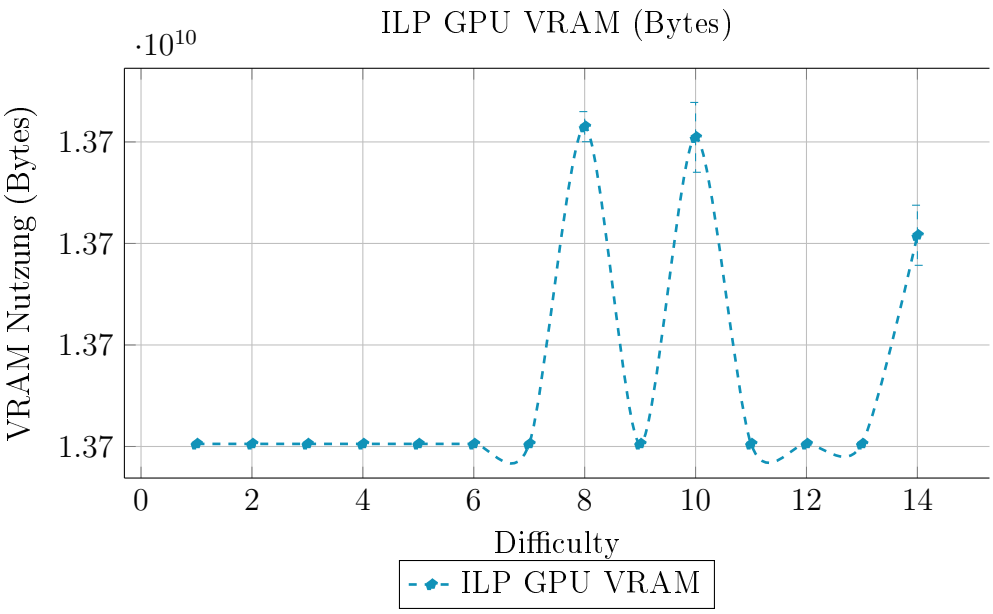

ILP GPU VRAM (Bytes)
$\cdot 10^{10}$
VRAM Nutzung (Bytes)
1.37
1.37
1.37
1.37
1.37
1.37
0
2
4
6
8
10
12
14
Difficulty
ILP GPU VRAM

6.3 Szenario 3: Jeder Tag der Woche bis 9:45 frei

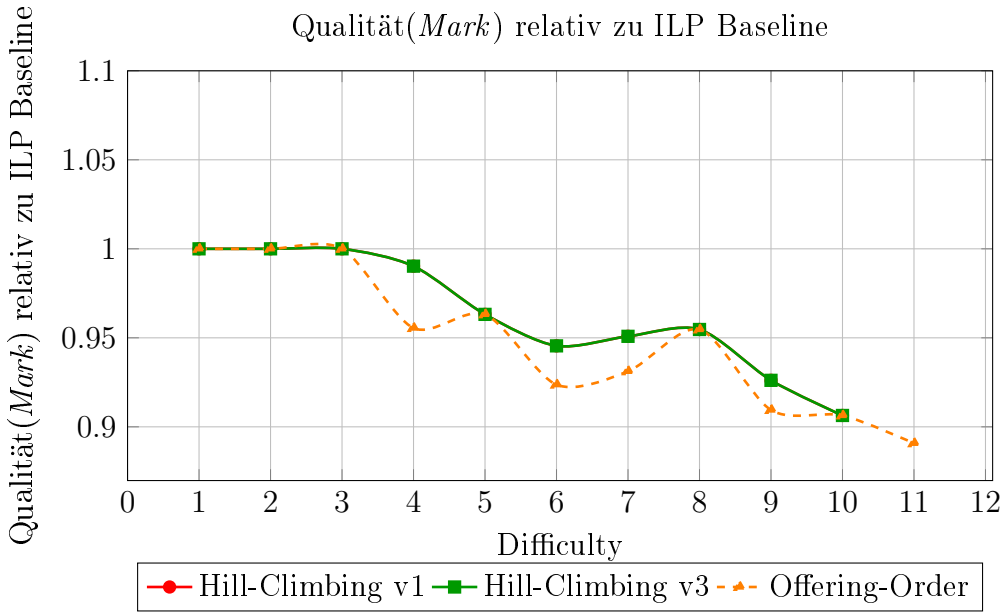


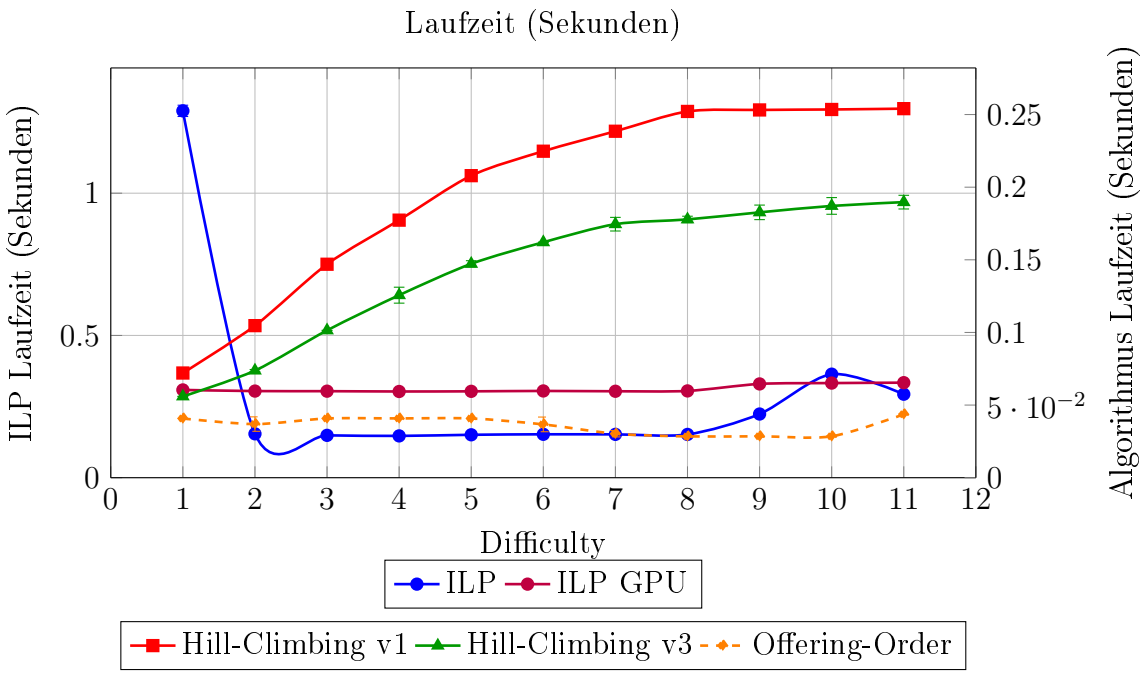


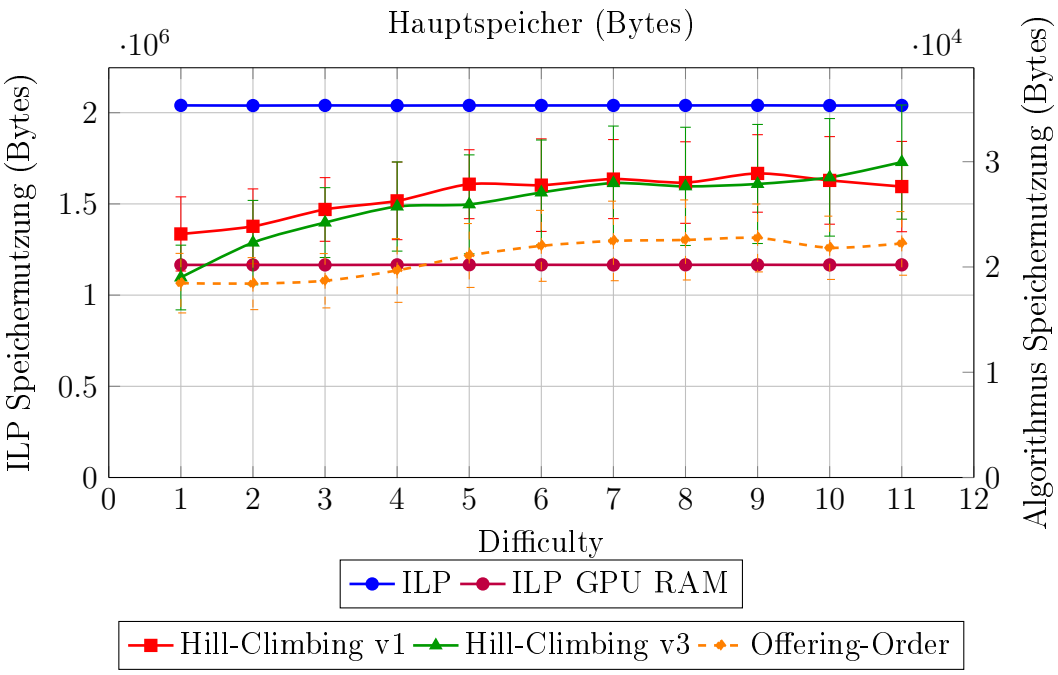

Hauptspeicher (Bytes)
$\cdot 10^6$
$\cdot 10^4$
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
0
1
2
3
0
1
2
3
4
0
2
4
6
8
10
12
14
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order

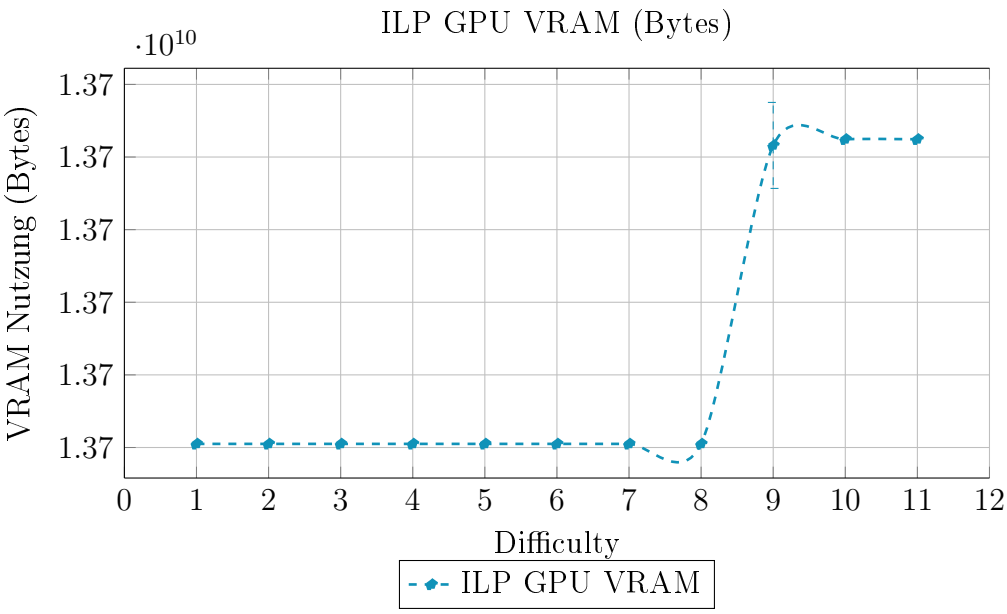

ILP GPU VRAM (Bytes)
$\cdot 10^{10}$
VRAM Nutzung (Bytes)
1.37
1.37
1.37
1.37
0
2
4
6
8
10
12
14
Difficulty
ILP GPU VRAM

6.4 Szenario 4: Keine Kurse an Donnerstagen und Freitagen

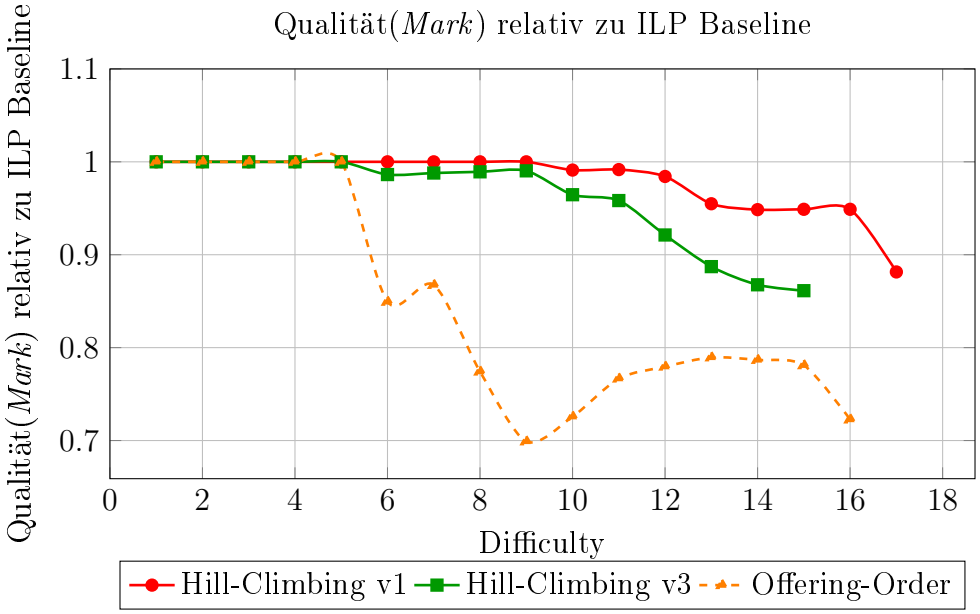


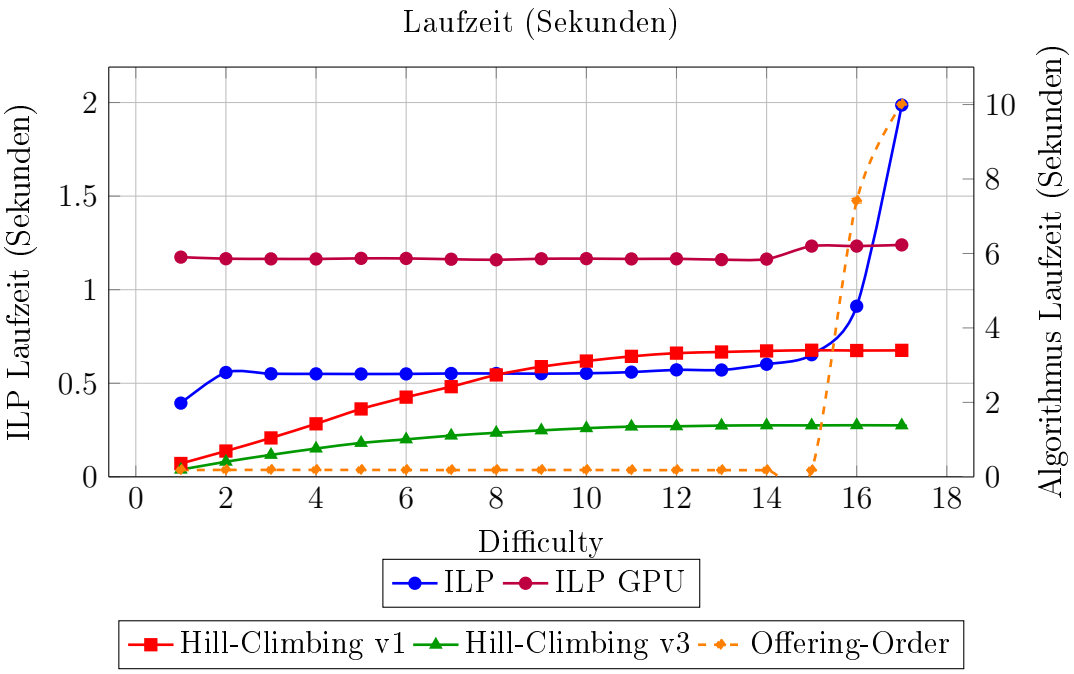


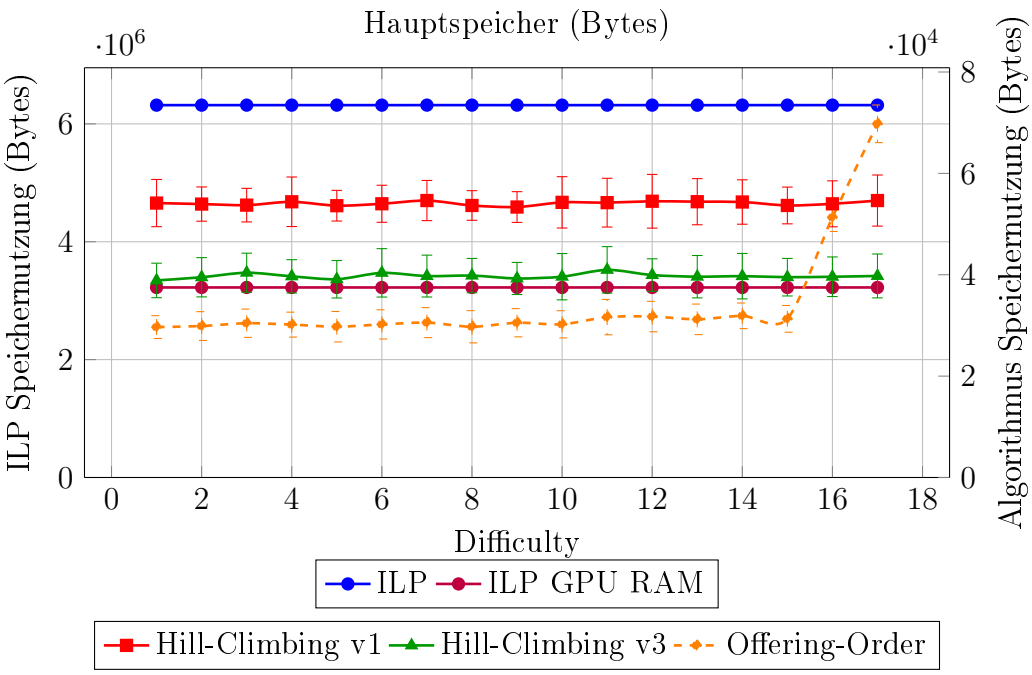

Hauptspeicher (Bytes)
·10^6
·10^4
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
0
0.5
1
1.5
2
0
1
2
3
0 1 2 3 4 5 6 7 8 9 10 11 12
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order

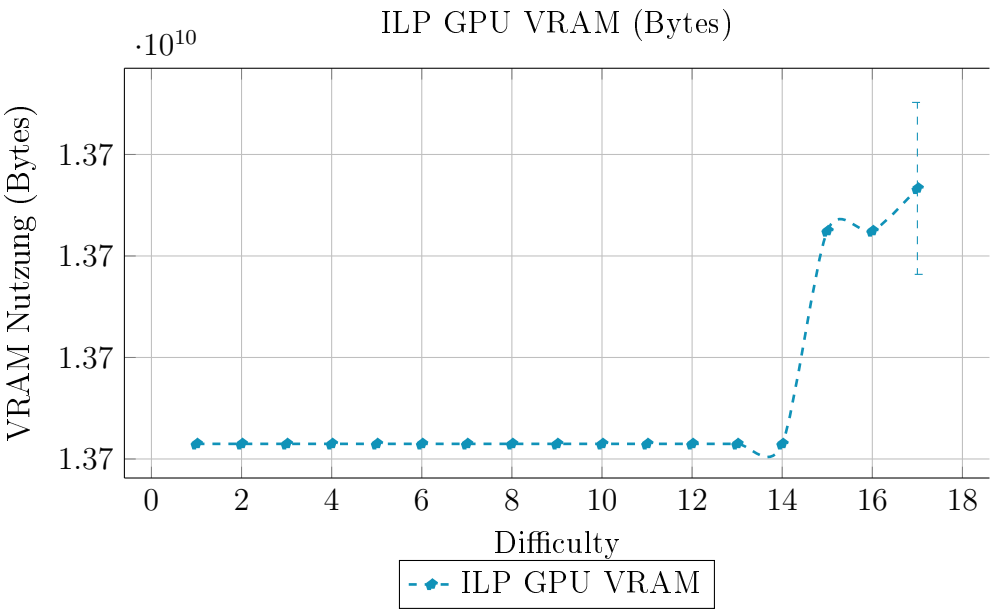

ILP GPU VRAM (Bytes)
·10^10
VRAM Nutzung (Bytes)
1.37
1.37
1.37
1.37
1.37
1.37
0 1 2 3 4 5 6 7 8 9 10 11 12
Difficulty
ILP GPU VRAM

6.5 Szenario 5: Erhöhte Priorität (+90) für Nachmittags, negative Priorität (-90) für Abend

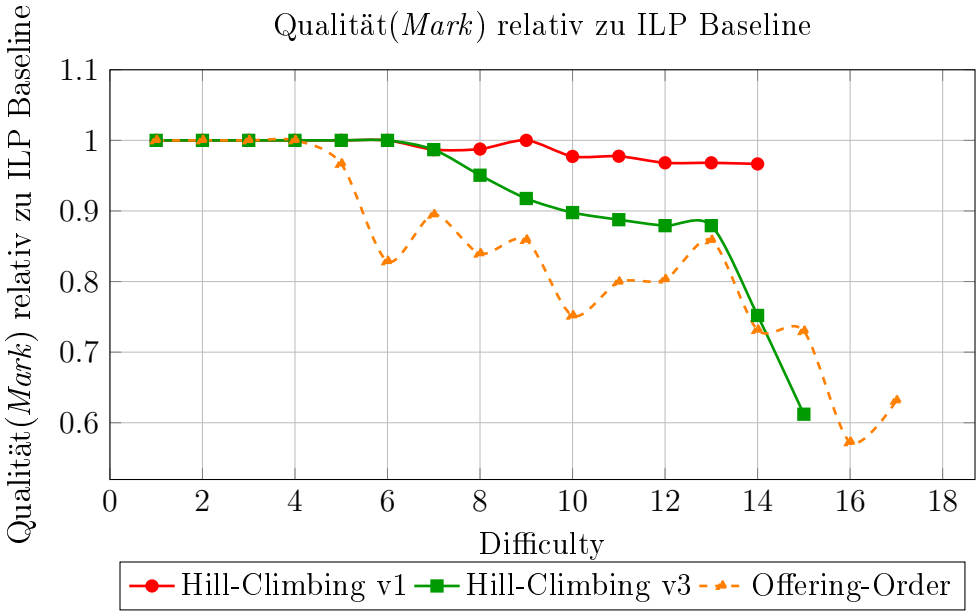


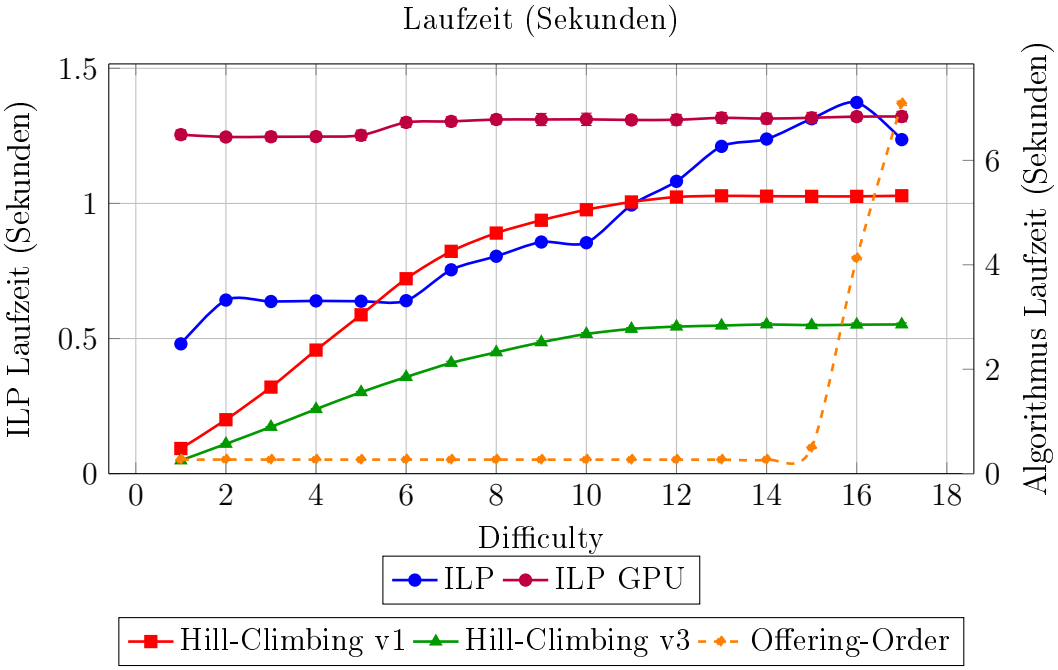


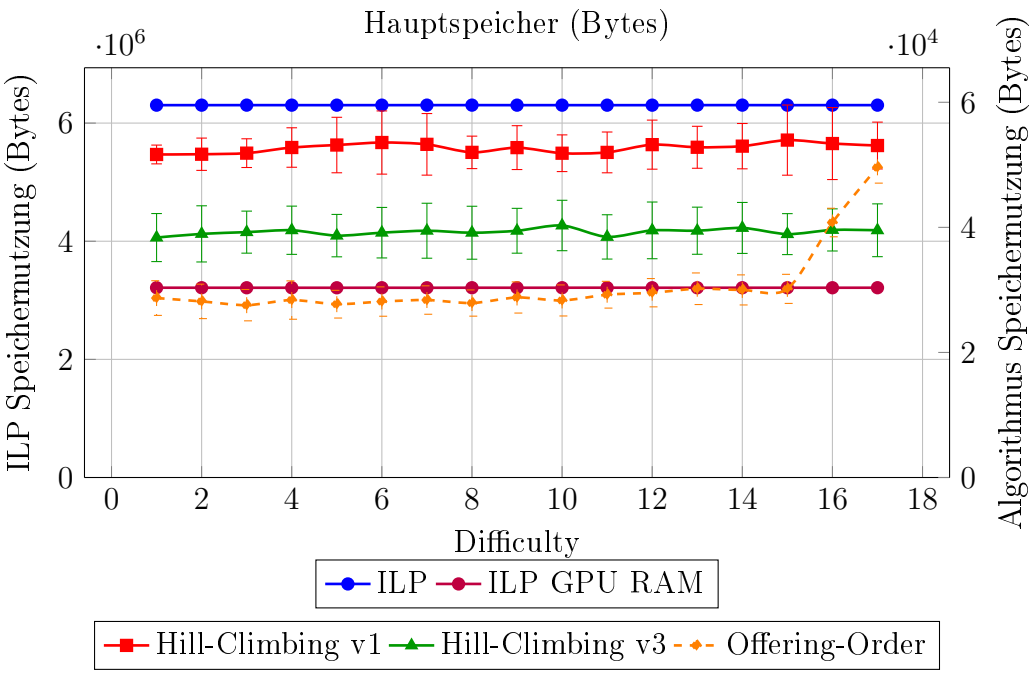

Hauptspeicher (Bytes)
$\cdot 10^6$
$\cdot 10^4$
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
0
2
4
6
8
0
2
4
6
8
10
12
14
16
18
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order

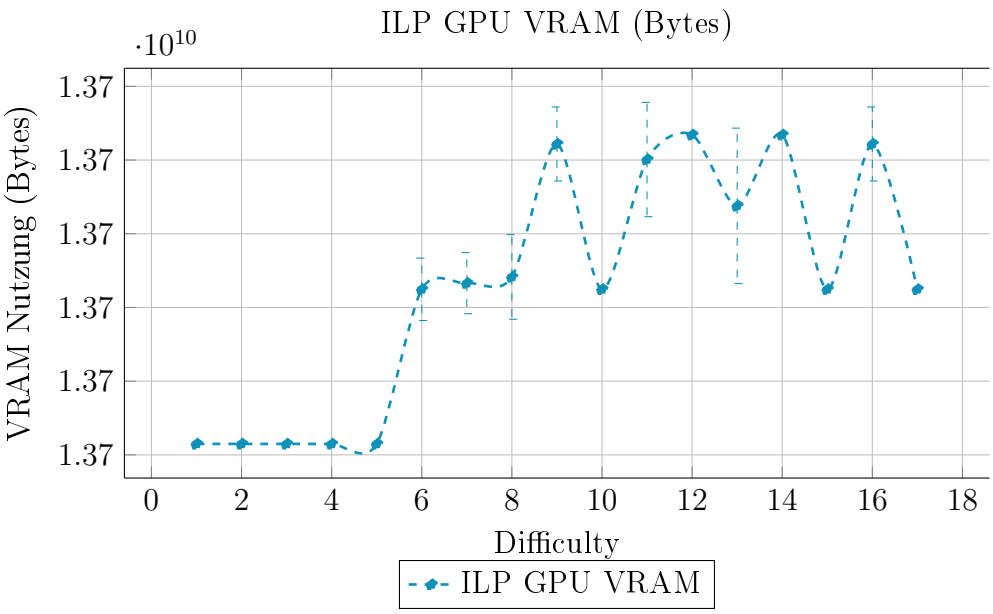

ILP GPU VRAM (Bytes)
$\cdot 10^{10}$
VRAM Nutzung (Bytes)
1.37
1.37
1.37
1.37
0
2
4
6
8
10
12
14
16
18
Difficulty
ILP GPU VRAM

6.6 Szenario 6: Erhöhte Priorität (+70) für Vormittag, neutrale Priorität (0) für Nachmittag, negative Priorität (-80) für Abend

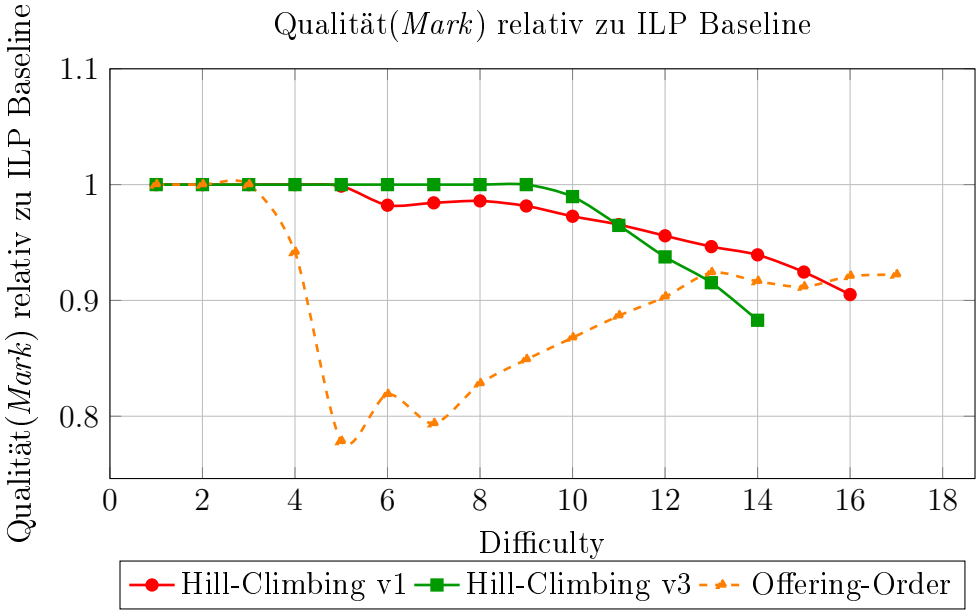


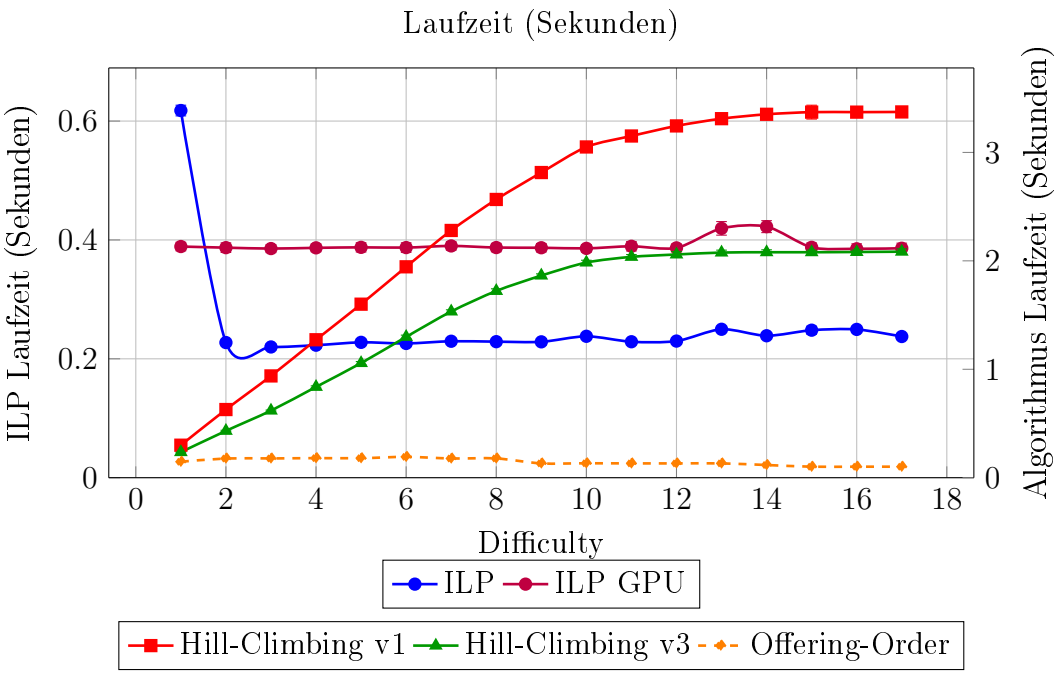


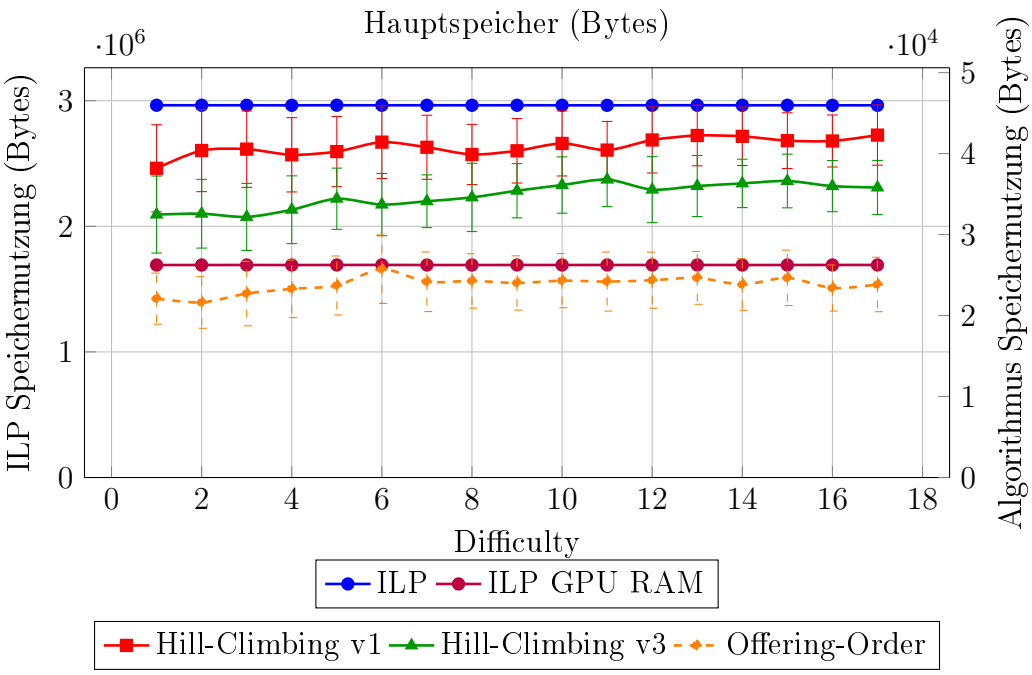

Hauptspeicher (Bytes)
$\cdot 10^6$
$\cdot 10^4$
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
0
2
4
6
0
2
4
6
8
10
12
14
16
18
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order

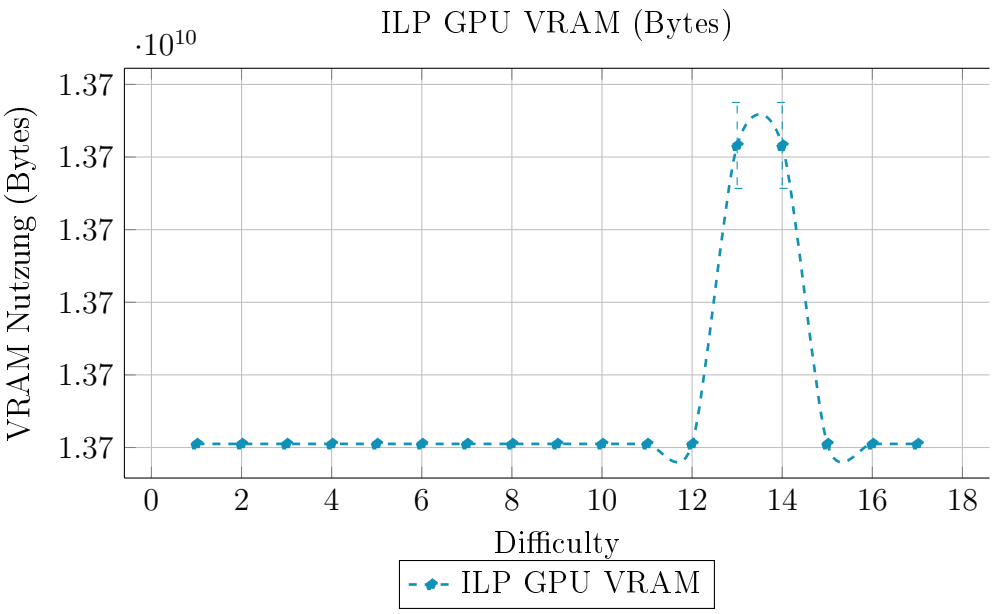

ILP GPU VRAM (Bytes)
$\cdot 10^{10}$
VRAM Nutzung (Bytes)
1.37
1.37
1.37
1.37
1.37
1.37
0
2
4
6
8
10
12
14
16
18
Difficulty
ILP GPU VRAM

6.7 Szenario 7: Studierende müssen von 12:00 bis 13:00 Zeit für ein Mittagessen haben

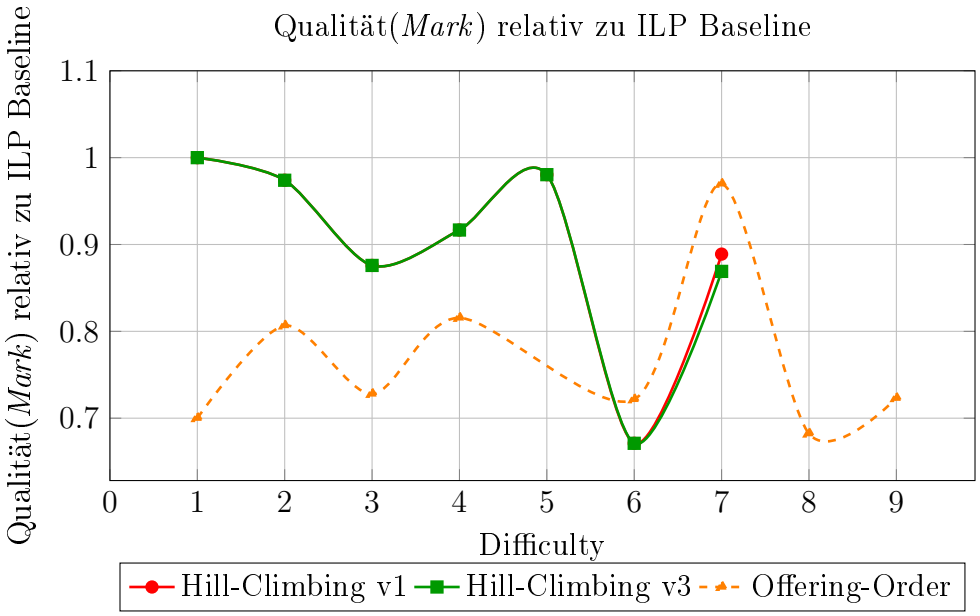


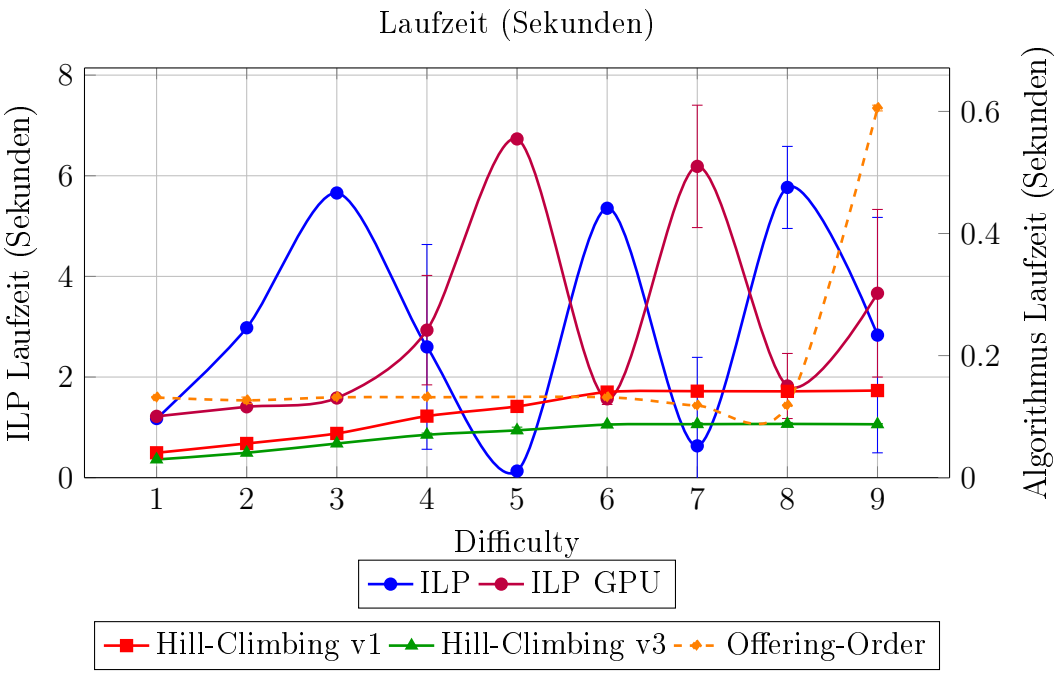


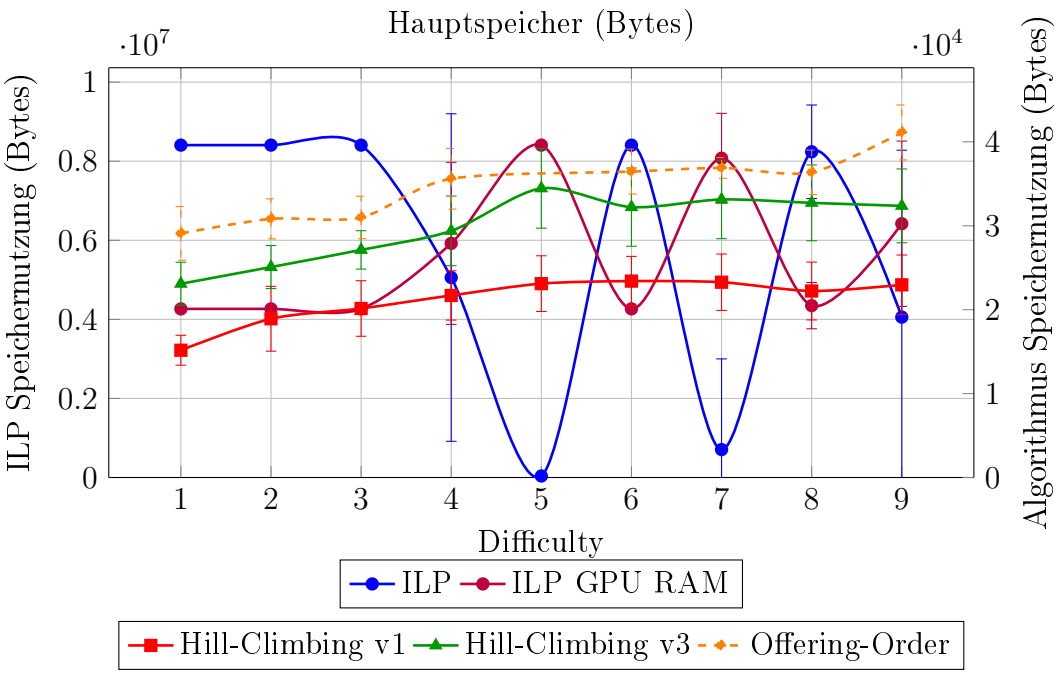

Hauptspeicher (Bytes)
$\cdot 10^6$
$\cdot 10^4$
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
0
1
2
3
0
1
2
3
4
5
0
2
4
6
8
10
12
14
16
18
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order

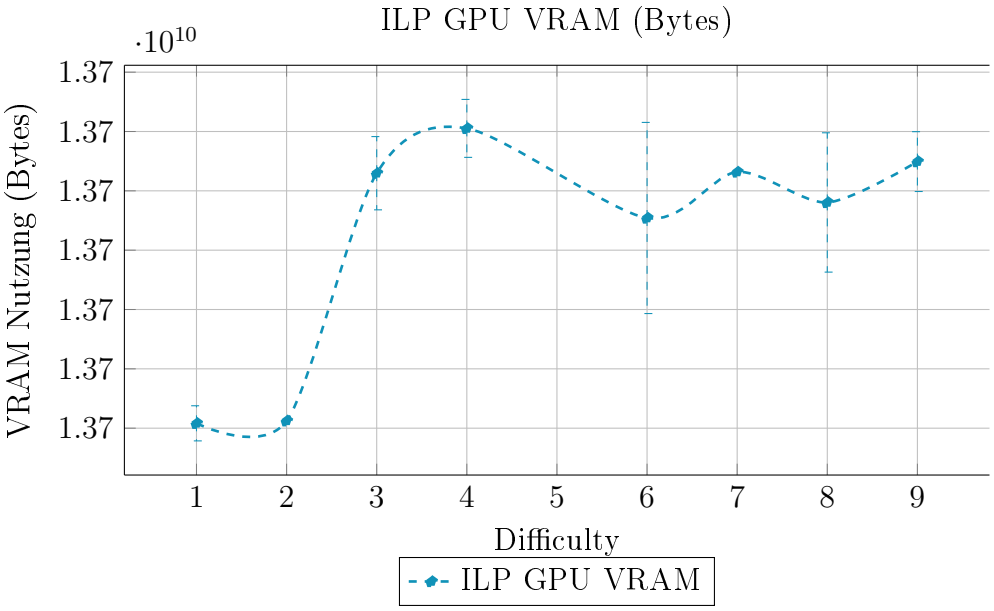

ILP GPU VRAM (Bytes)
$\cdot 10^{10}$
VRAM Nutzung (Bytes)
1.37
1.37
1.37
1.37
1.37
1.37
0
2
4
6
8
10
12
14
16
18
Difficulty
ILP GPU VRAM

6.8 Szenario 8: Studierende müssen mehr als einen Kurs pro Tag besuchen

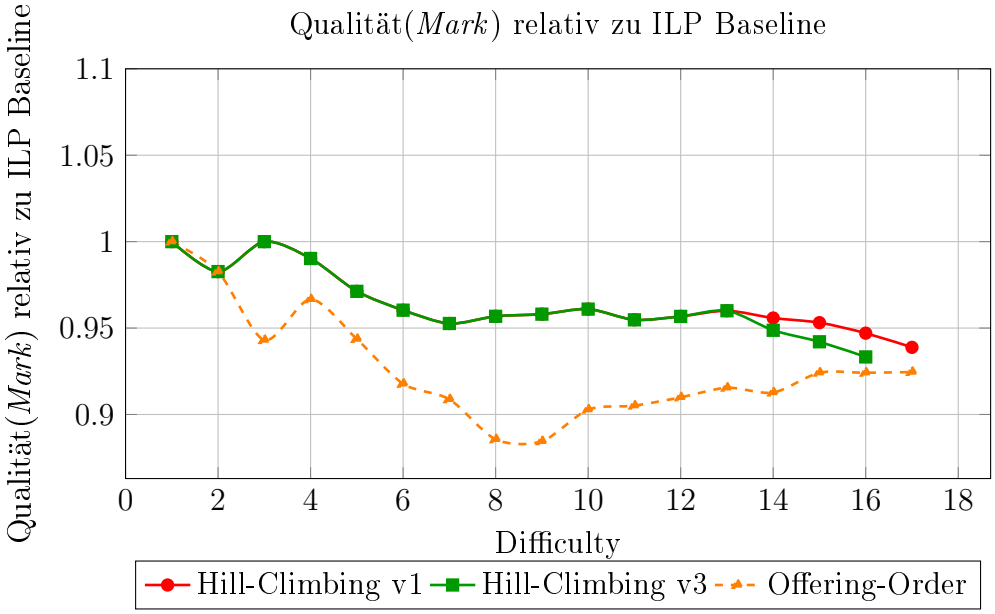


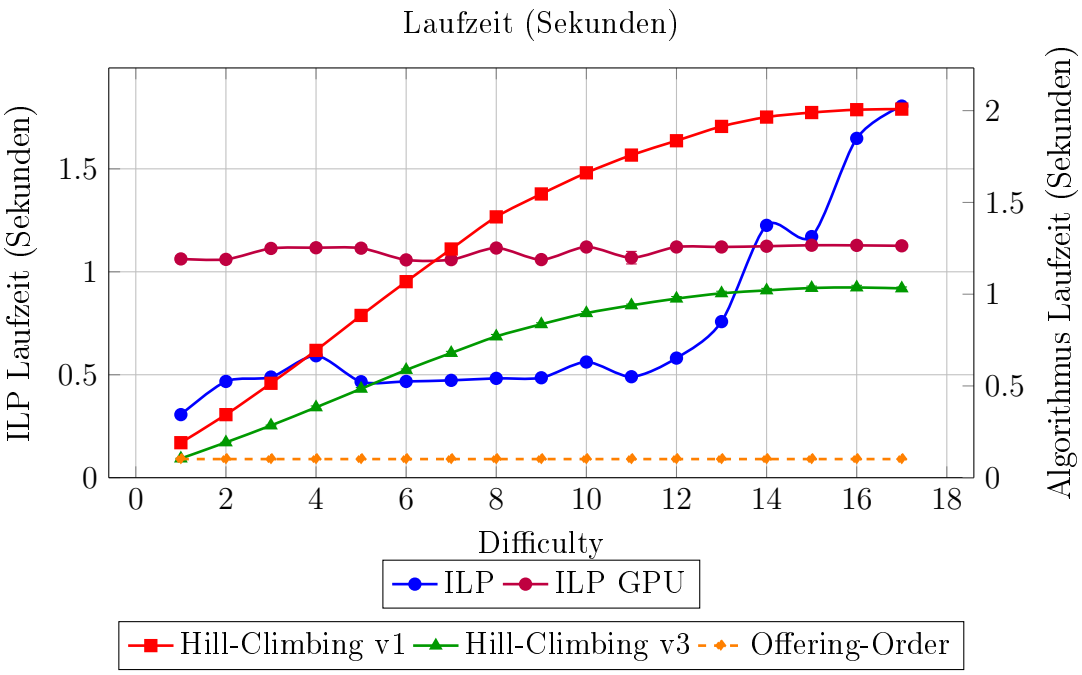


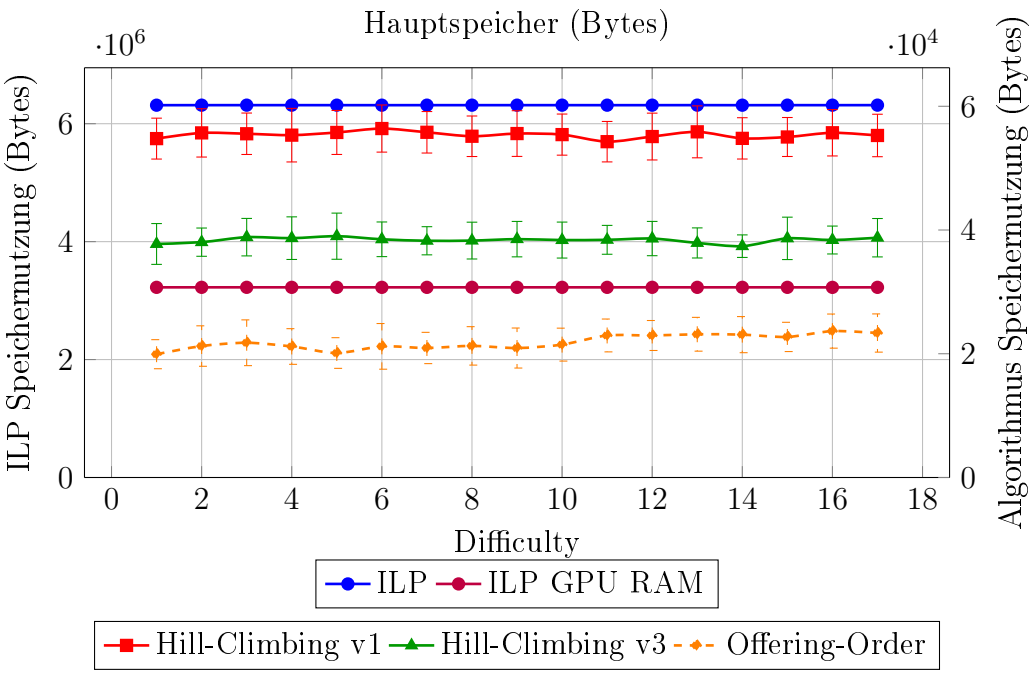

Hauptspeicher (Bytes)
$\cdot 10^7$
$\cdot 10^4$
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
0
0.2
0.4
0.6
0.8
1
0
1
2
3
4
1
2
3
4
5
6
7
8
9
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order

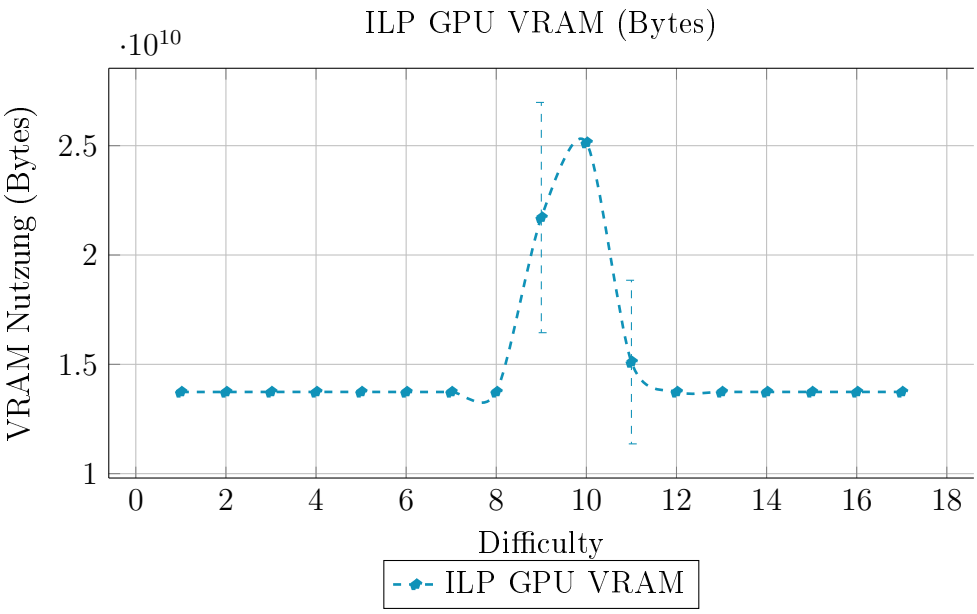

ILP GPU VRAM (Bytes)
$\cdot 10^{10}$
VRAM Nutzung (Bytes)
1.37
1.37
1.37
1.37
1.37
1.37
1.37
1
2
3
4
5
6
7
8
9
Difficulty
ILP GPU VRAM

6.9 Szenario 9: Studierende dürfen nicht mehr als drei Kurse pro Tag besuchen

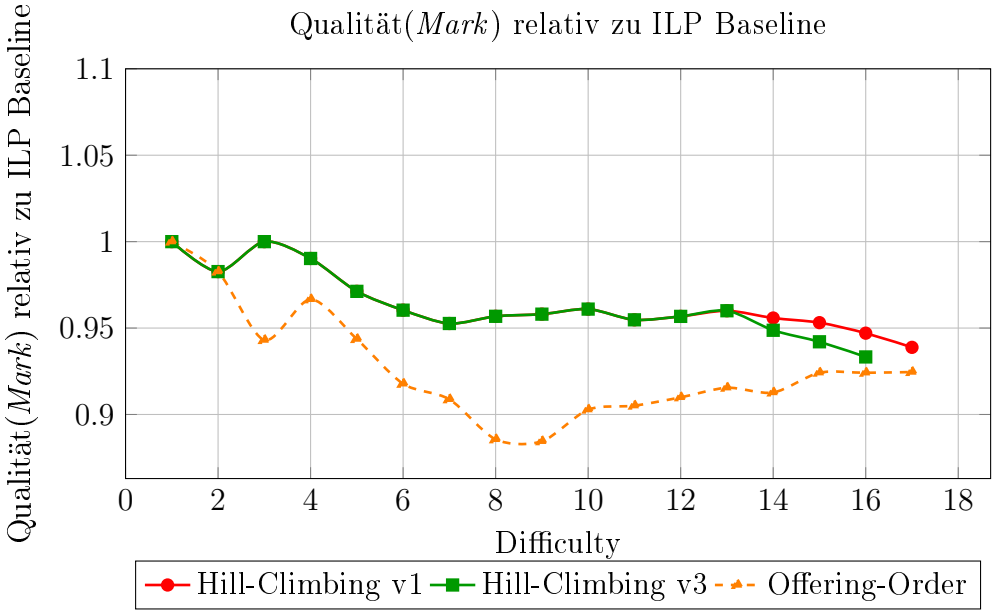


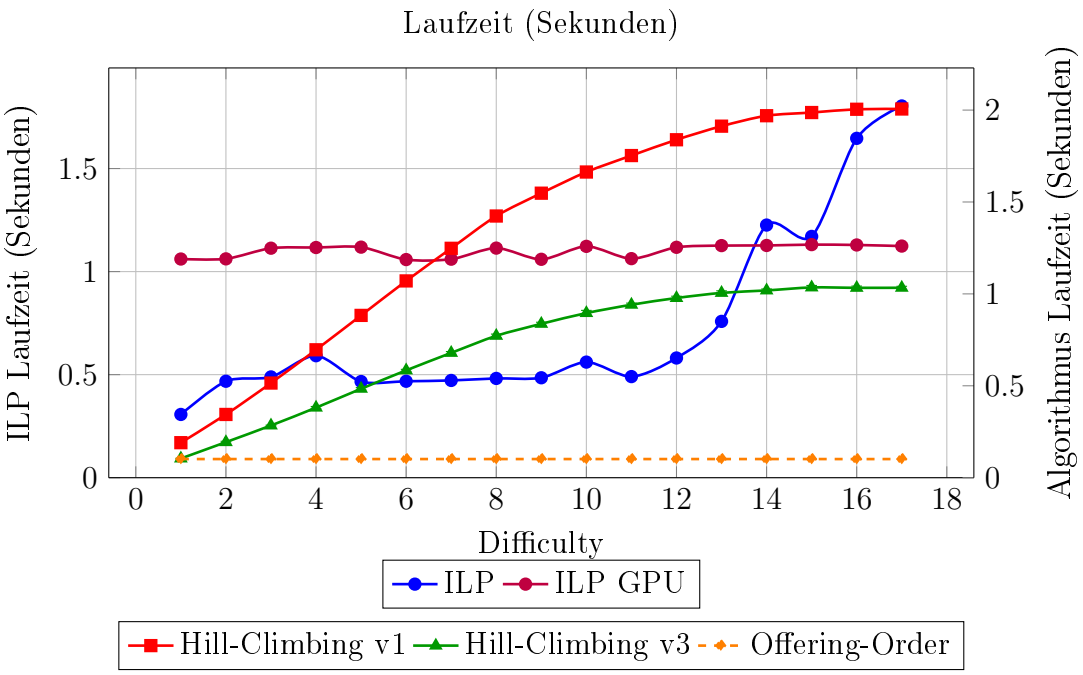


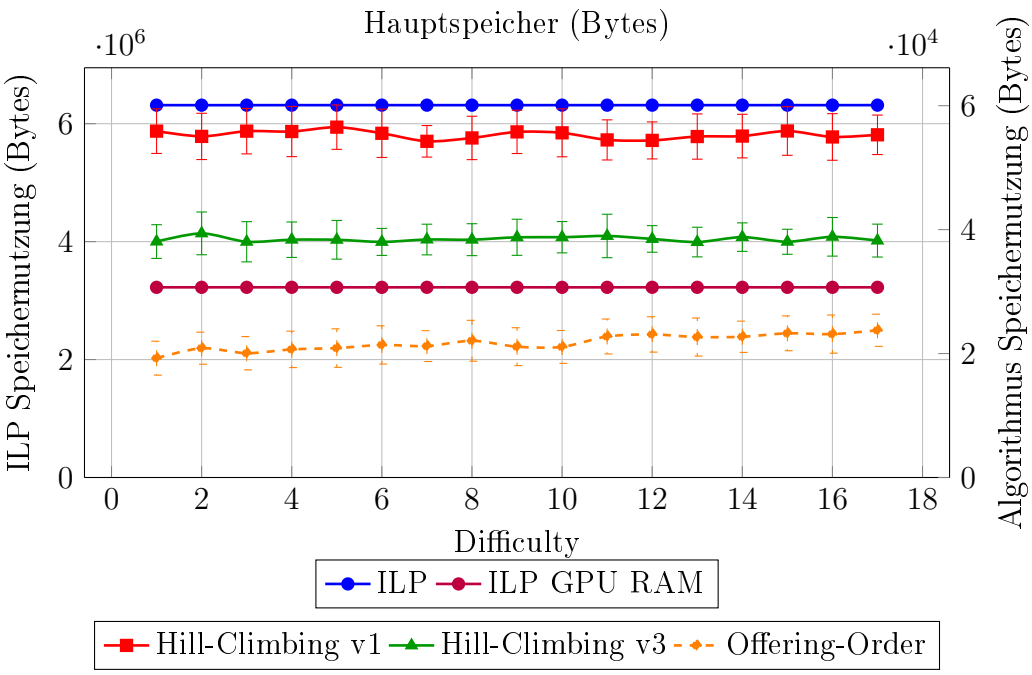

Hauptspeicher (Bytes)
$\cdot 10^6$
$\cdot 10^4$
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
0
2
4
6
0
2
4
6
8
10
12
14
16
18
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order

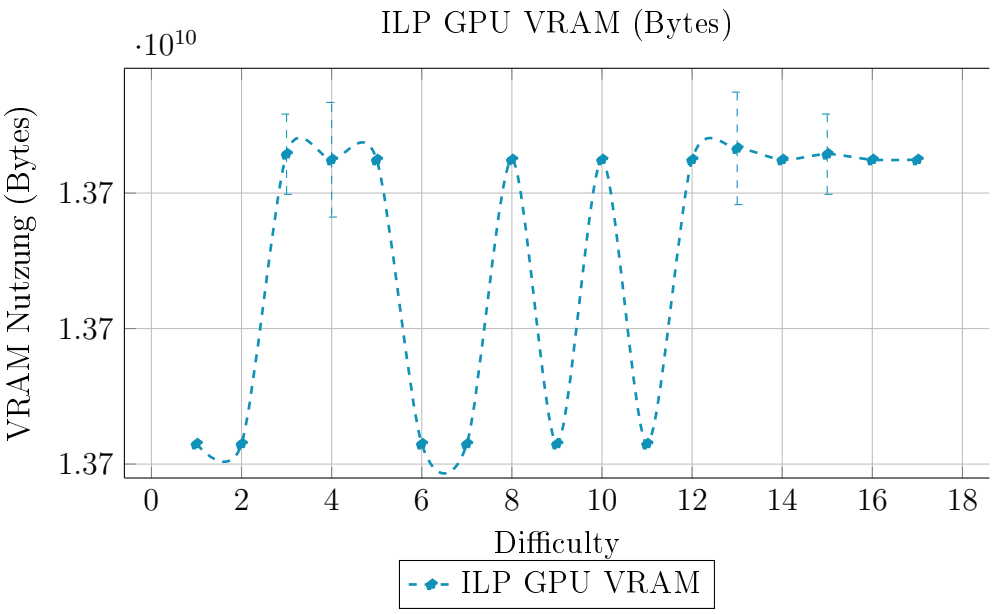

ILP GPU VRAM (Bytes)
$\cdot 10^{10}$
VRAM Nutzung (Bytes)
1
1.5
2
2.5
0
2
4
6
8
10
12
14
16
18
Difficulty
ILP GPU VRAM

6.10 Szenario 10: Alle Kurse können frei von Constraints gewählt werden

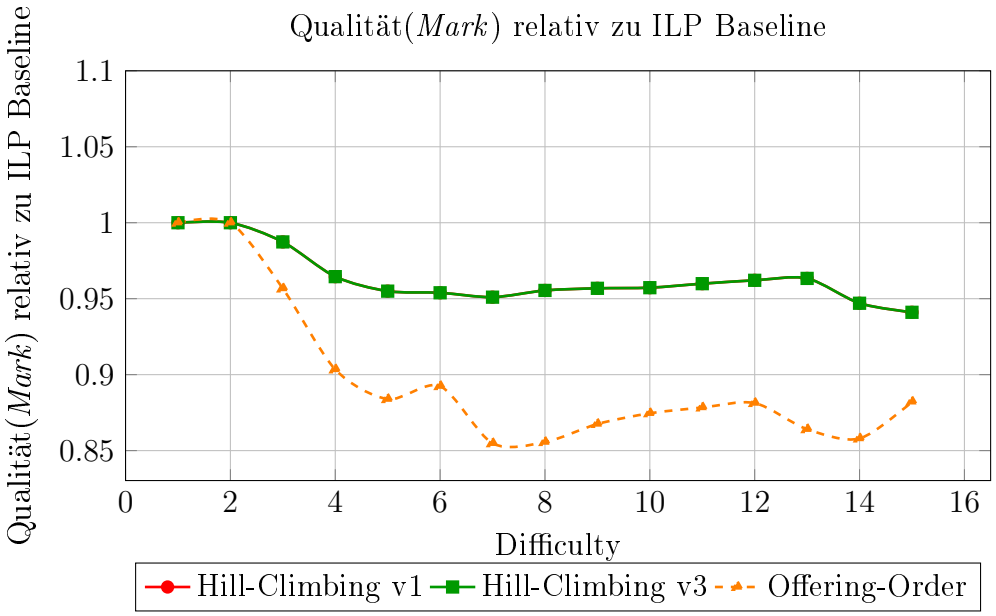


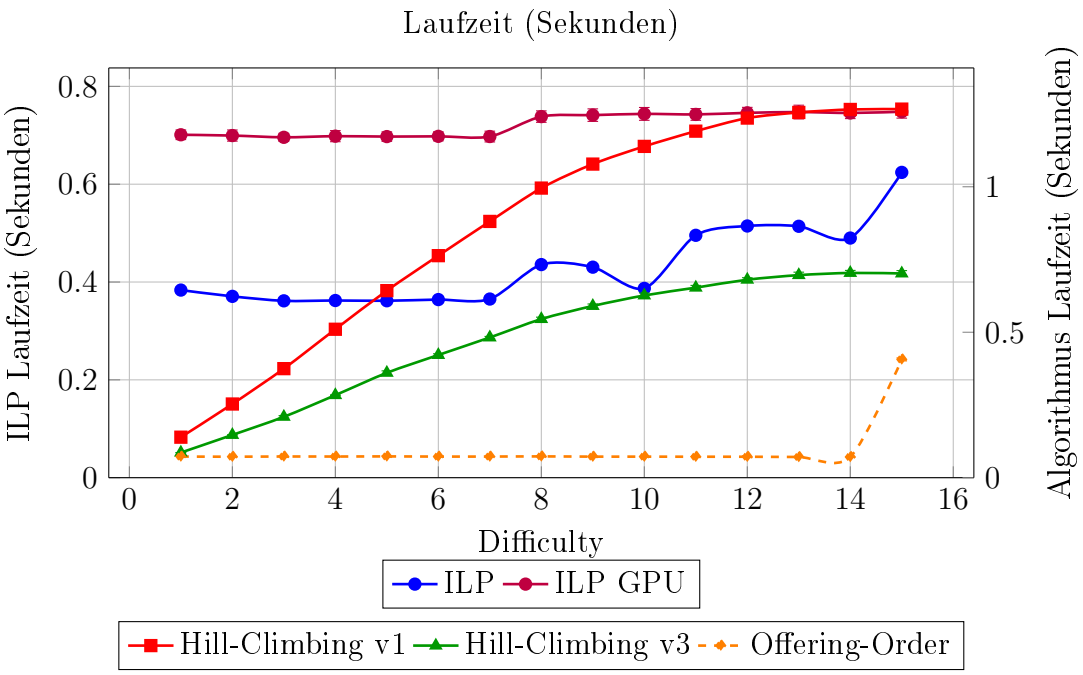


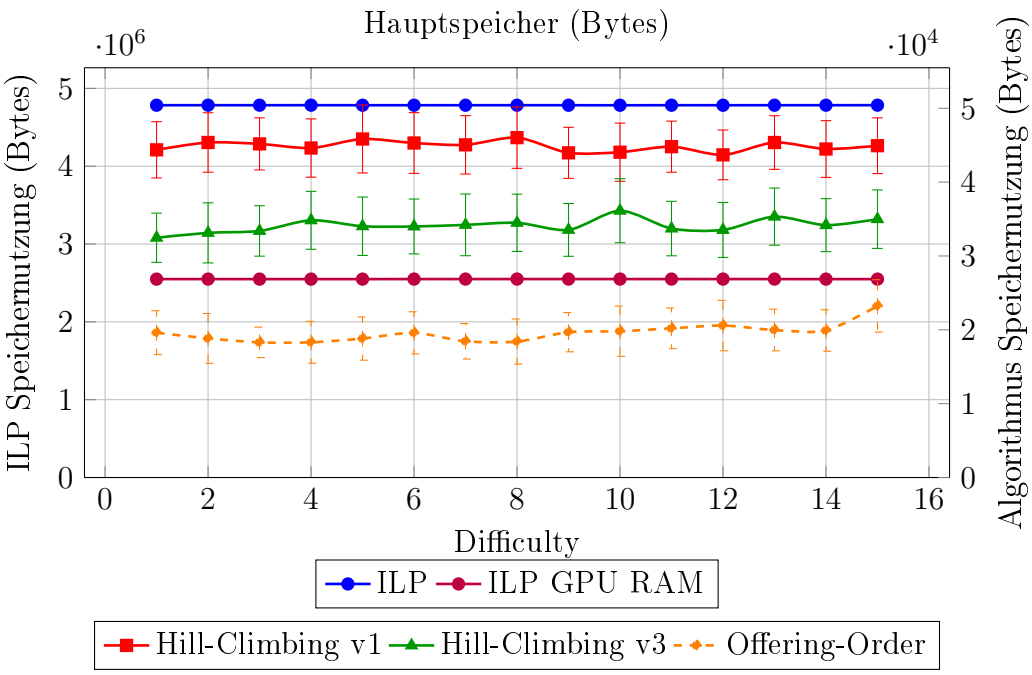

Hauptspeicher (Bytes)
$\cdot 10^6$
$\cdot 10^4$
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
0
2
4
6
0
2
4
6
8
10
12
14
16
18
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order

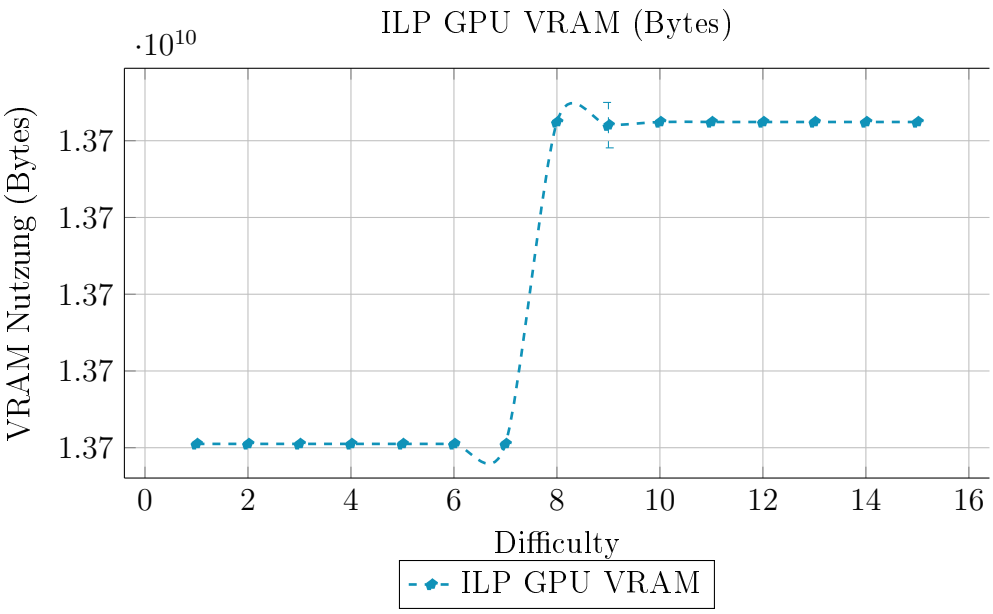

ILP GPU VRAM (Bytes)
$\cdot 10^{10}$
VRAM Nutzung (Bytes)
1.37
1.37
1.37
0
2
4
6
8
10
12
14
16
18
Difficulty
ILP GPU VRAM

6.11 Szenario 11: 25%- 75% aller Kurse können nicht belegt werden.

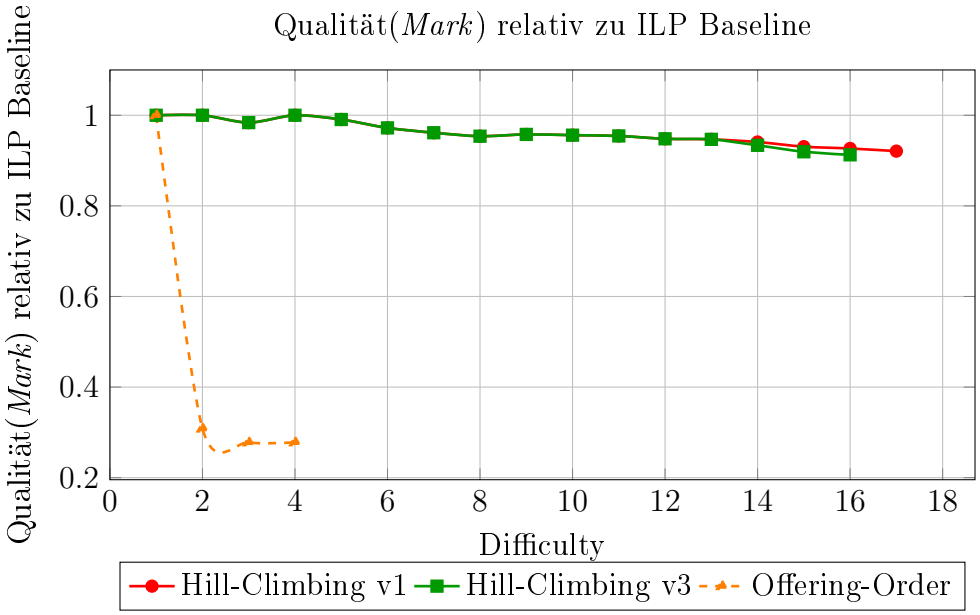


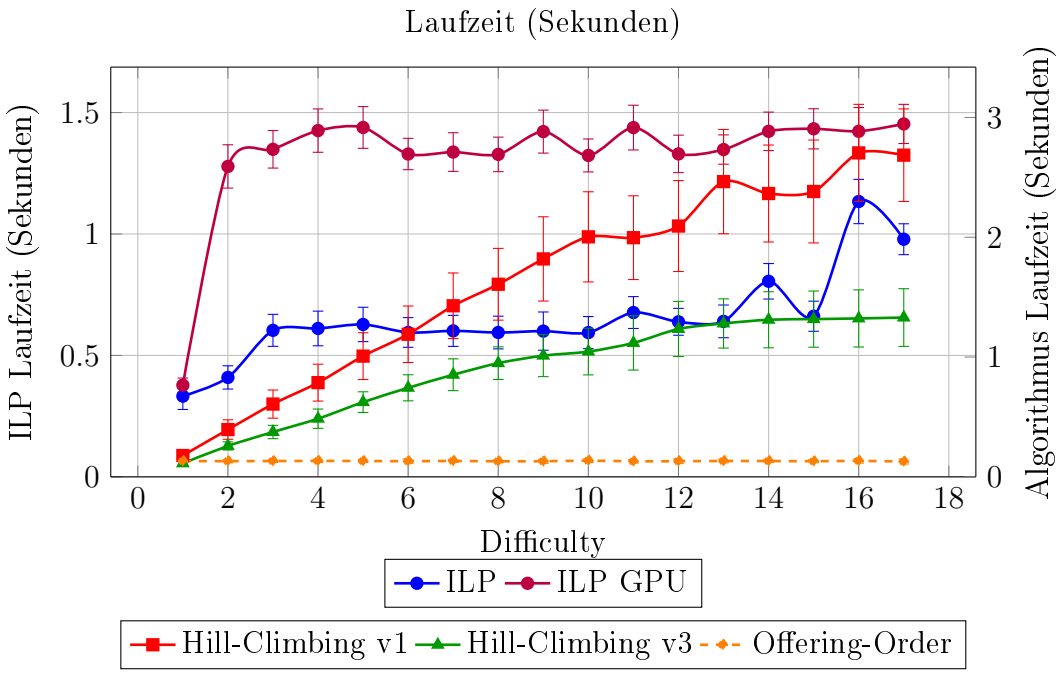


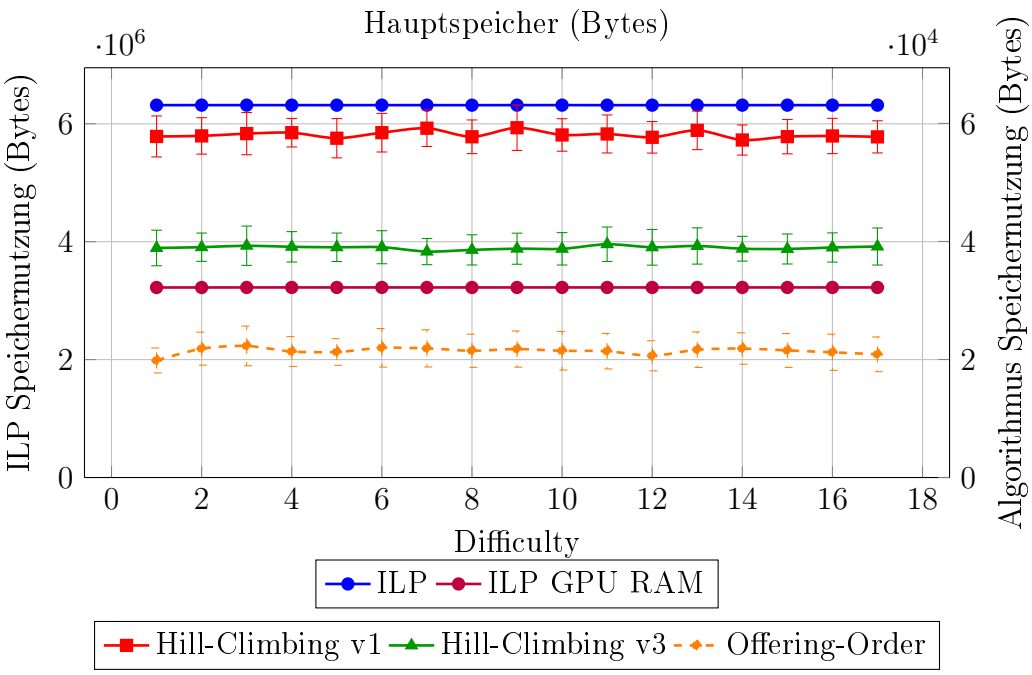

Hauptspeicher (Bytes)
$\cdot 10^6$
$\cdot 10^4$
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
0
1
2
3
4
5
0
2
4
6
8
10
12
14
16
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order

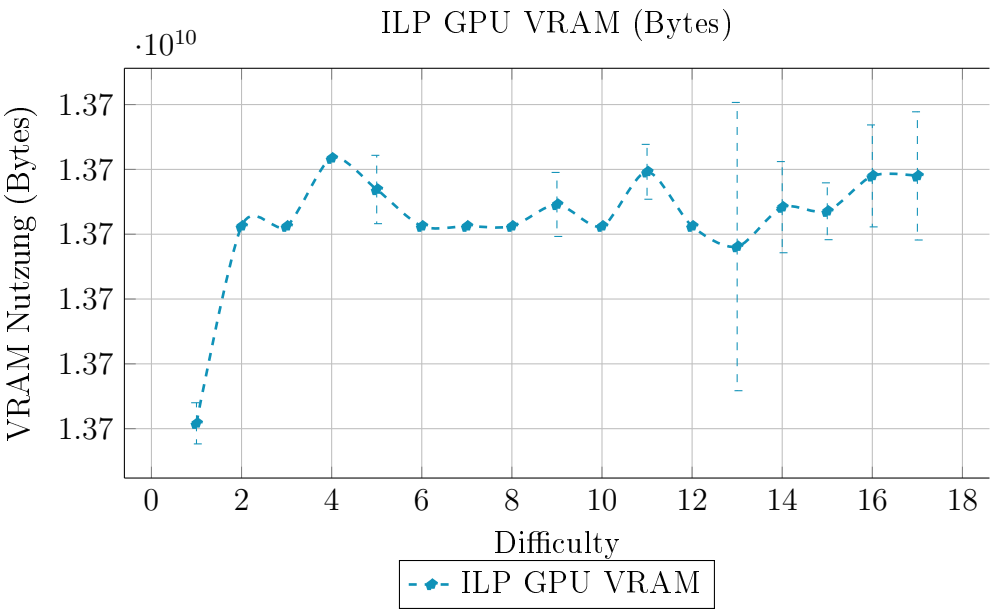

ILP GPU VRAM (Bytes)
$\cdot 10^{10}$
VRAM Nutzung (Bytes)
1.37
1.37
1.37
1.37
1.37
0
2
4
6
8
10
12
14
16
Difficulty
ILP GPU VRAM

6.12 Szenario 12: 1 - 7 Kurse sind vorgegeben

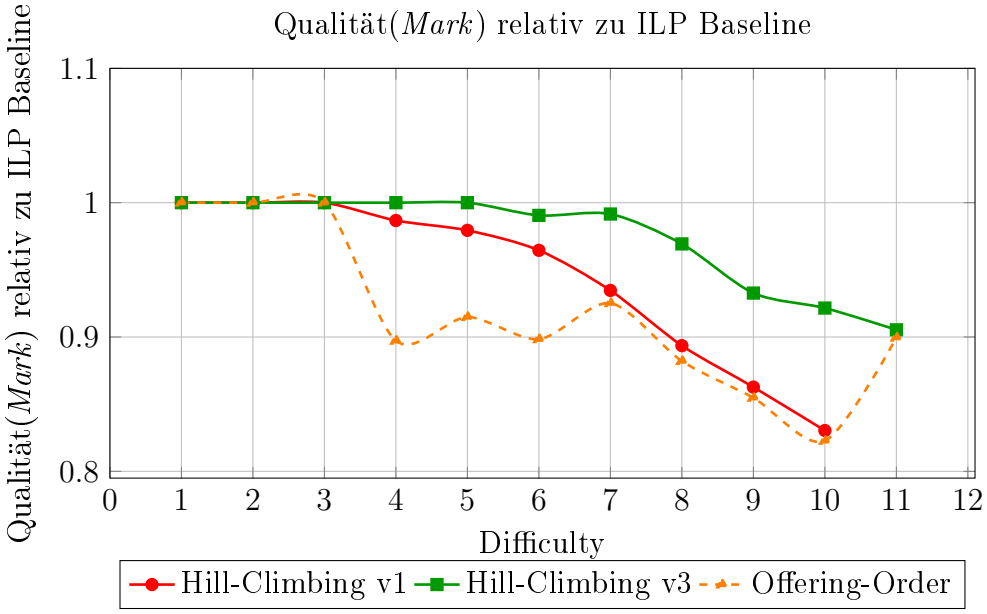


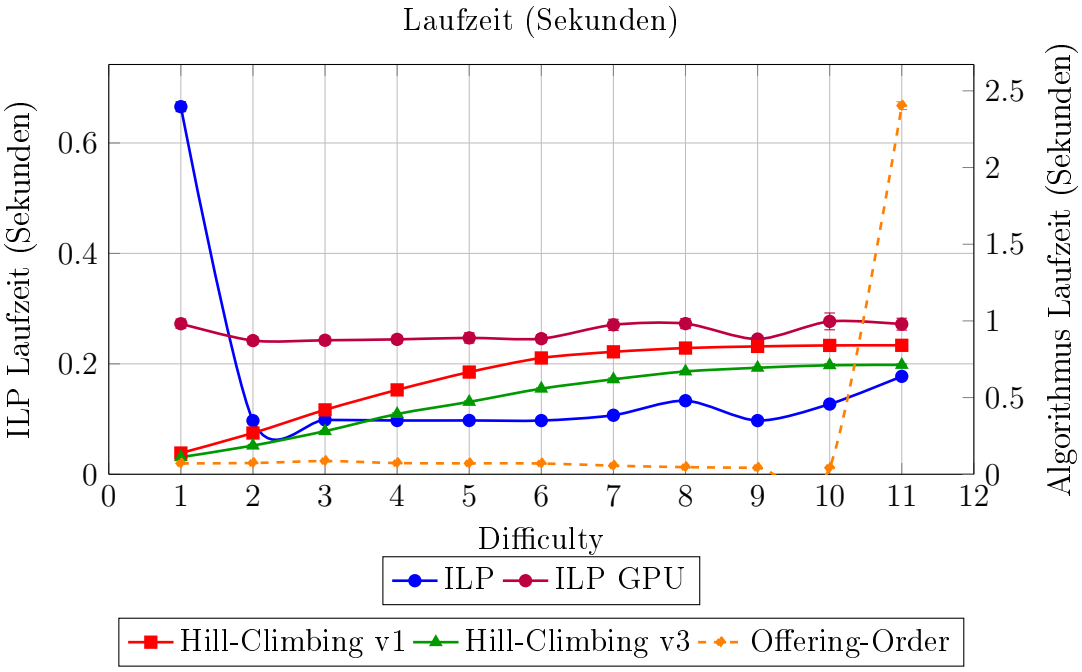


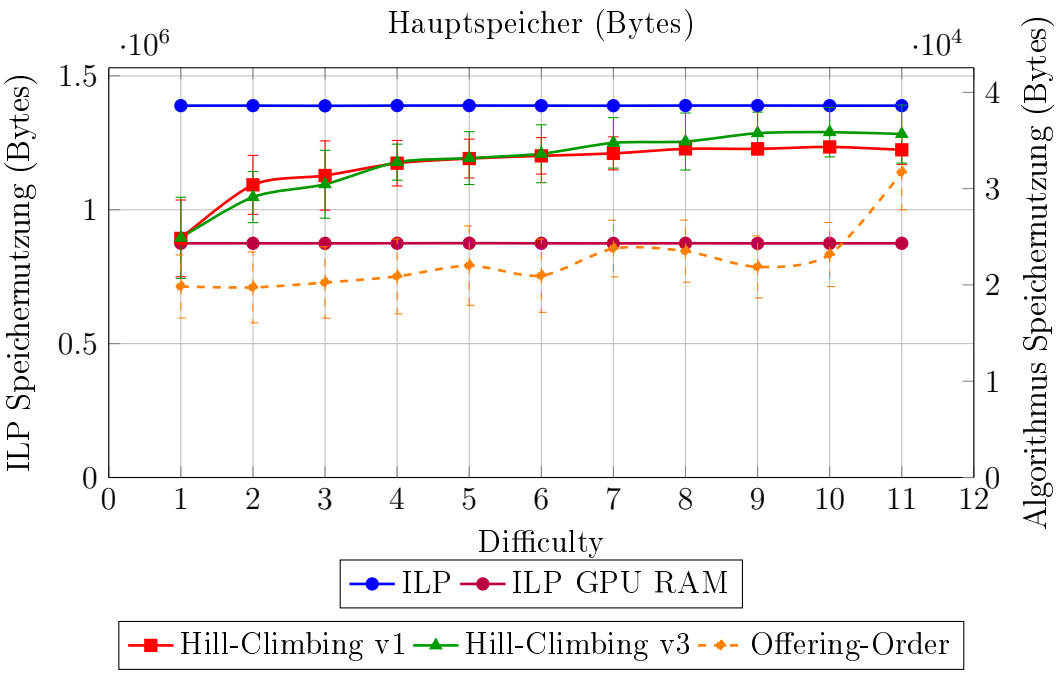

Hauptspeicher (Bytes)
$\cdot 10^6$
$\cdot 10^4$
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
0
2
4
6
0
2
4
6
8
10
12
14
16
18
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order

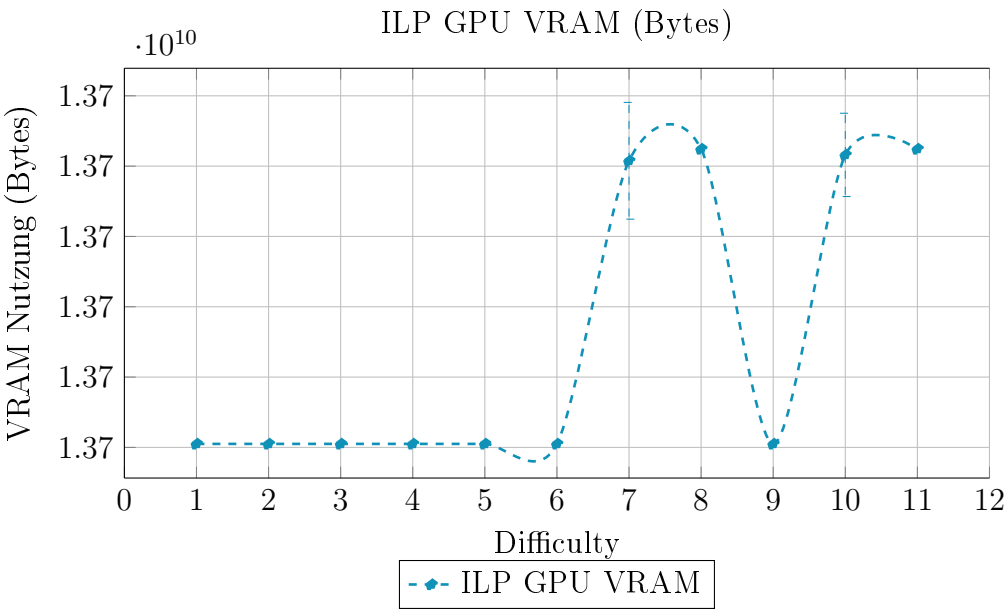

ILP GPU VRAM (Bytes)
$\cdot 10^{10}$
VRAM Nutzung (Bytes)
1.37
1.37
1.37
1.37
1.37
1.37
0
2
4
6
8
10
12
14
16
18
Difficulty
ILP GPU VRAM

6.13 Szenario 13: Keine Kurse zwischem 6:00 morgens und 13:00

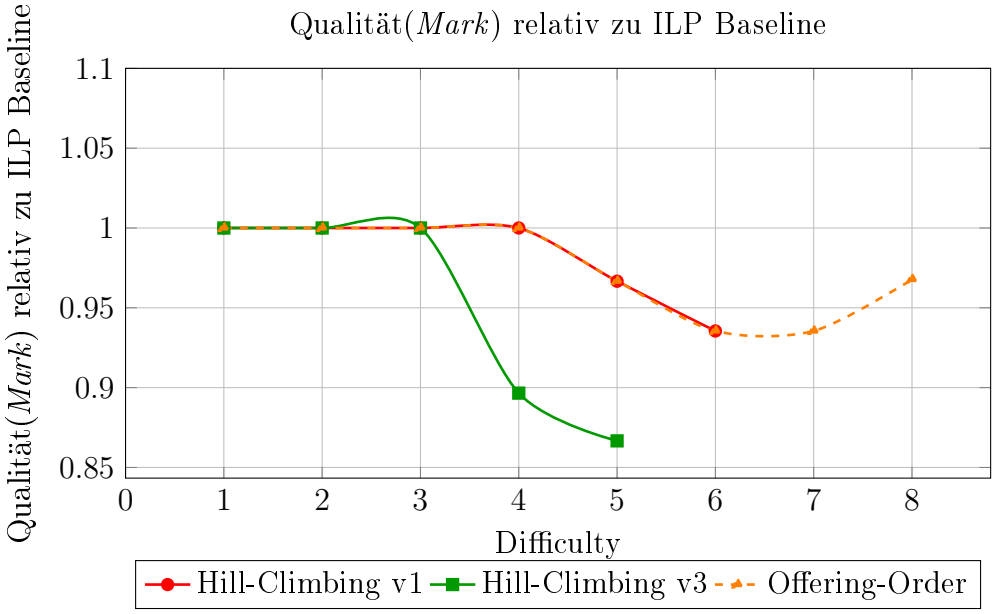


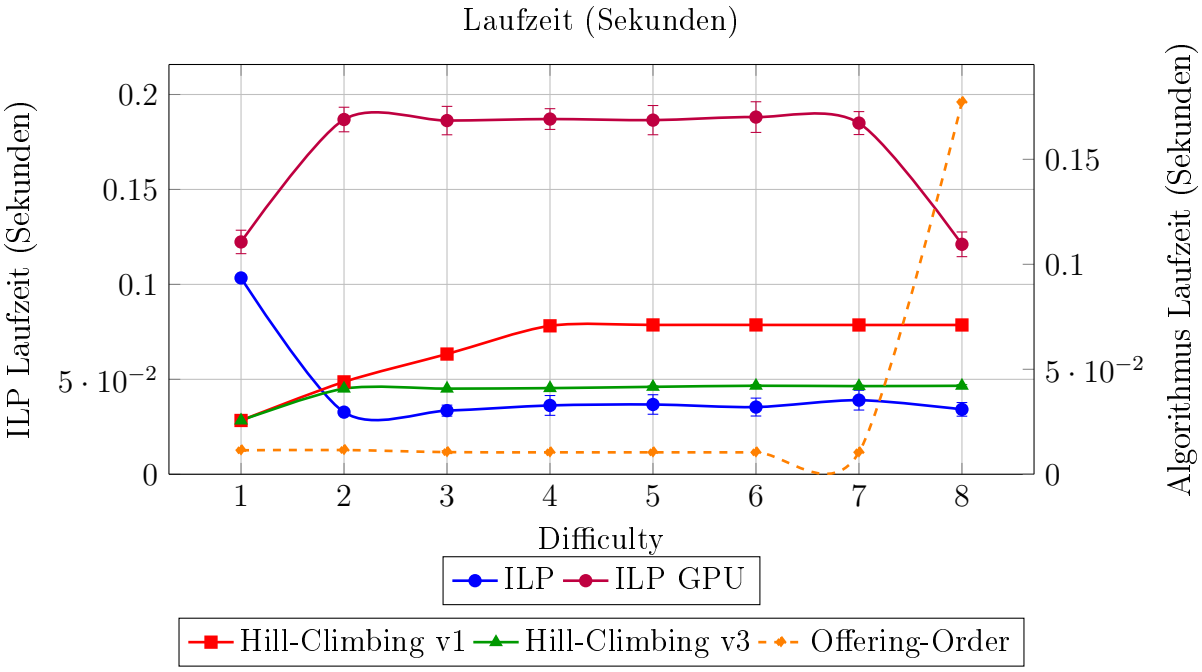


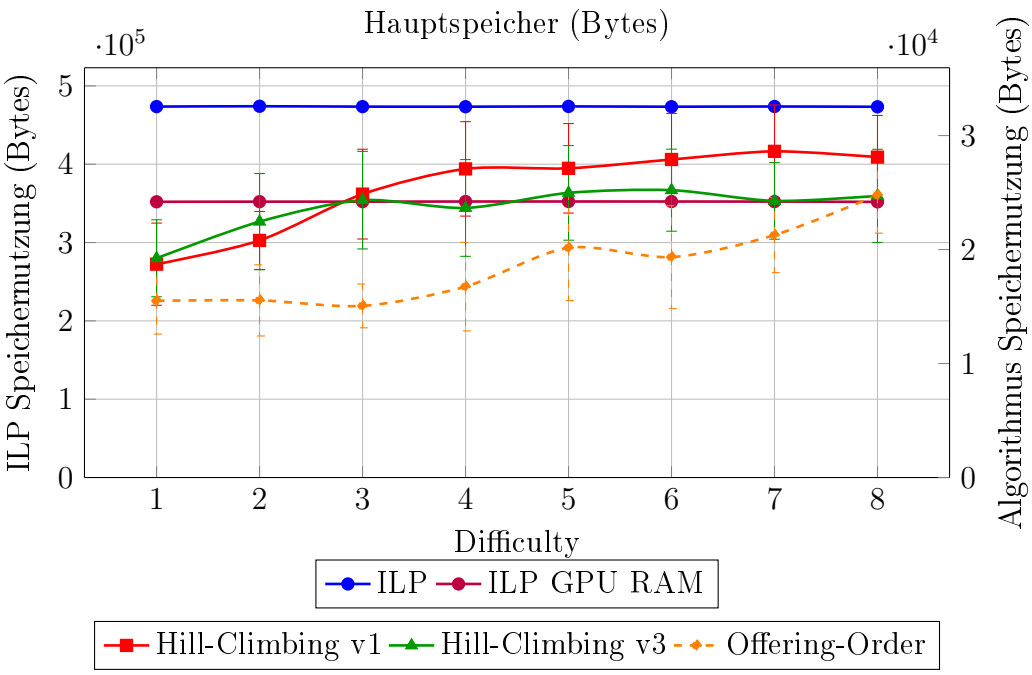

Hauptspeicher (Bytes)
·10^6
·10^4
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
1.5
1
0.5
0
4
3
2
1
0
0 1 2 3 4 5 6 7 8 9 10 11 12
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order

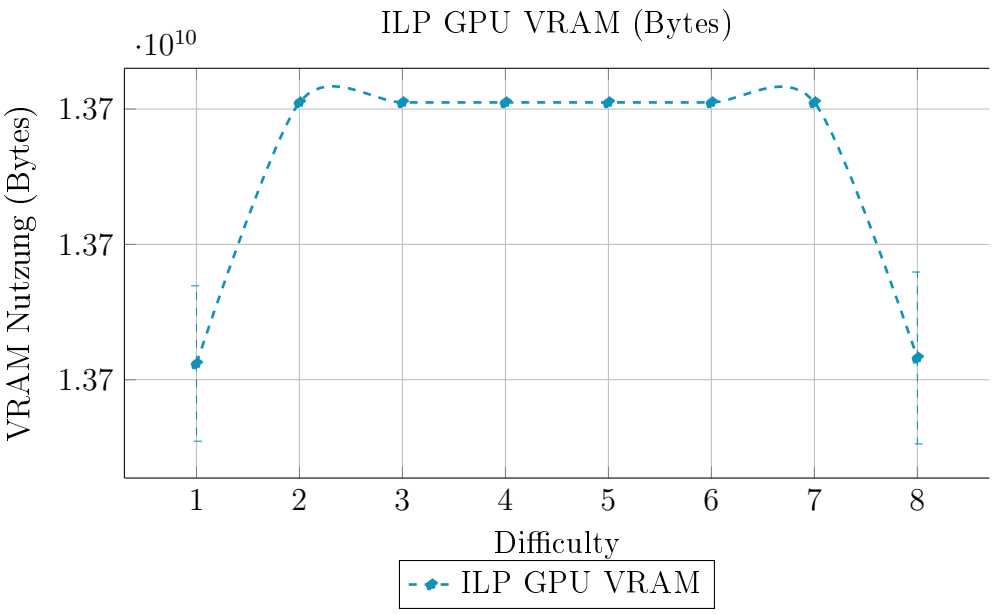

ILP GPU VRAM (Bytes)
·10^10
VRAM Nutzung (Bytes)
1.37
1.37
1.37
1.37
1.37
1.37
0 1 2 3 4 5 6 7 8 9 10 11 12
Difficulty
ILP GPU VRAM

6.14 Szenario 14: Keine Kurse Montags, Dienstags und Mittwochs






Hauptspeicher (Bytes)
$\cdot 10^5$
$\cdot 10^4$
ILP Speichernutzung (Bytes)
Algorithmus Speichernutzung (Bytes)
0
1
2
3
4
5
0
1
2
3
1
2
3
4
5
6
7
8
Difficulty
ILP
ILP GPU RAM
Hill-Climbing v1
Hill-Climbing v3
Offering-Order


ILP GPU VRAM (Bytes)
$\cdot 10^{10}$
VRAM Nutzung (Bytes)
1.37
1.37
1.37
1
2
3
4
5
6
7
8
Difficulty
ILP GPU VRAM

7 Diskussion

7.1 Ergebnisse in der Ursprungsarbeit

Feldman und Golumbic [14] verglichen in ihrer Arbeit einen Offering-Order-Algorithmus mit zwei Hill-Climbing-Algorithmen. Sie stellten fest, dass der Offering-Order-Algorithmus (insbesondere in der "Update"-Variante, in der die *Mark* nach jeder Auswahl des nächsten Kurses neu berechnet wurde) die Hill-Climbing-Ansätze (getestet als Version 2 und 3) in der Lösungsqualität häufig übertrifft oder zumindest gleichwertige Ergebnisse liefert. Hinsichtlich der Laufzeit weist Offering-Order eine kürzere Laufzeit als die Hill-Climbing-Implementierungen vor. Gemessen wurde die Lösungsqualität als Prozentsatz relativ zum Optimum (welches mittels Brute-Force ermittelt wurde). Die Laufzeit wurde in Sekunden gemessen.

7.2 Vergleich mit der Ursprungsarbeit

Die Ergebnisse dieser Bachelorarbeit zeigen sowohl Bestätigungen als auch interessante Abweichungen zu den Erkenntnissen von Feldman und Golumbic [14].

7.2.1 Lösungsqualität (Score)

Ein wesentlicher Unterschied zeigt sich in der Lösungsqualität. Feldman und Golumbic stellten fest, dass der Offering-Order-Algorithmus (insbesondere in der "Update"-Variante) den Hill-Climbing-Ansatz (getestet als Version 2 und 3) häufig übertrifft oder zumindest gleichwertige Ergebnisse liefert, besonders bei der Berücksichtigung von Constraints. In den vorliegenden Ergebnissen zeigt sich aber ein anderes Bild: Die implementierten Hill-Climbing-Algorithmen (v1 und v3) erzielten in den meisten Szenarien (z. B. Szenario 1, 2, 4) durchgehend höhere Scores als der Offering-Order-Algorithmus. Der Offering-Order-Ansatz performte in dieser Arbeit fast durchgehend schlechter als die Hill-Climbing-Varianten.

Dieser scheinbare Widerspruch lässt sich erklären: In dieser Arbeit wurde die Variante "Offering-Order - Fixed" implementiert. Ein Blick auf die Daten von Feldman und Golumbic zeigt, dass auch dort die "Fixed"-Variante deutlich volatiler und oft schlechter abschnitt als die "Update"-Variante. Dass der Offering-Order-Algorithmus in dieser Arbeit besonders schlecht abschnitt, wenn bereits Kurse vorgegeben waren (Szenario 12), zeigt die Schwäche der statischen Sortierung: Sobald der Suchraum durch fixe Kurse eingeschränkt ist, "verläuft" sich Offering-Order in lokalen Optima, die eine niedrige *Mark*

haben oder die Mindestanzahl an gewählten *Offerings* nicht erfüllt.

Interessant ist außerdem der Vergleich der Hill-Climbing-Varianten. Während angenommen werden könnte, dass Version 1 (größerer Suchraum) immer bessere Ergebnisse liefert als Version 3 (reduzierter Suchraum), zeigte sich in Szenario 1 ("Keine Kurse an Montagen"), dass Hill-Climbing v3 teilweise bessere Ergebnisse lieferte als v1. Dies deutet darauf hin, dass eine Einschränkung des Suchraums in v3 nützlich sein kann und den Algorithmus davor bewahrt, sich in seltenen lokalen Optima zu verfangen, die in v3 vorzeitig eliminiert werden, aber in v1 aufgrund der breiteren Suche gewählt werden.

7.2.2 Laufzeit und Performance

Hinsichtlich der Laufzeit decken sich die Ergebnisse weitgehend mit der Ursprungsarbeit. Feldman und Golumbic identifizierten Offering-Order als schnellste Methode. Das konnte bestätigt werden; Offering-Order war in allen Messungen schneller als die Hill-Climbing-Ansätze. Auch die Beobachtung, dass Hill-Climbing v1 aufgrund des größeren Suchraums langsamer ist als v3, entspricht den Erwartungen.

Die Ergebnisse hinsichtlich der Laufzeit decken sich mit der Ursprungsarbeit auch unter den veränderten Rahmenbedingungen dieser Replikationsarbeit. Feldman und Golumbic [14] identifizierten Offering-Order als die schnellste Methode unter den verglichenen Algorithmen — dies zeigt sich auch im Kontext des realen WU-Datensatzes: Offering-Order war in allen 14 Szenarien konsistent schneller als beide Hill-Climbing-Varianten. Der Laufzeitvorteil von Offering-Order ist damit nicht auf die spezifischen Daten oder Rahmenbedingungen von 1990 beschränkt, sondern erweist sich als robust gegenüber einem neuen Datensatz. Des Weiteren bestätigt sich die Beobachtung, dass Hill-Climbing v1 aufgrund des größeren Suchraums langsamer ist als v3 — ein Ergebnis, das sich in dieser Arbeit über alle Szenarien hinweg zeigt und die theoretische Erwartung erfüllt.

7.3 Eigene Ergebnisse im erweiterten Kontext

7.3.1 Der Einfluss moderner Solver (ILP)

Ein Aspekt, den die Ursprungsarbeit von 1990 technologisch bedingt nicht untersuchen konnte, ist der Vergleich mit modernen exakten Verfahren. Feldman und Golumbic nutzten Brute-Force (siehe Kapitel 2.3.1) als Baseline (siehe Kapitel 2.3.1). Durch die begrenzte Rechenleistung waren ILP-Solver zur Zeit der Ursprungsarbeit noch keine praktikable Lösung (siehe Kapiel

2.3.2). Die Ergebnisse dieser Arbeit zeigen jedoch, dass moderne ILP-Solver (hier: CBC/Gurobi via Python-MIP) für die betrachteten Problemgrößen extrem effizient sind. Entgegen der Erwartung, dass Heuristiken für diese kombinatorischen Probleme notwendig sind, um — beispielsweise für eine Webanwendung — akzeptable Laufzeiten von maximal 10 Sekunden [21] zu erreichen, war der ILP-Ansatz (CPU) in fast allen Szenarien schneller als die Heuristiken Hill-Climbing und Offering-Order. Lediglich in Szenario 8 ("Mehr als ein Kurs pro Tag") war die Laufzeit von ILP durchgehend über jenen von Offering-Order und Hill-Climbing.

7.3.2 GPU vs. CPU

Der Vergleich zwischen CPU- und GPU-basiertem ILP zeigte zudem, dass die GPU-Implementierung zwar sehr konstante Laufzeiten aufweist, jedoch bei kleineren und mittleren Problemgrößen langsamer ist als die CPU-Variante. Erst bei steigender Komplexität steigt auch die Effizienz der GPU, was die CPU-Variante für typische Probleminstanzen zur bevorzugten Wahl macht.

7.4 Einschränkungen

Die Generalisierbarkeit und Gültigkeit der Ergebnisse unterliegt bestimmten Einschränkungen, die bei der Interpretation berücksichtigt werden müssen.

- **Spezifität des Datensatzes:** Die Evaluation erfolgte ausschließlich auf Basis der Daten der Wirtschaftsuniversität Wien für das Sommersemester 2025. Die Struktur dieses Datensatzes (Dichte der Konflikte, Verteilung der Kurse auf Wochentage) beeinflusst die Performance der Algorithmen. An Universitäten mit anderen Wahlmöglichkeiten könnten die Heuristiken (insbesondere Offering-Order) andere Ergebnisse liefern.

- **Algorithmen-Varianten:** Wie in Abschnitt 6.1 diskutiert, wurde beim Offering-Order-Algorithmus die statische Variante ("Fixed") implementiert. Auf die Implementierung der "Update"-Variante wurde in dieser Arbeit verzichtet. Der Grund hierfür ist derselbe wie für den Verzicht auf Hill-Climbing v2: Es fehlen im vorliegenden Kontext die notwendigen Abhängigkeiten (beispielsweise zwischen Pflicht- und Wahlfachgruppen), auf denen diese Varianten basieren. Da diese Abhängigkeiten für die spezifische Logik der Update- bzw. v2-Varianten essenziell sind, beschränkte sich die Untersuchung auf die Varianten, die ohne diese Abhängigkeiten operieren (Hill-Climbing v1/v3 und Offering-Order Fixed). Eine Implementierung unter Berücksichtigung

solcher Abhängigkeiten könnte zu einer anderen Lösungsqualität und Laufzeit führen.

- **Abhängigkeit von Hardware:** Die Laufzeitmessungen wurden auf einem spezifischen Server der Wirtschaftsuniversität Wien durchgeführt. Während die relativen Unterschiede zwischen den Algorithmen übertragbar sein dürften, sind die absoluten Laufzeiten hardwareabhängig. Insbesondere der Break-Even-Point zwischen CPU- und GPU-ILP hängt von der CUDA-Leistung ab.

- **Zielfunktion:** Die Definition der Zielfunktion und die Gewichtung der Penalties sind subjektiv gewählt. Eine Veränderung dieser Parameter verändert die Suchlandschaft. Heuristiken bleiben je dementsprechend öfter oder seltener in lokalen Optima stecken.

8 Fazit

8.1 Verwandte Arbeiten

8.2 Anwendungsfälle und Weiterentwicklung

Die Ergebnisse dieser Arbeit legen nahe, dass ein ILP-basierter Ansatz zur automatisierten Semesterplanung für WU-Studierende aufgrund von Laufzeit und Lösungsqualität die beste Wahl ist. Der naheliegendste Anwendungsfall ist die Integration der entwickelten Algorithmen in bestehende studentische Tools: Der ÖH-WU Studienplaner [38] bietet bereits eine Weboberfläche zur manuellen Prüfung von Überschneidungsfreiheit. Eine Erweiterung um eine automatisierte Stundenplanoptimierung — auf Basis des in dieser Arbeit implementierten ILP-Ansatzes — würde den Mehrwert des Tools steigern, da Studierende nicht mehr manuell kombinieren müssten, sondern einen nach individuellen Präferenzen optimierten Stundenplan erhalten könnten.

Hinsichtlich einer möglicher Weiterentwicklung bietet das ILP-Modell eine solide Grundlage. Durch die Zielfunktion lassen sich prinzipiell beliebe weitere Präferenzen modellieren — etwa die Bevorzugung bestimmter Lektoren oder die Berücksichtigung von Gehzeiten zwischen den Gebäuden der WU. Weiters wäre denkbar, das Modell auf einen Multi-Student-Schedulung Ansatz zu erweitern, also die simultane Optimierung der Stundenpläne mehrere Studierender unter Berücksichtigung sozialer Präferenzen wie gemeinsamer Vorlesungen. Um den praktischen Nutzen der Arbeit für Studierende unmittelbar erfahrbar zu machen, wäre ein Web- oder Mobile-Interface der logische nächste Schritt.

8.3 Schlussworte

Diese Bachelorarbeit hatte zum Ziel, die Algorithmen von Feldman und Golumbic [14] unter heutigen Bedingungen zu evaluieren und ihre Anwendbarkeit auf einen realen Datensatz der Wirtschaftsuniversität Wien zu überprüfen. Rückblickend lässt sich festhalten, dass dieses Ziel erreicht wurde — und dass die Arbeit dabei sowohl erwartete als auch überraschende Erkenntnisse geliefert hat.

Die Kernaussage von Feldman und Golumbic, dass Offering-Order hinsichtlich der Laufzeit den Hill-Climbing-Ansätzen überlegen ist, konnte bestätigt werden. Ebenso zeigte sich, dass Hill-Climbing in der Lösungsqualität robust abschneidet.

Die vielleicht überraschendste Erkenntnis dieser Arbeit ist jedoch die Effizienz moderner ILP-Solver. Dass ein exaktes Verfahren die heuristischen

Algorithmen in der Laufzeit in fast allen Szenarien übertrifft, war im Vorfeld nicht erwartet worden. Für die Praxis bedeutet das, dass der vermeintliche Trade-off zwischen Lösungsqualität und Geschwindigkeit — der die Motivation für Heuristiken historisch begründet hat — für Probleminstanzen der hier betrachteten Größenordnung nicht mehr zwingend gilt.

Dennoch sollte man die Limitierungen dieser Arbeit beachten. Die Evaluation beschränkt sich auf einen einzigen Datensatz eines einzelnen Semesters, und die implementierten Algorithmusvarianten decken nicht das gesamte Spektrum der Originalarbeit ab. Ob die Ergebnisse auf andere Universitäten oder größere Probleminstanzen übertragbar sind, bleibt eine offene Frage.

References

[1] benchexec. https://github.com/sosy-lab/benchexec.

[2] Robert E Bixby. A brief history of linear and mixed-integer programming computation. In *Optimization Stories*, pages 107–121. European Mathematical Society-EMS-Publishing House GmbH, 2012.

[3] Douglas G. Bonett. Design and Analysis of Replication Studies. *Organizational Research Methods*, 24(3):513–529, 3 2020.

[4] Mark J. Brandt, Hans IJzerman, Ap Dijksterhuis, Frank J. Farach, Jason Geller, Roger Giner-Sorolla, James A. Grange, Marco Perugini, Jeffrey R. Spies, and Anna Van 't Veer. The Replication Recipe: What makes for a convincing replication? *Journal of Experimental Social Psychology*, 50:217–224, 10 2013.

[5] Leonard E. Burman, W. Robert Reed, and James Alm. A Call for Replication Studies. *Public Finance Review*, 38(6):787–793, 10 2010.

[6] Michael W. Carter, Gilbert Laporte, and Sau Yan Lee. Examination Timetabling: Algorithmic Strategies and Applications. *Journal of the Operational Research Society*, 47(3):373–383, 3 1996.

[7] Sara Ceschia, Luca Di Gaspero, and Andrea Schaerf. Educational timetabling: Problems, benchmarks, and state-of-the-art results. *European Journal of Operational Research*, 308(1):1–18, 7 2022.

[8] Mei Ching Chen, San Nah Sze, Say Leng Goh, Nasser R. Sabar, and Graham Kendall. A survey of university course timetabling problem: Perspectives, trends and opportunities. *IEEE Access*, 9:106515–106529, 2021.

[9] I-Chant A. Chiang, Rajiv S. Jhangiani, and Paul C. Price. Reliability and validity of measurement, 10 2015.

[10] François Clautiaux and Ivana Ljubić. Last fifty years of integer linear programming: A focus on recent practical advances. *European Journal of Operational Research*, 324(3):707–731, 11 2024.

[11] Peter Cowling, Graham Kendall, and Naimah Mohd Hussin. A survey and case study of practical examination timetabling problems. *Proceedings of the 4th International Conference on the Practice and Theory of Automated Timetabling (PATAT 2002*, 01 2002.

[12] Vorlesungsverzeichnis der Wirtschaftsuniversität Wien. `https://vvz.wu.ac.at/`.

[13] M. Dostert, A. Politz, and H. Schmitz. A complexity analysis and an algorithmic approach to student sectioning in existing timetables. *Journal of Scheduling*, 19(3):285–293, 4 2015.

[14] R. Feldman and M. C. Golumbic. Optimization Algorithms for Student Scheduling via Constraint Satisfiability. *The Computer Journal*, 33(4):356–364, 4 1990.

[15] Fiona Middleton. Reliability vs Validity in Research | Differences, Types & Examples, 10 2022.

[16] Wan Zuki Azman Wan Muhamad, Farah Adibah Adnan, Zainor Ridzuan Yahya, Ahmad Kadri Junoh, and Mohd Hafiz Zakaria. Solving university course timetabling problems using FET software. *AIP conference proceedings*, 2013:020052, 1 2018.

[17] Ahmad Muklason, Andrew J. Parkes, Ender Özcan, Barry McCollum, and Paul McMullan. Fairness in examination timetabling: Student preferences and extended formulations. *Applied Soft Computing*, 55:302–318, 1 2017.

[18] MyStudyLife. https://mystudylife.com/.

[19] Tomáš Müller and Keith Murray. Comprehensive approach to student sectioning. *Annals of Operations Research*, 181(1):249–269, 3 2010.

[20] Alexandra M. Newman and Martin Weiss. A survey of Linear and Mixed-Integer optimization tutorials. *INFORMS Transactions on Education*, 14(1):26–38, 9 2013.

[21] Jakob Nielsen. Response times: The 3 important limits, 1 2024.

[22] Hochschule Trier Trier University of Applied Sciences. Evaluation zu den Freiräumen im Stundenplan. 1 2024.

[23] National Academy of Sciences. Reproducibility and replicability in science. 5 2019.

[24] Hana Rudová and Keith Murray. *University Course Timetabling with Soft Constraints*. 1 2003.

[25] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten Simple Rules for Reproducible Computational Research. *PLoS Computational Biology*, 9(10):e1003285, 10 2013.

[26] A. Schaerf. A survey of Automated Timetabling. *Artificial Intelligence Review*, 13(2):87–127, 4 1999.

[27] Philipp Scheer. `https://github.com/philippscheer/bachelorarbeit`.

[28] Philipp Scheer. `https://github.com/philippscheer/bachelorarbeit/blob/main/models/ilp.py`.

[29] Philipp Scheer. `https://github.com/philippscheer/bachelorarbeit/blob/main/models/ilp_gpu.py`.

[30] Philipp Scheer. `https://github.com/philippscheer/bachelorarbeit/blob/main/models/hill_climbing_v1.py`.

[31] Philipp Scheer. `https://github.com/philippscheer/bachelorarbeit/blob/main/models/hill_climbing_v3.py`.

[32] Philipp Scheer. `https://github.com/philippscheer/bachelorarbeit/blob/main/models/offering_order.py`.

[33] Philipp Scheer. `https://github.com/philippscheer/bachelorarbeit/blob/main/data/raw/vvz_model.pkl`.

[34] Philipp Scheer. `https://github.com/philippscheer/bachelorarbeit/blob/main/bachelorarbeit/dataset.py`.

[35] Philipp Scheer. `https://github.com/philippscheer/bachelorarbeit/blob/main/results/raw/results.zip`.

[36] Philipp Scheer. `https://github.com/philippscheer/bachelorarbeit/blob/main/models/results_to_csv.py`.

[37] Daniel J. Simons. The value of direct replication. *Perspectives on Psychological Science*, 9(1):76–80, 1 2014.

[38] ÖH-WU Studienplaner. https://studienplaner.oeh-wu.at/.

[39] uAchieve Schedule Builder. https://collegesource.com/degree-planning-tools/uachieve-schedule-builder.

[40] UniTime. `https://www.unitime.org/`.